Arculus Recovery v1.5.0

User Guide and Technical Reference

Offline BIP39/BIP32 recovery, encrypted seed backup, QR export, file-format reference, and operational security procedures.

Primary GUI	Standalone HTML application
Desktop GUI	PySide6 WebEngine wrapper
Desktop Package	Tauri wrapper with native export bridge
CLI	Python derivation and export interface
Generated	June 6, 2026

Arculus Recovery v1.5.0 User Guide

Purpose

Arculus Recovery is an offline BIP39/BIP32 recovery and key-derivation tool for users who need to verify or recover wallet addresses after an Arculus hardware wallet failure, backup audit, or controlled migration. The application derives addresses and private keys locally from a mnemonic or encrypted .arc backup. It does not require a network connection and does not call remote services.

The standalone HTML application is the canonical graphical interface. The PySide6 Python GUI and the Tauri desktop package render the same HTML application, so the controls and recovery workflow are intentionally identical across those three GUI surfaces. The Python CLI exposes the same derivation engine for scripted or text-only recovery sessions.

Application Editions

```
Standalone HTML:
  Open Arculus_Recovery.html directly in a browser on an offline machine. The
  HTML file contains its PDF export library inline, so it does not need npm,
  Python, Tauri, vendor files, cdnjs, or any network resource at runtime.

Python GUI:
  Run "python Arculus_Recovery.py --gui" after installing the GUI dependency.
  The PySide6 shell opens the packaged HTML application in Qt WebEngine. This
  mode is useful when a native desktop window is preferred but the same browser
  workflow should be preserved.

Python CLI:
  Run "python Arculus_Recovery.py" with --mnemonic, --derivation, --coin,
  --script-type, --count, and --output-format flags. CLI output is written to
  stdout, making it suitable for controlled redirection to a RAM-backed file.
  The CLI supports JSON, CSV, and TXT output.

Tauri desktop package:
  The Tauri build packages the same HTML application as a native desktop app.
  Browser-style exports are routed through an injected WebView bridge named
  window.arculusTauriSaveExport and the Rust save_export command. The bridge is
  governed by the Tauri v2 files src-tauri/capabilities/default.json and
  src-tauri/permissions/export.toml. Validate the final packaged build on the
  target operating system before relying on it for an operational recovery.
```

Before You Begin

Use Arculus Recovery only on a trusted offline machine. Disconnect Ethernet, disable wireless networking and Bluetooth at the hardware level, and prefer a live ephemeral operating system that writes nothing to persistent storage. The network indicator in the application is a convenience check; physical confirmation of disconnection is authoritative.

Verify the release hash before using the tool with real seed material. Expected hashes are maintained in README.md. If any file hash is unexpected, stop and obtain a trusted copy before continuing.

Have the following inputs ready before the session begins:

```
  - The 12-word or 24-word BIP39 mnemonic, or a .arc encrypted seed file.
  - The BIP39 passphrase if the wallet used one. Leave it empty if no
    passphrase was used.
  - The target coin.
  - The derivation path and script type if they are known.
  - At least one known address or a recorded root fingerprint for verification.
```

Do not continue past derivation until the output has been verified against a known address or root fingerprint. A wrong passphrase, path, script type, or mnemonic can produce valid-looking but unrelated keys.

Quick Recovery Workflow

```
1. Open the standalone HTML app, Python GUI, or Tauri desktop app on the
   offline machine.

2. Select 12 words or 24 words, then enter the mnemonic in the text area or
   numbered word fields. Alternatively, click Import Seed and decrypt a .arc
   backup.

3. Click Validate Mnemonic. Resolve any word-list or checksum errors before
   deriving addresses.
```

4. Enter the BIP39 passphrase only if the wallet used one. The .arc decryption password and the BIP39 passphrase are different secrets.

5. Choose the coin, script type, derivation path, and address count. Use Auto script type when testing standard BIP44, BIP49, BIP84, or BIP86 paths.

6. Click Derive Keys + Addresses. Review the Table View first, then use Raw JSON or Extended Keys when those outputs are needed.

7. Verify the root fingerprint and at least one derived address before exporting or sweeping funds.

8. Export only the format you need. JSON, CSV, TXT, and PDF exports contain private key material and must be handled as funds-controlling secrets.

9. Click Clear All at the end of the session, close the app, and power off the live environment.

Main Controls

Mnemonic entry:
Paste a full mnemonic into the main text area or enter words into the numbered word grid. The 12/24 selector controls which word slots are active. Generate Random Seed creates a test or backup mnemonic with browser cryptographic randomness, then hides it until Show Seed is held.

Passphrase:
Optional BIP39 passphrase input. This field affects the BIP39 seed and root fingerprint. It is not used to decrypt .arc files.

Coin:
Select Bitcoin, Litecoin, Dogecoin, Ethereum / ERC-20, or XRP. Changing the coin updates the default derivation behavior.

Derivation path:
Account-level BIP32 path. Arculus-native Bitcoin recovery defaults to m/0'. Standard wallet paths include m/44'/0'/0', m/49'/0'/0', m/84'/0'/0', and m/86'/0'/0'. If m/0' is active, the interface displays an informational reminder that this path differs from BIP44 convention.

Script type:
Controls UTXO address encoding for Bitcoin, Litecoin, and Dogecoin. Auto infers the type from standard BIP purpose numbers when possible. Ethereum and XRP ignore UTXO script type.

Address count and range:
Address Count controls how many receiving and change addresses are derived. Range can target a specific index window such as 0-9 or 25-50.

Output tabs:
Table View provides a scannable row layout with copy controls. Raw JSON shows the complete structured output. Extended Keys isolates account and root extended-key material.

Export controls:
Export writes derived output in JSON, CSV, TXT, or PDF. The HTML app uses browser downloads. The PySide6 GUI uses native save behavior where available. The Tauri app uses its injected native export bridge before falling back to WebView download behavior.

Encrypted seed controls:
Encrypt/Export Seed stores the active mnemonic in an authenticated .arc file. Import Seed decrypts a .arc file and loads the mnemonic into a hidden-seed workflow. Imported seeds remain hidden unless Show Seed is pressed and held.

QR Export:
Opens a modal for generating a QR code from an address. XRP addresses may prompt for a destination tag. Save PNG exports the rendered QR image.

Settings:
Theme selection supports Light, Dark, Dark+, and Terminal. Auto-Derive on Settings Change can re-run derivation automatically after supported settings are changed, but only after a successful derivation already exists.

Supported Default Paths

Coin	Default path	Notes
Bitcoin	m/0'	Arculus-native default
Litecoin	m/84'/2'/0'	Native SegWit default
Dogecoin	m/44'/3'/0'	Legacy BIP44 default
Ethereum	m/44'/60'/0'	Also used by ERC-20 tokens
XRP	m/44'/144'/0'	Destination tags are not derived

When recovering a wallet created by software other than Arculus, change the path to match that wallet's convention before relying on any output.

Export Sensitivity

Treat every export that contains private keys, extended private keys, or a mnemonic as funds-controlling material. The .arc format encrypts only the mnemonic backup. It does not encrypt derived-output JSON, CSV, TXT, or PDF files. Store those files only on media intended to hold private key material.

Printed PDFs are physical key documents. Do not print them on a networked printer. Do not photograph them with a cloud-connected device.

End-of-Session Checklist

- [ ] The BIP39 checksum passed.
- [ ] The root fingerprint or a known address was verified.
- [ ] The coin, derivation path, script type, passphrase, count, and range were checked before export.
- [ ] Any export file was stored according to Category B handling requirements.
- [ ] Clear All was clicked before closing the app.
- [ ] The live or offline machine was powered down after the session.

Interface Guide

The screenshots below show the primary user-facing layout and theme variants. The HTML application is the canonical GUI surface. PySide6 and Tauri package the same interface for desktop use, so the screenshots apply to all GUI editions.

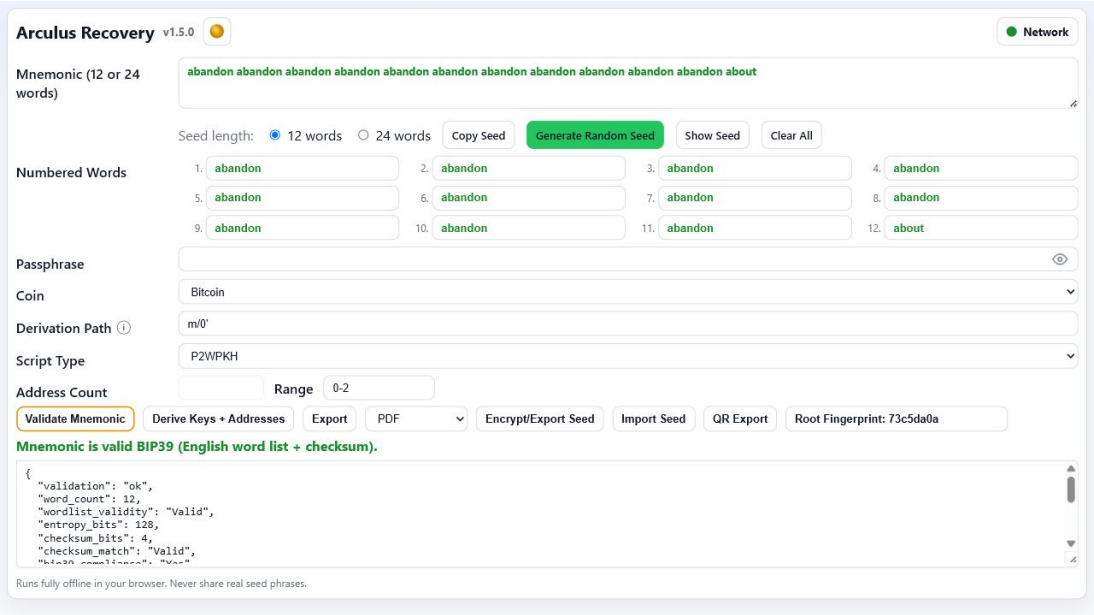

Figure 1. Main recovery workspace with test-vector mnemonic validation controls.

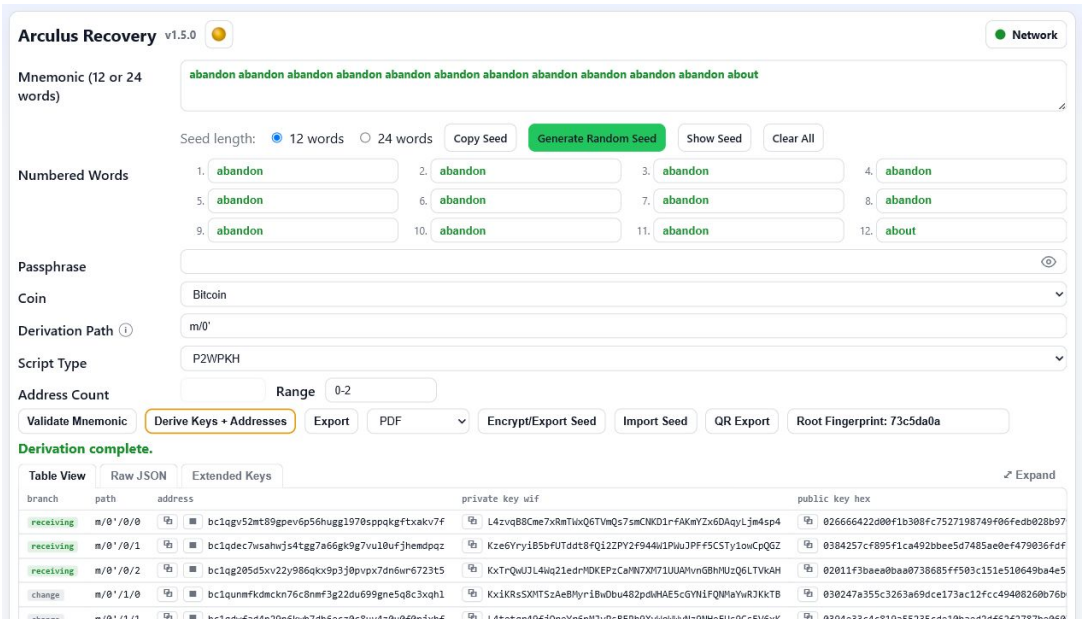

Figure 2. Derived output table after address derivation, with output tabs and export controls.

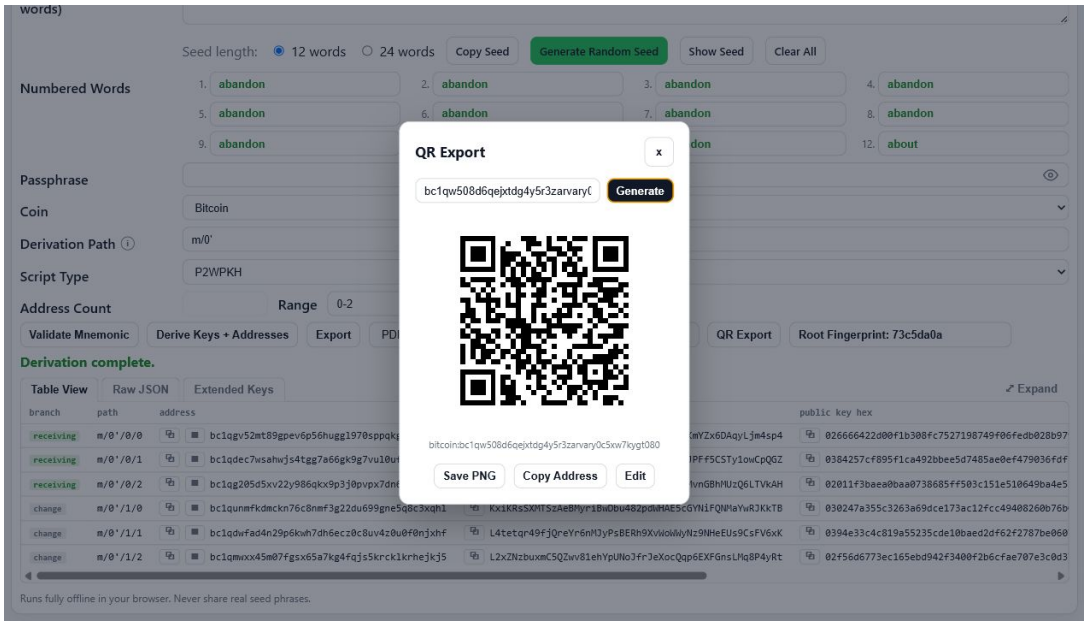

Figure 3. QR Export modal generating an address QR code without external services.

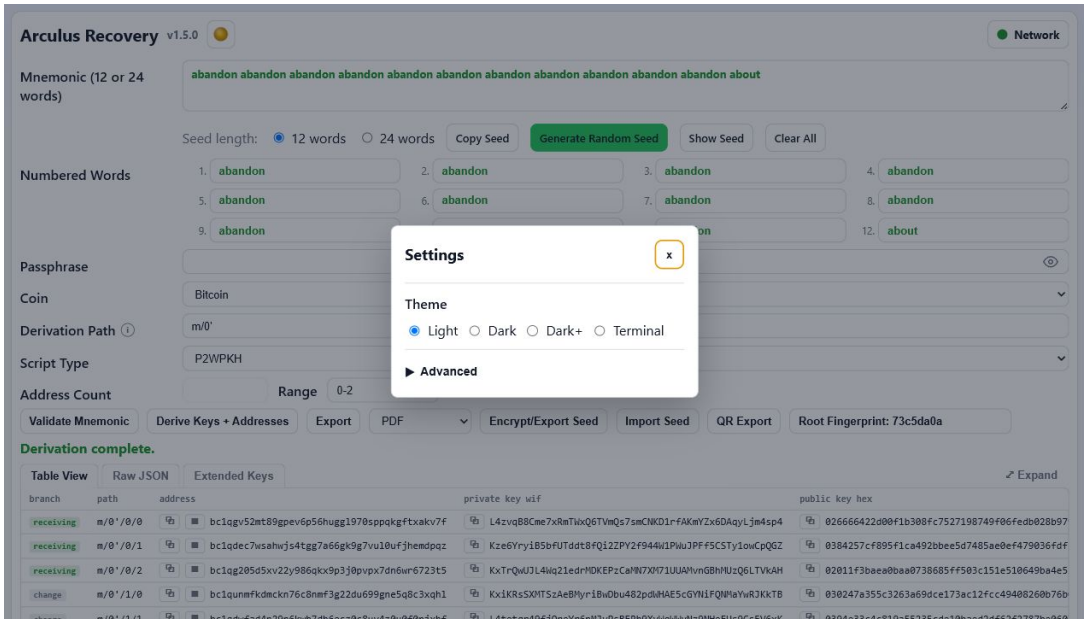

Figure 4. Settings dialog with theme selection and auto-derive preference.

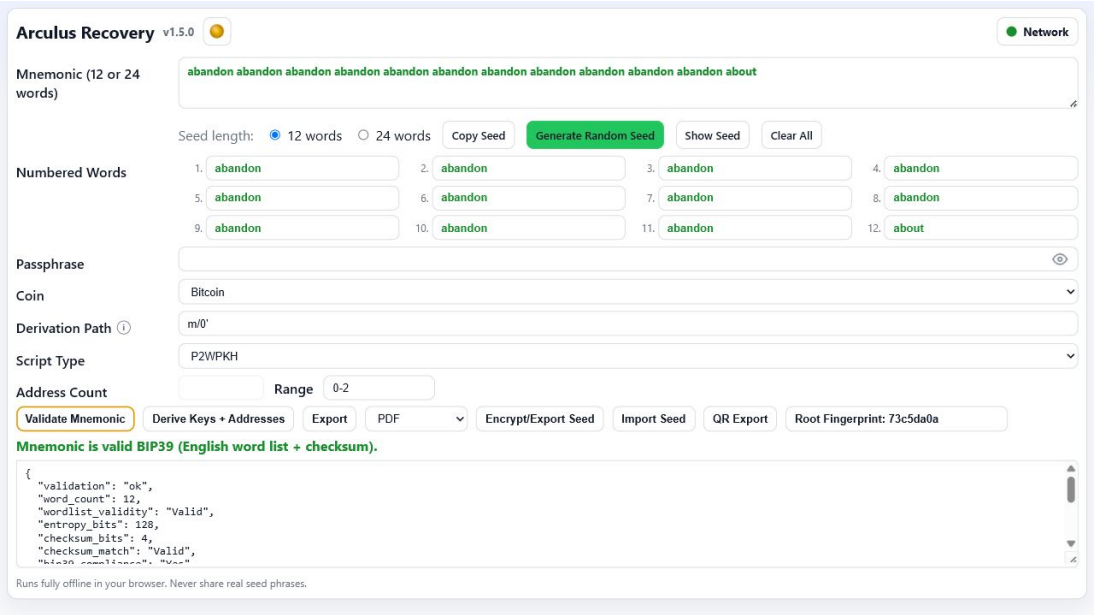

Figure 5. Light theme interface.

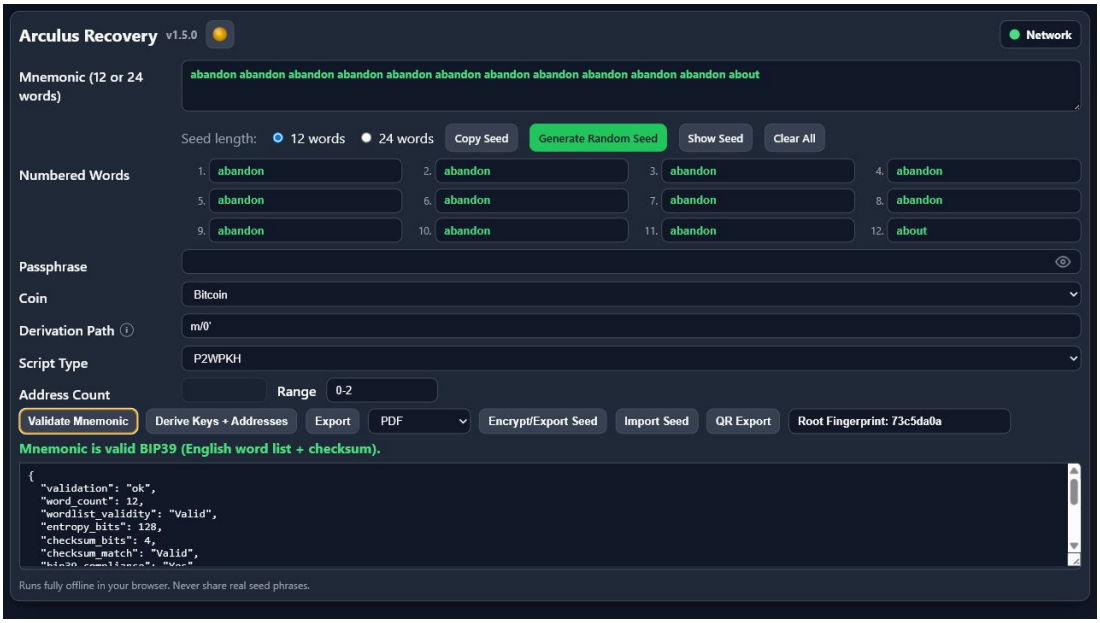

Figure 6. Dark theme interface.

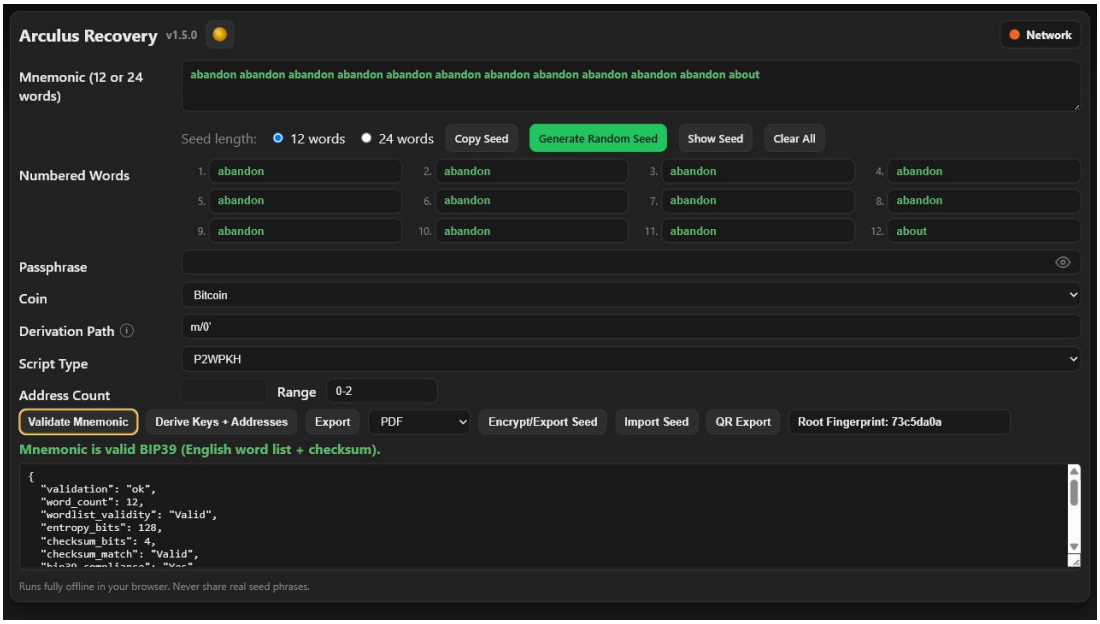

Figure 7. Dark+ theme interface.

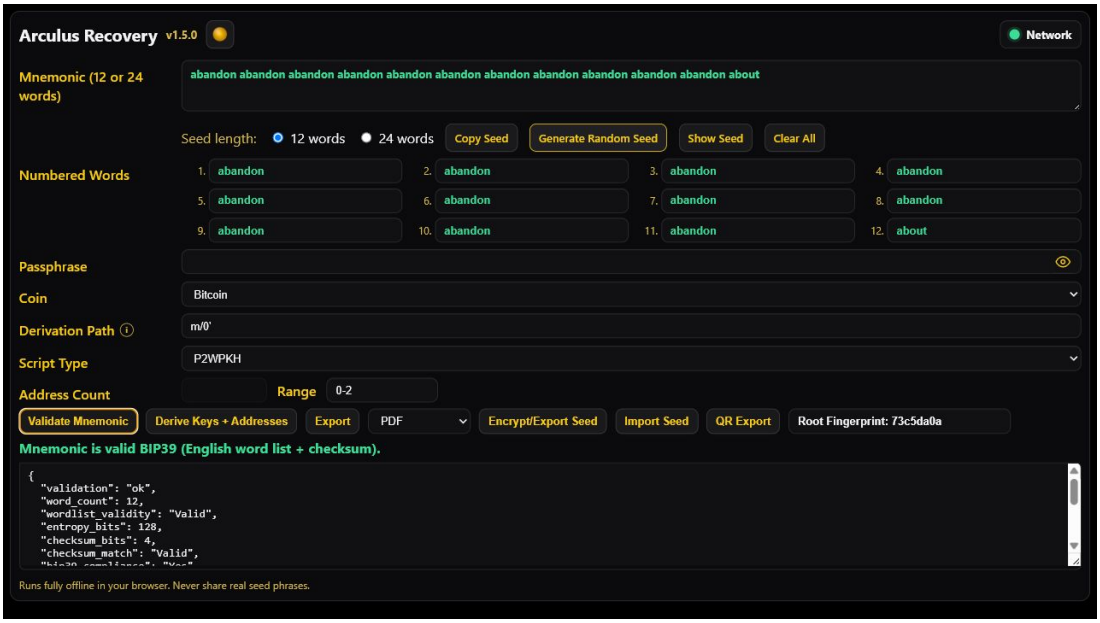

Figure 8. Terminal theme interface.

Arculus Recovery v1.5.0 ? Recovery Procedures

Scope

This document defines the step-by-step procedures for recovering a BIP39 wallet using Arculus Recovery v1.5.0. It covers every supported recovery entry point: direct mnemonic entry, import from a .arc encrypted seed file, and CLI-based recovery. For each entry point it defines the inputs required, the verification steps that must be completed before any output is acted on, the decision tree for locating addresses when the originating wallet configuration is unknown, and the procedure for sweeping funds after a hardware wallet failure.

This document describes what to do and in what order. It does not re-derive the cryptographic constructions underlying the derivation pipeline; those are defined in Derivation.txt. It does not re-specify the .arc file format or encryption scheme; those are defined in Encryption.txt and FileFormat.txt. It does not re-enumerate the threat model or operational security requirements; those are defined in OpSec.txt. All of those constraints apply during every recovery session described here. This document assumes they are satisfied.

The procedures below are written for a user who has already prepared an airgapped machine and a live ephemeral operating system as specified in OpSec.txt. Do not begin any recovery procedure on a machine that does not meet the airgap and ephemeral OS requirements in that document.

Prerequisites Checklist

Before beginning any recovery session, confirm the following conditions are met. A session that proceeds without satisfying these conditions may produce output that is cryptographically correct but operationally unusable, or may expose seed material to a compromised environment.

```
[ ] The machine has no active network route. Physical cables are disconnected.
    Wireless and Bluetooth interfaces are disabled at the hardware level.
    The network status indicator in the tool title bar shows red (no route).
    The indicator is a convenience check; physical confirmation is authoritative.

[ ] The operating system is live-booted and ephemeral. It writes nothing to
    persistent disk. Hibernation is disabled. Swap is RAM-backed or disabled.

[ ] The Arculus Recovery file has been verified by SHA256 hash before use.
    Expected hashes are published in the README. An unverified file must not
    be used for any recovery session involving real key material.

[ ] You have identified all inputs required for this recovery: the mnemonic
    or .arc file, the BIP39 passphrase if applicable, the originating wallet
    software and derivation path if known, and the target coin.

[ ] You are in a private space with no cameras pointed at the screen and no
    other persons with line of sight to the display.
```

Inputs Required for a Complete Recovery

A deterministic BIP39 wallet requires at most four inputs to fully recover:

```
Input 1 ? Mnemonic (required):
  The 12-word or 24-word BIP39 seed phrase. Every wallet address and private
  key derives deterministically from this value. If the mnemonic is lost and
  no .arc backup exists, the wallet is unrecoverable. The mnemonic must be
  entered exactly as originally generated, with correct spelling and word order.
  Partial mnemonics, mnemonics with transposed words, or mnemonics with one or
  more words replaced produce a completely different key tree with no error or
  warning other than BIP39 checksum failure, if the checksum happens to fail.
  A mnemonic with a transposition error that passes the BIP39 checksum is
  indistinguishable from a valid but wrong mnemonic.

Input 2 ? BIP39 passphrase (required if used, otherwise omit):
  An optional additional string that was entered at wallet setup time. It is
  separate from the .arc encryption password. If a passphrase was used and
  is omitted or entered incorrectly, the derived key tree is cryptographically
  unrelated to the one that holds funds. The tool produces no error; it derives
  valid-looking but wrong addresses and keys. There is no mechanism to detect
  a wrong passphrase other than comparing derived addresses against known on-
  chain history or against a previously recorded root fingerprint. If you are
  uncertain whether a passphrase was used, refer to the passphrase decision
  procedure in the Address Location section below.
```

```
Input 3 ? Derivation path (required if non-standard):
  The BIP32 account path that the originating wallet used to derive addresses.
  If the originating wallet was an Arculus hardware wallet using the native
  default, the path is m/0'. If the originating wallet used a standard BIP44/
  49/84/86 path, the path follows the conventions in the derivation path table
  below. If the originating wallet software is unknown, the path must be
  determined by trial; the address location procedure in this document covers
  this case.

Input 4 ? Script type (required for UTXO coins if non-standard):
  For Bitcoin, Litecoin, and Dogecoin, the script type determines the address
  format and the encoding of derived addresses. If the script type is known
  from the originating wallet, select it directly. If unknown, use the Auto
  selector and try each standard path in turn. For Ethereum and XRP, the
  script type is fixed and not selectable.
```

All four inputs must be correct simultaneously for the derived output to match the addresses and keys associated with on-chain history. Errors in any single input produce silently wrong output. Verification against a known address or root fingerprint is mandatory before acting on any derived output.

Standard Derivation Path Reference

This table is the first lookup reference when the originating wallet software is known. Identify the row that matches the originating wallet, coin, and address format, and enter the path and script type exactly as shown.

```
  Coin      Script type    Path              Address prefix  Notes
  --------- -------------- ----------------- --------------- ----------------------
  Bitcoin   P2PKH          m/44'/0'/0'       1...            Legacy BIP44
  Bitcoin   P2WPKH-P2SH    m/49'/0'/0'       3...            Wrapped SegWit
  Bitcoin   P2WPKH         m/84'/0'/0'       bc1q...         Native SegWit
  Bitcoin   P2TR           m/86'/0'/0'       bc1p...         Taproot, BIP86
  Bitcoin   P2WPKH         m/0'              bc1q...         Arculus native default
  Litecoin  P2PKH          m/44'/2'/0'       L...            Legacy BIP44
  Litecoin  P2WPKH-P2SH    m/49'/2'/0'       M...            Wrapped SegWit
  Litecoin  P2WPKH         m/84'/2'/0'       ltc1q...        Native SegWit (tool default)
  Litecoin  P2TR           m/86'/2'/0'       ltc1p...        Taproot-style output
  Dogecoin  P2PKH          m/44'/3'/0'       D...            Default Dogecoin path
  Dogecoin  P2WPKH-P2SH    m/49'/3'/0'       9... or A...    Supported; wallet support
varies
  Dogecoin  P2WPKH         m/84'/3'/0'       doge1q...       Supported; wallet support
varies
  Ethereum  Account        m/44'/60'/0'      0x...           ERC-20 tokens same address
  XRP       Account        m/44'/144'/0'     r...            Destination tags not derived
```

For the Arculus hardware wallet native path (m/0'), the tool displays an info tooltip beside the Derivation Path field as a reminder that this path differs from BIP44 convention. The tooltip is informational only and does not affect derivation behavior.

If the originating wallet used a non-account-0 path (e.g. m/84'/0'/1' for a second account), change the trailing account index from 0 to the correct value. Account indices above 0 are uncommon but legal. Most wallet software creates only account 0 unless the user explicitly created additional accounts.

Procedure 1 ? Recovery from Direct Mnemonic Entry (HTML)

Use this procedure when the mnemonic is available as a written or memorized phrase and no .arc file is involved.

1. Open Arculus_Recovery.html in the browser on the airgapped machine.
   Confirm the network status indicator shows red before proceeding.
   The same HTML workflow is used by the PySide6 GUI because it renders the
   packaged HTML application inside Qt WebEngine.

- Select the word count radio button (12 or 24) that matches the mnemonic.

3. Enter each mnemonic word into its numbered word field. Each field provides
   inline validation against the BIP39 word list; a word that is not in the
   list is flagged immediately. Correct any flagged words before proceeding.
   Note: a word that is in the word list but wrong for the position passes
   inline validation silently. Inline validation confirms only that each word
   exists in the BIP39 word list, not that the complete mnemonic is the
   correct one.

4. After all words are entered, the tool performs full BIP39 validation
   including checksum verification. If the checksum fails, the tool displays
   a checksum error. A checksum failure means at least one word is incorrect,
   or the word order is wrong, or both. Correct the mnemonic until the
   checksum passes.

Note: some transcription errors produce a different valid mnemonic that passes the BIP39 checksum. Checksum verification is a necessary but not sufficient condition for mnemonic correctness.

5. If a BIP39 passphrase was used, enter it in the Passphrase field. If no passphrase was used, leave the field empty. Do not guess. If you are uncertain, proceed with the empty passphrase first and compare derived addresses against known on-chain history. See the Address Location Procedure section if the originating configuration is unknown.

- Select the target coin from the Coin selector.

7. Select the script type. If the originating wallet software is known, select the script type from the reference table above. If unknown, select Auto and proceed to the Address Location Procedure section.

8. Enter the derivation path. If the originating wallet is known, enter the path from the reference table. If the tool default is already correct (m/0' for Arculus native, or the coin-appropriate standard path), no change is required.

9. Set the Address Count to the number of addresses to derive. The default of 5 covers most cases. If the target address is beyond index 4, increase the count or use the Range input to target a specific index window.

10. Click Derive Keys + Addresses. Wait for the derivation progress bar to complete. The output panel populates with derived addresses and keys.

11. Verify the root fingerprint displayed in the toolbar against a previously recorded root fingerprint. If the fingerprints match, the mnemonic and passphrase are correct. If they do not match, and you have a recorded fingerprint from a prior session, the inputs are wrong; do not act on the output.

12. Locate the target address in the output table. Confirm it appears in the on-chain history of the blockchain explorer (performed offline by comparison against a previously exported address list, or after the session on a trusted device). Do not connect the airgapped machine to any network at this step.

13. Export the output in the required format using the Export selector. Transfer the file to a USB drive for handling on a separate trusted machine. The export file contains private key material and must be handled per the Category B requirements in OpSec.txt.
    If using the Tauri desktop wrapper, verify the specific packaged build's export behavior before relying on it during a recovery session. The Tauri package routes PDF, JSON, CSV, TXT, encrypted seed, and QR PNG exports through an injected WebView bridge and native save_export command that writes to the user's Downloads directory.

14. When the recovery session is complete, click Clear All to wipe all fields and in-memory state. Close the browser tab. If the OS is ephemeral, power off the machine to complete RAM clearance.

Procedure 2 ? Recovery from .arc Encrypted Seed File (HTML)

Use this procedure when the mnemonic is stored in a .arc backup file and is not available as a written phrase, or when importing a .arc backup to verify its integrity.

- Complete the airgap and OS prerequisites as specified.

2. Transfer the .arc file to the airgapped machine via USB drive. Do not connect the machine to any network for this transfer.

- Open Arculus_Recovery.html in the browser.

4. Click Import Seed. In the file picker, select the .arc file from the USB drive. The tool reads the file, detects its format version, and prompts for the decryption password.

5. Enter the .arc decryption password. This is the password chosen at export time. It is not the BIP39 passphrase. Click Decrypt. The tool performs MAC verification followed by decryption. If MAC verification fails, the file has been tampered with or the password is wrong. The tool does not distinguish between these two failure modes in the error message; both produce an authentication failure.

6. If decryption succeeds, the mnemonic is loaded into the hidden-seed workflow. The word grid is populated with obfuscated placeholder bullets. The root fingerprint display updates immediately. The mnemonic is held

in protected memory and is not displayed unless Show Seed is pressed and held.

7. Verify the root fingerprint against a previously recorded value. If the fingerprint matches, the .arc file contains the correct mnemonic and the password was entered correctly. If the fingerprint does not match a recorded value, do not proceed until the discrepancy is resolved.

8. Continue from step 5 of Procedure 1 (passphrase entry, coin selection, derivation, and output verification). The derivation pipeline is identical regardless of whether the mnemonic was entered directly or imported from a .arc file.

9. The Show Seed button is available if visual confirmation of the full mnemonic is needed. Press and hold to reveal the mnemonic temporarily. Release to re-obscure. Minimize the duration for which the mnemonic is visible.

- Complete the session per steps 13 and 14 of Procedure 1.

Procedure 3 ? Recovery via Python CLI

Use this procedure when the HTML browser interface is not available or when scripted derivation of a large address range is required.

1. Copy Arculus_Recovery.py and the src/ directory to a RAM-backed tmpfs location on the airgapped machine before execution. Verify with the mount command that the target directory is tmpfs-backed. For GUI recovery, copy the full project folder, including Arculus_Recovery.html, src/, pyproject.toml, requirements.txt, and packaged assets. The current HTML file embeds jsPDF inline and does not require vendor/ at runtime.

2. For direct mnemonic entry, invoke the CLI with the mnemonic, passphrase, coin, derivation path, script type, and address count as required:

```
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
  --passphrase "optional passphrase" \
  --derivation "m/84'/0'/0'" \
  --script-type p2wpkh \
  --coin bitcoin \
  --count 10 \
  --output-format json
```

3. To derive all common paths for a coin in a single invocation when the originating wallet is unknown:

```
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
  --coin bitcoin \
  --all-common \
  --count 5 \
  --output-format txt
```

The --all-common flag derives m/44', m/49', m/84', and m/86' purpose paths for UTXO coins, and m/44' only for Ethereum and XRP.

- Redirect output to a file on the tmpfs volume:

```
python Arculus_Recovery.py ... > /tmp/recovery_output.json
```

Do not write output to a persistent volume. The /tmp directory on a live Linux environment is tmpfs-backed and writes to RAM only.

5. Verify the root_fingerprint field in the JSON output against a previously recorded value before relying on any derived address or key.

6. Transfer the output file to a USB drive for handling on a separate trusted machine per Category B requirements in OpSec.txt.

7. On session completion, remove the output file from /tmp and power off the machine. RAM is cleared at power loss on an ephemeral OS.

Address Location Procedure

Use this procedure when the originating wallet software, derivation path, script type, or passphrase usage is unknown. The goal is to systematically locate the derivation configuration that produces addresses matching known on-chain history. This procedure requires a reference address: at least one address that is known to have been funded from the wallet being recovered.

If no reference address is available, the procedure below cannot definitively confirm a match. In that case, derive all standard paths and inspect each for plausible on-chain history after the airgapped session ends, on a separate trusted networked device.

```
  Step 1 ? Determine passphrase usage:

    Derive with an empty passphrase on the most likely path for the coin and
    script type (see reference table). Compare the derived addresses against
    the reference address. If the reference address appears, the wallet used
    no passphrase. Proceed to step 2.

    If the reference address does not appear with an empty passphrase, the
    wallet likely used a passphrase. Try each known or candidate passphrase
    in turn, repeating the derivation. If none produce the reference address,
    the passphrase is unknown and the funds may be inaccessible by this path.
    Record the root fingerprint for each passphrase attempt for future reference.

  Step 2 ? Identify the derivation path and script type:

    For Bitcoin, try the paths below in order. For each, derive at least 10
    receiving branch (branch index 0) addresses and compare against the
    reference address. The address prefix is a fast filter: if the reference
    address starts with bc1p, test only m/86' paths; if it starts with bc1q,
    test only m/84' and m/0' paths; if it starts with 1, test only m/44' paths;
    if it starts with 3, test only m/49' paths.
```

- m/84'/0'/0' (P2WPKH, most common for modern Bitcoin wallets)
- m/0' (P2WPKH, Arculus native default)
- m/44'/0'/0' (P2PKH, legacy BIP44)
- m/49'/0'/0' (P2WPKH-P2SH, wrapped SegWit)
- m/86'/0'/0' (P2TR, Taproot)

```
    For Litecoin, try in order: m/84'/2'/0', m/44'/2'/0', m/49'/2'/0',
    m/86'/2'/0'.

    For Dogecoin, try in order: m/44'/3'/0', m/49'/3'/0', m/84'/3'/0'.

    For Ethereum and XRP, the path is fixed (m/44'/60'/0' and m/44'/144'/0'
    respectively). Only the passphrase can vary.

    Use --all-common in the CLI to derive all four Bitcoin purpose paths in one
    invocation and inspect the combined output.

  Step 3 ? Extend the address window if needed:

    If none of the receiving branch addresses at indices 0?9 match the reference
    address, increase the address count to 25, then 50, then 100. A wallet that
    has been in use for a long time or that uses a gap-aware derivation policy
    may have funded addresses at high indices. The derivation is fast; a count
    of 100 adds negligible time.

    Also check the change branch (branch index 1) if the reference address is
    a change output. Change addresses are commonly funded in UTXO wallets and
    are on branch index 1 rather than branch index 0.

  Step 4 ? Check non-standard account indices:

    If account index 0 does not produce the reference address, try account
    index 1 and 2 (e.g. m/84'/0'/1', m/84'/0'/2'). Some wallet software
    creates multiple accounts and the wallet in question may have used a
    non-default account.

  Step 5 ? Record findings:

    Once the correct path, script type, and passphrase are confirmed, record:
      - The root fingerprint for this mnemonic and passphrase combination
      - The derivation path
      - The script type
      - Whether a passphrase was used (but not the passphrase value itself,
```

```
    unless it is stored in a separately secured location)

These four values are sufficient to reproduce the derivation configuration
in future sessions without repeating the discovery procedure.
```

Sweeping Funds After Hardware Wallet Failure

Use this procedure when a hardware wallet has failed, been lost, or been destroyed and funds must be moved to a new wallet. This procedure assumes the recovery session is complete (Procedure 1, 2, or 3) and the derived addresses and private keys have been exported to a file on an encrypted USB drive.

This procedure involves importing private key material into a software wallet on a networked device. That device should be considered exposed to the private keys after this operation is complete. The hardware wallet and its mnemonic should be treated as compromised from the moment the private keys are imported into any networked software.

```
1. On the airgapped machine, complete the derivation and export the relevant
   private keys in JSON or CSV format to an encrypted USB drive.

2. On a trusted, clean, networked device, open the software wallet of your
   choice (Electrum for Bitcoin, MetaMask for Ethereum, etc.) in sweep or
   watch-only mode.

3. Import the private key for each address that holds funds. Most software
   wallets accept WIF (for UTXO coins) or hex private keys (for Ethereum)
   for individual address sweeps. Do not import the extended private key into
   a networked wallet; prefer per-address WIF imports to minimize exposure.

4. Construct a transaction from each funded address to an address in the new
   hardware wallet. Broadcast all sweep transactions before considering the
   old wallet retired.

5. Verify on-chain that each sweep transaction has confirmed before the next
   step. Do not retire the old wallet backup until all funds have moved and
   confirmed.

6. After all funds have swept and confirmed, the old mnemonic and its .arc
   backup are no longer needed for fund access. They should be retained as
   historical records of the old wallet or destroyed per your operational
   policy for retired key material. A mnemonic that has been imported into
   any networked device should be considered permanently exposed.

7. Generate a new mnemonic for the replacement hardware wallet using the
   hardware wallet's own CSPRNG. Do not reuse the recovered mnemonic as a
   new wallet seed. Do not use Arculus Recovery to generate the replacement
   hardware wallet seed; use the hardware wallet's own generation mechanism.

8. Back up the new mnemonic and set up a new .arc file on the airgapped
   machine following the backup verification procedure in OpSec.txt.
```

Recovery Verification Checklist

Before ending any recovery session and acting on derived output, confirm all of the following. Each unchecked item is a potential source of irreversible error.

- The BIP39 checksum passed during mnemonic validation.

```
[ ] The root fingerprint matches a previously recorded value, OR this is
    the first session for this mnemonic and the fingerprint has been recorded
    for future reference.

[ ] At least one derived address has been confirmed to match known on-chain
    history, either against a recorded address list or against a prior export.

[ ] The passphrase field contained the correct value (including empty if no
    passphrase was used). A single character difference produces a completely
    different key tree.
```

- The derivation path and script type match the originating wallet.

```
[ ] The export file has been saved to an encrypted USB drive and has not been
    written to any persistent or networked filesystem on the airgapped machine.

[ ] Clear All has been clicked or the session will be ended by powering off
    the machine. In-memory seed state is not cleared by navigating away from
    the tab or minimizing the window; it persists until Clear All is used or
    the tab is closed.

[ ] The USB drive containing the export file is physically secured and will
```

```
    be handled under Category B controls as defined in OpSec.txt.
```

Common Recovery Failures and Resolutions

The following failures account for the majority of unsuccessful recovery attempts. Each is listed with its diagnostic indicator and corrective action.

```
Failure: No derived addresses match on-chain history.
  Indicator: All derived addresses are unfamiliar. Root fingerprint does not
    match any recorded value.
  Most likely cause: Wrong passphrase, wrong mnemonic word, or wrong derivation
    path. These three causes are indistinguishable by inspection of the output.
  Resolution: Verify the BIP39 checksum passes. Try the empty passphrase if
    passphrase usage is uncertain. Try each standard derivation path using the
    address location procedure above. If the mnemonic was hand-transcribed,
    check each word against the BIP39 word list for plausible transcription
    errors.

Failure: BIP39 checksum fails.
  Indicator: The tool displays a checksum validation error.
  Most likely cause: One or more mnemonic words are misspelled, in wrong order,
    or from the wrong word list. The BIP39 English word list is the only
    supported word list.
  Resolution: Review each word against the BIP39 word list. The 2048-word list
    is published in the BIP39 specification. Common errors include: visually
    similar words (witch/which, right/light), words differing by one letter
    (able/able), and words from non-English BIP39 word lists.

Failure: .arc import fails with authentication error.
  Indicator: The tool displays an authentication or MAC verification failure
    after password entry.
  Most likely cause: Wrong password, or the file was modified after export.
    Both causes produce identical error messages.
  Resolution: Re-enter the password carefully. If the password contains Unicode
    characters, confirm they are entered in the same normalized form used at
    export time. If the correct password is known with certainty and verification
    still fails, the file has been corrupted or tampered with and cannot be
    decrypted. Recover from an alternative backup.

Failure: .arc import fails with format error before password prompt.
  Indicator: The tool rejects the file immediately without prompting for a
    password.
  Most likely cause: The file is corrupted, truncated, or is not a .arc file.
  Resolution: Verify the file is a valid UTF-8 text file beginning with
    "ARCULUS-ARC-V2". Confirm the file transfer was complete and the USB drive
    is readable. Try a second copy of the file from a separate USB drive if one
    exists.

Failure: Derived output matches the mnemonic but not the expected addresses.
  Indicator: Root fingerprint matches a recorded value, but the specific
    addresses in the output are unfamiliar.
  Most likely cause: Wrong derivation path, wrong script type, wrong account
    index, or wrong address count (the target address is beyond the derived
    window).
  Resolution: Try each standard derivation path. Increase the address count.
    Check the change branch (branch index 1). Try account indices 1 and 2.

Failure: Passphrase-derived addresses are correct but passphrase is forgotten.
  Indicator: Derivation with the empty passphrase produces addresses with no
    on-chain history. The passphrase-derived root fingerprint (recorded
    previously) does not match the empty-passphrase fingerprint.
  Resolution: There is no recovery path for a forgotten BIP39 passphrase from
    the mnemonic alone. The passphrase is not stored in the .arc file; the .arc
    file stores the mnemonic only. If the passphrase was written down separately,
    locate that record. If no backup of the passphrase exists, the funds in the
    passphrase-derived key tree are inaccessible.
```

Arculus Recovery v1.5.0 ? Operational Security Guide

Scope

This document defines the operational security requirements, recommended procedures, and threat mitigations for all use of Arculus Recovery v1.2.0- production. It covers the full operational lifecycle: airgap preparation, session execution, output handling, .arc file management, long-term storage, and incident response. It does not repeat the cryptographic specifications defined in Derivation.txt and Encryption.txt except where those specifications have direct operational consequences the user must act on.

This document is not optional reading. The cryptographic guarantees of the engine are only realized if the operational conditions below are satisfied. A mathematically sound derivation pipeline operating on a compromised machine or with output handled insecurely provides weaker protection than the most naive tool operated correctly. The weakest link in the security chain is never the algorithm.

Threat Model

The engine is designed to resist the following adversary classes under the stated conditions.

```
Adversary class 1 ? Offline file acquisition:
  An adversary who obtains a .arc file without the decryption password gains
  no access to the mnemonic. The PBKDF2-HMAC-SHA512 construction at 1,000,000
  iterations is the resistance barrier. This barrier is linear in iteration
  count and does not protect a low-entropy password against an adversary with
  GPU acceleration. See the password entropy requirements section below for the
  minimum entropy threshold that makes this barrier meaningful.

Adversary class 2 ? Network adversary:
  The engine performs no network I/O of any kind. A network adversary who
  intercepts traffic to and from a machine running the engine observes nothing
  related to the mnemonic, its derived keys, or the .arc file contents.
  This protection is contingent on the machine being airgapped or the network
  being fully disconnected before mnemonic input begins. It does not hold if
  the machine is connected to any network while seed material is in memory.

Adversary class 3 ? Post-session disk forensics:
  An adversary who performs disk forensics on a machine after a recovery session
  may recover mnemonic material from swap space, hibernation files, crash dumps,
  or unwiped free space. This risk is addressed by the airgap and ephemeral OS
  requirements below. It is not mitigated by the engine itself; it must be
  mitigated operationally.

Adversary class 4 ? .arc file tampering:
  The encrypt-then-MAC construction ensures that any modification to any field
  of a .arc file ? including KDF parameters, nonce, ciphertext, or MAC ? is
  detected at import time as an authentication failure. An adversary who obtains
  a .arc file cannot modify it in any way without the MAC verification gate
  detecting the tampering. This protection is unconditional and does not depend
  on password strength.

Adversary class 5 ? Clipboard and screen interception:
  An adversary with access to clipboard contents or screen capture can observe
  mnemonic material directly. This risk exists during any session in which the
  mnemonic is displayed or copied. Mitigations are operational only: minimize
  display duration, avoid clipboard use for seed material, do not capture
  screenshots during a session.

Adversary class 6 ? Physical access during a live session:
  An adversary with physical access to a machine during an active session can
  observe the screen, photograph keyboard input, or perform a cold boot attack
  to extract RAM contents. This risk is addressed by physical access controls
  during the session: closed private room, no cameras pointed at the screen,
  session duration minimized.
```

Out-of-scope adversaries ? The following are explicitly outside the threat model. No operational procedure in this document provides protection against them:

```
  - An adversary with full root access to the operating system before the
    session begins (rootkit, firmware compromise, supply chain attack on the
    hardware, evil maid attack on a non-ephemeral OS).
  - An adversary who learns the mnemonic by other means before the session
    (prior compromise of the source hardware wallet, coercion of the user).
  - A hardware keylogger installed between the keyboard and the machine.
```

```
- An adversary who holds the correct .arc password and uses it to decrypt
  legitimately; a correct password always unlocks the file, as designed.
```

Airgap Requirements

A recovery session involving live mnemonic material must be conducted on a machine that is fully airgapped for the duration of the session. Airgap means:

```
Physical disconnection of all network interfaces:
  - Remove or disable Ethernet cables. If the NIC cannot be disabled in
    firmware, remove the cable physically.
  - Disable Wi-Fi at the hardware level. On laptops, this is the hardware
    wireless kill switch if present, or removal of the wireless card if not.
    Disabling Wi-Fi in the operating system is insufficient if the kernel
    module remains loaded; a firmware or RF-level disable is required.
  - Disable Bluetooth at the hardware level by the same criteria.
  - If the machine has a cellular modem (some laptops), it must be disabled
    at the same level.

Verification of disconnection:
  - Confirm the machine has no active network route before beginning. In the
    Python GUI, the network status indicator in the title bar provides a real-
    time probe. The indicator polls a UDP connection attempt to 8.8.8.8:80
    every five seconds and displays the result. A green indicator means a
    route exists; a red indicator means no route was detected. The indicator
    is a convenience check, not a security control. Physical confirmation of
    cable removal and hardware switch state is authoritative.

Duration of airgap:
  - The airgap must be maintained from before any mnemonic input begins until
    after all derived output has been recorded and all in-memory state has
    been cleared. Reconnecting to a network while the mnemonic is displayed
    or while output containing private keys is on screen defeats the airgap
    entirely. There is no safe reconnection point during an active session.
```

Ephemeral Operating System Requirements

Disk forensics on a machine that was previously used for a recovery session can recover mnemonic material from locations the user never explicitly wrote to: swap files, hibernation images, crash dumps, browser caches (in the HTML version), and the filesystem journal. The only complete mitigation is to conduct the session from an operating system that writes nothing to persistent storage.

Recommended approach ? Live bootable OS:

```
Boot the machine from a live USB drive running a read-only operating system
such as Tails or a standard Linux live environment. A live environment that
has not been configured to persist writes to the USB drive operates entirely
from RAM. When the machine is powered off after the session, all in-memory
state ? including mnemonic, derived keys, swap contents, and any temporary
files ? is destroyed at power loss, provided the machine does not hibernate.
Hibernation must be disabled. Suspend-to-RAM is acceptable if the machine
will not leave physical control during the session.

For Python CLI use from the repository launcher: copy Arculus_Recovery.py
and the src/ directory to a RAM-backed tmpfs (e.g. /tmp on most Linux live
environments) before running. For Python GUI use, copy the full project
folder, including Arculus_Recovery.html, src/, pyproject.toml,
requirements.txt, and the packaged assets under src/arculus_recovery/assets/.
The vendor/ tree is a source copy used for regeneration and fallback
packaging; the current standalone HTML file embeds jsPDF inline and does not
require vendor/ at runtime. Do not execute the tool from a persistent volume
mounted read-write. Verify with the mount command that /tmp is tmpfs-backed
before proceeding.

For the HTML implementation: open the local HTML file from the live
environment's file manager or browser. Verify that the browser is not
configured to sync history, passwords, or form data to any cloud service.
Clear browser history and cache before closing the session. On a live OS,
this state is discarded at shutdown regardless, but explicit clearing is
good practice.
```

Alternative ? Dedicated offline machine:

```
A machine purchased new and never connected to any network provides a stronger
baseline than a machine that has previously been online. However, a dedicated
offline machine retains persistent storage and therefore retains data across
```

```
  sessions. After any session on a dedicated offline machine, the swap partition
  and any temporary files should be explicitly wiped using a tool that overwrites
  with random data. On SSDs, wear-leveling means software wiping does not
  reliably reach all cells that held the data; this limitation is an inherent
  property of SSD architecture and is not addressable through software alone.
  A live USB approach does not have this limitation.

Python version requirement:
  The CLI engine requires Python 3.10 or later and uses the Python standard
  library. GUI mode additionally requires PySide6, which provides the Qt
  WebEngine window used to render the local HTML interface. A minimal Python
  installation is sufficient for CLI use; install requirements.txt only when
  the --gui flag is needed.

Desktop wrapper requirement:
  The Tauri desktop wrapper is a separately built native package around the
  canonical HTML application. It adds an injected WebView export bridge, a
  native save command, and Tauri v2 capability/permission files for desktop
  exports. Validate the final packaged application before relying on it for an
  operational recovery session. If Tauri export behavior is uncertain on a
  specific host, use the standalone HTML application or the PySide6 GUI for GUI
  exports, or use the Python CLI with redirected stdout for JSON, CSV, and TXT
  output.
```

Session Execution Procedure

The following procedure defines the correct sequence of operations for a recovery session. Deviation from this sequence does not make the session invalid, but may expose mnemonic material beyond the minimum necessary.

Pre-session:

```
1. Prepare the physical environment. Close the room. Ensure no cameras,
   whether device cameras, security cameras, or other observers' phones, have
   a line of sight to the screen or keyboard.

2. Boot the airgapped machine from the live OS or, if using a dedicated
   offline machine, confirm it has not been connected to any network since
   the last secure wipe.

3. Confirm the airgap. Remove network cables physically. Verify the hardware
   wireless kill switch position. If using the Python GUI, confirm the network
   indicator is red before proceeding.

4. Copy the tool files to a RAM-backed location if on a live OS. Do not run
   directly from a mounted persistent volume.

5. If importing from a .arc file, transfer the .arc file to the machine via a
   method that does not require network connectivity: USB drive, SD card, or
   QR code scan. The USB drive should be dedicated to this purpose and not
   used with networked machines after the session. If the USB drive was
   previously connected to a networked machine, its filesystem may contain
   traces of the file regardless of deletion.
```

During session:

```
6. Enter the mnemonic using the keyboard. Do not paste from a clipboard unless
   the clipboard was populated on this same airgapped machine immediately
   before use. External clipboard managers, clipboard history, and cloud
   clipboard sync features must be disabled. The Copy Seed button in the
   Python GUI triggers an explicit warning before placing seed material on the
   clipboard; this warning is informational, not a safety control.

7. Validate the mnemonic before deriving. A BIP39 checksum failure indicates
   a word was entered incorrectly. Deriving keys from an incorrect mnemonic
   produces a completely different key tree with no connection to any on-chain
   history. The root fingerprint displayed in the UI after validation is the
   correct verification artifact: compare it against a previously recorded
   fingerprint to confirm mnemonic entry was correct before exporting any key
   material.

8. Confirm the derivation path and script type before deriving. An incorrect
   path or script type produces different addresses silently. There is no error
   produced when a correct mnemonic is derived with a wrong path; the engine
   produces valid key material for whatever path is specified.

9. If exporting derived key material (JSON, CSV, TXT, PDF, or QR PNGs that
   encode sensitive material), do so to a RAM-backed location. Do not write
   to a persistent volume unless the persistent volume
   will be explicitly wiped after the session. Recognize that any export file
```

```
   containing private_key_hex, private_key_wif, or any extended private key
   field is operationally equivalent in sensitivity to the mnemonic itself.
   There is no partial sensitivity: one exported private key gives access to
   exactly one derived address; a full export gives access to all of them.

10. If exporting a .arc file, copy it to a USB drive dedicated to encrypted
    seed storage. Do not copy it to a drive that will be connected to a
    networked machine for any other purpose without full-disk encryption
    protecting that machine's storage.
```

Post-session:

```
11. Clear all on-screen mnemonic display. If using the Python GUI, the
    imported seed overlay hides the mnemonic after import; if words were typed
    directly, clear the mnemonic fields explicitly before closing the window.
    Clear the output panel.

12. If on a live OS, power the machine off completely. Do not suspend or
    hibernate. Power-off destroys all RAM contents, clearing all session state
    permanently.

13. If on a dedicated offline machine, explicitly wipe the swap partition and
    any RAM-backed temp directories that were written to during the session.
    Power off completely.

14. Store the USB drive containing the .arc file in a physically secure
    location separate from the hardware wallet and separate from any written
    backup of the mnemonic. Physical separation of secrets reduces the blast
    radius of any single physical compromise.
```

Password Entropy Requirements for .arc Encryption

The .arc format's protection against offline guessing attacks is a direct function of the password's entropy. The PBKDF2-HMAC-SHA512 construction at 1,000,000 iterations imposes a fixed computational cost per guess. The cost is multiplicative with password entropy: high entropy makes the cost irrelevant because the search space is too large to exhaust; low entropy makes the cost irrelevant because the password can be recovered in hours or days regardless.

The following guidance defines minimum-acceptable and recommended entropy levels:

```
Unacceptable (below minimum):
  - Any single dictionary word in any language.
  - Any PIN of any length. A 16-digit PIN has 53.2 bits of entropy at best and
    is typically far less in practice due to birth dates, patterns, and reuse.
  - Personal information: names, dates, addresses, phone numbers, combinations
    thereof. These have entropy far below what the label implies because
    adversaries construct custom wordlists from available personal data.
  - Passwords used for any other account or system, regardless of length or
    apparent complexity. Credential databases leaked from other services enable
    targeted attacks against reused passwords.
  - Short character strings under 12 characters, regardless of character set.

Minimum acceptable (marginal):
  - A randomly selected sequence of 4 words from a large wordlist (the Diceware
    standard or equivalent), providing approximately 51 bits of entropy from a
    7776-word list. This is the absolute floor for a password that provides any
    meaningful resistance to a motivated adversary with GPU acceleration.
    This level is not recommended; it should be considered temporary and upgraded.

Recommended:
  - A randomly selected sequence of 6 or more words from a large wordlist,
    providing approximately 77 bits of entropy from a 7776-word list. At this
    entropy level, exhaustive search is computationally infeasible for any
    adversary, regardless of hardware capabilities, for the foreseeable future.
  - A randomly generated alphanumeric string of 20 or more characters from a
    full character set (uppercase, lowercase, digits, symbols), providing
    approximately 130+ bits of entropy. This is the strongest practical option
    for users comfortable managing such a string.
```

The word-based approach is strongly preferred over manually composed passwords for one reason: humans are systematically poor at composing high-entropy strings from memory. A randomly generated word sequence has provably measurable entropy; a human-composed password has unknown effective entropy that is almost always substantially lower than its apparent complexity suggests.

A password that meets the entropy threshold above is worth protecting with the same diligence as the mnemonic itself. Write it down and store it separately from the .arc file and from the mnemonic. A .arc file without the password is unrecoverable by any

means, including by the authors of this software.

The .arc password and the BIP39 wallet passphrase are different secrets managed on different key schedules. They may have the same value by user choice, but the engine treats them as entirely separate inputs. An incorrect BIP39 passphrase produces a valid-looking but wrong key tree with no error; an incorrect .arc password produces an authentication failure. The operational consequence is that both must be preserved separately and accurately. Loss of either results in permanent loss of access to different things.

Output Handling and Storage

Derived key material falls into two categories with fundamentally different handling requirements.

```
Category A ? Addresses only (public information):
  P2PKH, P2WPKH-P2SH, P2WPKH, P2TR, Ethereum, and XRP addresses are public by
  construction; they are published on-chain for every transaction. Recording,
  storing, or transmitting addresses does not require any special handling. Use
  addresses freely for the purpose of verifying that derivation matches
  expectations or for sharing with counterparties for payment.

Category B ? Private keys and seeds (funds-controlling secrets):
  private_key_hex, private_key_wif, internal_private_key_hex, output_private_
  key_hex, extended private keys (xprv, yprv, zprv, tprv), and the mnemonic
  itself are all funds-controlling secrets. Any of these, in any form, provides
  complete access to the associated funds. There is no partial exposure: a single
  exported private key controls exactly one address; the mnemonic controls the
  entire key tree across all paths, all coins, and all addresses.

  Operational requirements for Category B material:
    - Never transmit over any network connection, including encrypted ones,
      unless the receiving party is intended to have full control of the funds.
    - Never store on any device that has ever been connected to a network unless
      that device uses full-disk encryption and the encryption key itself has
      equivalent or higher entropy than the material it protects.
    - Never photograph or record video of Category B material with a device that
      uploads to cloud storage, even if the upload feature has been manually
      disabled. Cloud-connected cameras have historically re-enabled uploads
      through software updates without user awareness.
    - Paper copies of Category B material must be stored in physically secure
      locations: fire-resistant containers, safety deposit boxes, or equivalent.
      Multiple copies in geographically separated locations reduce single-point
      physical risk.
    - JSON, CSV, and TXT derived-output export files are not encrypted by the
      .arc format. They are plaintext files. If an export file contains any
      Category B field, it is a Category B artifact and must be handled as such.
      The .arc format encrypts only the mnemonic; it does not encrypt derivation
      output.
```

.arc File Backup and Verification Protocol

A .arc file that cannot be decrypted at recovery time is worse than no backup: it creates false confidence that a backup exists while providing no actual recovery capability. The following protocol is mandatory before treating any .arc file as a valid backup.

Export and verify at creation time:

- Export the .arc file from the mnemonic on the airgapped machine.

```
2. Without closing or restarting the tool, import the .arc file back into the
   same session using the Import Seed function. Enter the same password.

3. Confirm that the imported mnemonic matches the source mnemonic by verifying
   the root fingerprint. The root fingerprint is derived deterministically from
   the mnemonic and passphrase; if the imported round-trip produces the same
   fingerprint, the mnemonic was stored and recovered correctly.

4. If the root fingerprint does not match, do not use this .arc file as a
   backup. Identify the discrepancy before proceeding.

5. Transfer the .arc file to the storage USB drive. Copy it to at least two
   physically separate drives to protect against media failure.
```

Periodic verification (recommended every 12 months):

- On an airgapped machine, import the stored .arc file and enter the password.
- Verify the root fingerprint matches the expected value.

8. If verification fails due to a forgotten password, the .arc file is unrecoverable. The mnemonic must be recovered through other means (hardware wallet, paper backup, or other encrypted backup). If no other recovery path exists, the funds are inaccessible.

9. If verification succeeds, re-export a fresh .arc file. Re-export is recommended annually because it upgrades the salt, nonce, and any improvements to the KDF parameters introduced in newer versions, and provides evidence that the export chain remains intact.

Media longevity:

```
USB drives have an estimated data retention of 10 years under normal storage
conditions; actual retention varies substantially with temperature, humidity,
and usage frequency. For multi-year storage, test the drive's readability
periodically and maintain at least two copies on separate media. Metal-backed
storage solutions (stamped or engraved stainless steel or titanium) are
appropriate for a paper mnemonic backup that must survive fire, water, and
physical abuse. USB drives are not.
```

Multi-Factor Recovery Architecture

A recovery architecture that stores a single backup in a single location has a single point of failure. The recommended architecture separates secrets across multiple locations and media types such that no single physical event compromises recovery capability.

Recommended minimum architecture:

```
Component 1 ? Hardware wallet (primary):
  The production signing device. Located where it is used. Secured by PIN and,
  if the wallet supports it, a passphrase. Loss or destruction of this device
  triggers recovery from one of the components below.

Component 2 ? Mnemonic paper backup (primary recovery path):
  A physical record of the mnemonic words written in permanent ink on archival
  paper or engraved on metal. Stored in a fire-resistant container at a
  different physical location from the hardware wallet. This backup is
  sufficient to restore the wallet on any BIP39-compatible device. No password
  is required; if the wallet uses a BIP39 passphrase, the passphrase must be
  stored separately and is required for full recovery.

Component 3 ? .arc encrypted digital backup (redundant recovery path):
  A .arc file stored on one or more USB drives at one or more physical
  locations. The .arc password is stored separately from the USB drive,
  either as a second paper backup or in a separate physical location.
  This backup provides recovery capability without requiring the paper backup
  to be intact, and survives scenarios where the paper backup is destroyed but
  the USB drive is intact.

Component 4 ? BIP39 passphrase backup (if applicable):
  If a BIP39 wallet passphrase is in use, it must be backed up independently
  of the mnemonic. A mnemonic backed up without its passphrase recovers only
  the empty-passphrase key tree, not the passphrase-derived tree where funds
  may actually reside. The passphrase backup must be stored at a location and
  in a medium that survives independently of both the hardware wallet and the
  mnemonic backup.
```

No single physical location should hold both the mnemonic (in any form) and the means to access funds: the hardware wallet, a device with the .arc password, or any private key material. Geographic separation between backup components substantially reduces the probability that any single event ? fire, flood, theft, or natural disaster ? results in total loss.

Common Operational Errors and Their Consequences

This section enumerates the most frequently encountered operational failures and their consequences. Each failure is recoverable if caught early; some become permanent after sufficient time passes.

```
Error: Deriving with an incorrect BIP39 passphrase.
  Consequence: The derived addresses and keys are cryptographically unrelated to
  any on-chain history. No error is produced. The only detection mechanism is
  root fingerprint comparison against a previously recorded value. If the user
  sends funds to an address derived from an incorrect passphrase, those funds
  are recoverable only if the incorrect passphrase can be identified and re-
  entered. There is no other path; the engine cannot recover which passphrase
  was used from the mnemonic alone.

Error: Exporting derived output to a networked machine before recording addresses.
```

Consequence: If the export file reaches a networked machine, Category B key material has been exposed to the network threat surface. All derived private keys in the export should be considered potentially compromised. Funds controlled by those keys should be moved to addresses derived from a freshly generated mnemonic at the earliest practical opportunity.

Error: Using the same .arc password for multiple .arc files protecting different mnemonics, with any one file stored on a networked machine.
Consequence: If the password is compromised through the lower-security storage path, all .arc files using that password are compromised. Use distinct, independently generated passwords for .arc files protecting distinct mnemonics.

Error: Failing to record the root fingerprint at backup creation time.
Consequence: There is no way to verify that a subsequently recovered mnemonic is the correct one without access to the original hardware wallet or a known on-chain address. Record the root fingerprint alongside, but not inside, every mnemonic and .arc backup. A root fingerprint has no security-relevant value on its own (it does not enable key recovery); it is safe to store alongside other backup metadata.

Error: Operating the tool on a machine connected to a network during seed entry.
Consequence: Screen capture, clipboard interception, RAM scraping, and keylogging all become possible network-accessible attack vectors if the machine is compromised. The airgap requirement exists because these attacks are common against machines with value associated with cryptocurrency operations. If a session was conducted on a connected machine and the machine may be compromised, the mnemonic and all derived keys should be considered exposed.

Error: Failing to verify the .arc round-trip before destroying the mnemonic.
Consequence: If the .arc export was corrupted during write (a real failure mode on failing storage media), or if the password was mistyped at export time, the backup is unrecoverable. Never delete or destroy a mnemonic backup until a full export-import-verify cycle has been completed successfully.

Error: Storing the .arc file and its decryption password in the same location.
Consequence: A single physical compromise (theft, fire, or unauthorized access to the storage location) yields both the encrypted file and the key to decrypt it. The .arc encryption provides no protection in this scenario. The password and the file must be stored at physically separate locations.

Arculus Recovery v1.5.0 - BIP39 Passphrase Reference

Scope

This document defines the BIP39 passphrase as implemented in Arculus Recovery v1.5.0: its cryptographic role in seed derivation, how it interacts with the mnemonic, the precise consequences of every class of passphrase error, the distinction between the passphrase and the .arc encryption password, backup requirements, the visibility toggle introduced in v1.4.0, and the operational procedures for entering, verifying, and storing the passphrase correctly.

The BIP39 passphrase is the single most operationally dangerous input in the entire recovery workflow. A mnemonic entered with a wrong character produces a BIP39 checksum failure on most errors; a passphrase entered wrong produces no error of any kind and no warning. It silently derives a completely valid-looking key tree that has no connection to any funded wallet. This document exists because the gap between "no error was produced" and "the output is correct" is nowhere larger than it is for the passphrase.

This document is specific to the BIP39 passphrase. The .arc encryption password is a different secret, defined in Encryption.txt. The derivation pipeline that consumes the passphrase is defined in Derivation.txt. Operational security requirements for any session that handles the passphrase are defined in OpSec.txt.

What the BIP39 Passphrase Is

The BIP39 passphrase is an optional string that participates in seed derivation as a second factor. It was specified in BIP39 and is sometimes called the "25th word" by hardware wallet documentation, though it is not constrained to the BIP39 word list and is not a word in any technical sense. It may be any Unicode string of any length, including the empty string.

Its position in the derivation pipeline is defined precisely:

```
password_bytes = NFKD(mnemonic).encode("utf-8")
salt_bytes     = ("mnemonic" + NFKD(passphrase)).encode("utf-8")

seed = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = salt_bytes,
    iterations = 2048,
    dklen      = 64
)
```

The passphrase is concatenated onto the string "mnemonic" and used as the PBKDF2 salt, not as the PBKDF2 password. The mnemonic is the PBKDF2 password. This parameter assignment follows the BIP39 specification exactly. The reversal from intuitive expectations - passphrase-as-salt rather than passphrase-as-password - is a deliberate design in BIP39 and has no practical security consequence; PBKDF2 derives key material from both parameters under a construction that treats neither as more privileged than the other.

The output of this PBKDF2 invocation is a 512-bit (64-byte) seed. Every bit of every key derived from the seed - including every address, every private key, and every extended key at every depth - is a deterministic function of both the mnemonic and the passphrase together. Neither alone is sufficient to reconstruct the seed.

What the BIP39 Passphrase Is Not

The BIP39 passphrase is not any of the following:

```
It is not the .arc encryption password. The .arc file stores and protects
only the mnemonic. It does not store the passphrase. Decrypting a .arc file
recovers the mnemonic; re-entering the passphrase separately during derivation
is still required. See the section on password independence below.

It is not a PIN or login credential for any software. It does not protect
access to the tool itself. No part of Arculus Recovery uses it for
authentication.

It is not optional in the sense of "safe to omit at recovery time if you used
one at setup time." If a passphrase was used when the wallet was created, it is
required for recovery. Omitting it at recovery time does not produce an error;
it derives the empty-passphrase key tree, which is a completely different wallet
with a different root fingerprint, different addresses, and different private
keys. Funds in the passphrase-derived tree are not accessible from the empty-
passphrase tree and vice versa.

It is not recoverable from the mnemonic. There is no mathematical relationship
```

```
between the mnemonic and the passphrase. Given the mnemonic and a set of funded
addresses, there is no faster path to finding the passphrase than trying every
candidate value. If the passphrase is forgotten and no backup exists, the funds
in the passphrase-derived tree are permanently inaccessible by any means,
including by the authors of this software.

It is not validated by the BIP39 checksum. The BIP39 checksum covers the
mnemonic entropy bits only. It has no knowledge of the passphrase. A wrong
passphrase produces no checksum error, no validation warning, and no
computationally detectable signal of any kind other than a root fingerprint
mismatch against a previously recorded value.
```

Cryptographic Effect of the Passphrase

Every distinct passphrase string - including the empty string - produces a completely independent 512-bit seed from the same mnemonic. The seeds are unrelated: knowing the mnemonic and the seed derived from passphrase A gives no computational advantage in finding the seed derived from passphrase B, regardless of any relationship between A and B.

This independence is a consequence of PBKDF2's pseudorandom function properties. PBKDF2-HMAC-SHA512 is a keyed pseudorandom function family. Changing a single byte of the salt - which includes the passphrase - produces an output that is computationally indistinguishable from a random 64-byte string by any adversary who does not know the salt value. Two passphrases that differ by a single character produce seeds that share no bits in a predictable pattern.

Practical consequences:

```
"password" and "Password" derive completely different wallets. The passphrase
is case-sensitive. There is no case folding or normalization applied beyond
Unicode NFKD decomposition (described below). A capital letter in the wrong
position is not a similar passphrase; it is a different passphrase.

"password " (with a trailing space) and "password" (without) derive completely
different wallets. The passphrase is whitespace-sensitive. Leading and trailing
spaces are not stripped. No normalization of any kind is applied to whitespace.

"café" entered on a keyboard that produces the precomposed U+00E9 (é as a
single code point) and "café" entered on a keyboard that produces the
decomposed U+0065 U+0301 (e followed by combining acute accent) derive the
same wallet. Unicode NFKD decomposition is applied to the passphrase before
byte-encoding, as specified by BIP39. Visually identical strings that differ
only in Unicode composition form are normalized to the same byte sequence.
This is the only normalization applied.

No other normalization is performed. The passphrase is not lowercased, trimmed,
or otherwise transformed. What is typed is what is used, after NFKD
decomposition.
```

Unicode NFKD Normalization

BIP39 mandates Unicode NFKD (Normalization Form Compatibility Decomposition) for both the mnemonic and the passphrase before byte-encoding. The normalization is applied by normalizeNFKD() in the HTML version and by unicodedata.normalize('NFKD', ...) in the Python version.

NFKD normalization does two things:

```
Canonical decomposition: composed characters are decomposed into their base
characters and combining characters. U+00E9 (é, precomposed) becomes U+0065
U+0301 (e + combining acute accent). U+00C5 (Å, precomposed) becomes U+0041
U+030A (A + combining ring above).

Compatibility decomposition: compatibility variants are mapped to their
canonical equivalents. The ligature U+FB01 (■) becomes U+0066 U+0069 (fi).
The superscript U+00B2 (²) becomes U+0032 (2). The full-width U+FF41 (■)
becomes U+0061 (a).
```

For passphrases composed entirely of ASCII characters (letters, digits, spaces, punctuation in the ASCII range), NFKD normalization is an identity transformation: all ASCII code points are already in their canonical decomposed form with no compatibility variants. The NFKD-normalized and non-normalized byte sequences are identical for any ASCII-only passphrase.

For passphrases that contain non-ASCII Unicode characters, the behavior of NFKD normalization must be considered carefully:

```
Any character with a compatibility equivalent will be silently transformed.
A passphrase typed on a keyboard that produces full-width characters (common on
some CJK input methods) will be normalized to their half-width equivalents. A
passphrase containing ligatures will have them split. If the passphrase was
```

```
created on a system that does not apply NFKD normalization before storage, and
the original unnormalized byte sequence was what was intended, the BIP39-
compliant derivation using NFKD normalization will produce a different seed.

The NFKD normalization requirement is part of the BIP39 specification and is
applied by all conformant BIP39 implementations. It is not a behavior specific
to Arculus Recovery. Any hardware wallet that follows BIP39 applies NFKD
normalization to the passphrase before seed derivation. If the passphrase was
originally set on a conformant hardware wallet, the same NFKD normalization
will have been applied at setup time, and re-entering the same passphrase in
Arculus Recovery will produce the correct seed.

The operational recommendation is to use ASCII-only passphrases. NFKD
normalization over ASCII is a no-op, and the passphrase will produce identical
byte sequences on any conformant platform regardless of Unicode composition
form or platform-specific normalization settings.
```

The Empty Passphrase

The empty string is a valid BIP39 passphrase. It is the default when no passphrase is entered. The BIP39 salt in this case is:

```
salt_bytes = "mnemonic".encode("utf-8")   -- 8 bytes
```

A wallet set up without a passphrase derives from this empty-passphrase seed. Most hardware wallets that support BIP39 passphrases use the empty passphrase by default and require an explicit opt-in to use a non-empty one.

The empty passphrase and any non-empty passphrase derive completely disjoint key trees. A wallet that holds funds under the empty-passphrase tree has no funds accessible from any non-empty passphrase, and vice versa. There is no "master" wallet that contains both; they are independent derivations.

In the Passphrase field in Arculus Recovery, leaving the field empty and entering a passphrase that happens to be the empty string are equivalent: both result in the empty-passphrase derivation. There is no sentinel value, null encoding, or special handling for the empty case. The field value is used directly as the passphrase string, and the empty string is a fully valid passphrase value.

When recovering a wallet, the correct starting assumption is that no passphrase was used unless there is specific knowledge or documentation to the contrary. Deriving with an empty passphrase first and comparing the root fingerprint against a recorded value is the fastest path to confirming this assumption.

Independence from the .arc Encryption Password

The BIP39 passphrase and the .arc file encryption password are independent secrets that operate on unrelated key schedules. They are not derived from each other. Neither can be inferred from the other. Each must be preserved and recalled independently.

The .arc file stores the mnemonic under encryption. It does not store the passphrase. When a .arc file is exported, the passphrase in the Passphrase field at export time is not recorded anywhere in the file. Decrypting a .arc file recovers only the mnemonic. The passphrase must be remembered or retrieved from a separate backup and re-entered manually into the Passphrase field before derivation is run.

The two passwords differ structurally:

```
.arc encryption password:
  Used as input to PBKDF2-HMAC-SHA512 at 1,000,000 iterations to derive the
  encryption and authentication keys for the .arc file. Governs whether the
  .arc file can be decrypted. An incorrect .arc password produces a MAC
  authentication failure and decryption does not proceed.

BIP39 passphrase:
  Used as input to PBKDF2-HMAC-SHA512 at 2,048 iterations as the salt
  component of BIP39 seed derivation. Governs which key tree is derived from
  the mnemonic. An incorrect BIP39 passphrase produces no error; it derives a
  different valid-looking key tree silently.
```

The error behavior is inverted: a wrong .arc password is detected immediately and visibly; a wrong BIP39 passphrase is undetectable without a reference fingerprint or known address to compare against. This asymmetry makes the BIP39 passphrase operationally more hazardous than the .arc encryption password.

The two passwords may be set to the same string value by user choice. This does not cause any cryptographic conflict; the engine applies them on entirely separate key schedules with different parameters and different input contexts. Reusing the

same string for both is a user decision; the tool does not prevent it, warn about it, or encourage it. The operational consequence of reusing them is that compromise of one compromises the other.

Root Fingerprint as Passphrase Verification

The root fingerprint is the only reliable mechanism for verifying that the correct passphrase has been entered during a recovery session. It is defined as:

```
root_pubkey      = serP(master_private_key x G)   -- 33-byte compressed
root_fingerprint = HASH160(root_pubkey)[0:4]       -- 4-byte hex string
```

The fingerprint is derived deterministically from the mnemonic and passphrase together. Two sessions using the same mnemonic and passphrase always produce the same fingerprint. Two sessions using the same mnemonic but different passphrases produce different fingerprints with overwhelming probability.

The root fingerprint is displayed in the Arculus Recovery toolbar and is updated immediately and asynchronously after every change to the mnemonic state or after derivation completes. It is also included in every JSON and TXT export and in the PDF title block.

To use the root fingerprint as a passphrase verification mechanism:

```
At wallet setup or first recovery: record the root fingerprint immediately
after validating the mnemonic and entering the passphrase. The fingerprint is
not secret; it reveals no recoverable key material. It may be stored alongside
the mnemonic backup, alongside the .arc file, or in any accessible location.
Its sole purpose is to confirm that a given mnemonic and passphrase combination
reproduce the expected key tree.

At recovery: after entering the mnemonic and passphrase, compare the displayed
root fingerprint against the recorded value before exporting any derived output
or acting on any derived address. A fingerprint mismatch is the only signal
that a wrong passphrase was entered. In the absence of a recorded fingerprint,
comparison against a known funded address from the wallet's on-chain history
is the fallback verification mechanism.
```

The root fingerprint is a 4-byte (8 hex character) value. Fingerprint collisions - two different (mnemonic, passphrase) pairs producing the same fingerprint - are possible with probability approximately 1/2^32 per pair. For any practical recovery scenario, a fingerprint match constitutes reliable confirmation.

Passphrase Entry in the HTML Interface

The Passphrase field in Arculus Recovery v1.5.0 is a password-type input that masks its contents by default. This behavior was introduced in v1.4.0 to prevent shoulder-surfing exposure of the passphrase during session setup. The behavior is shared by the standalone HTML application and GUI surfaces that render the same HTML asset, including the PySide6 desktop GUI and the Tauri wrapper.

Visibility toggle:

```
An eye-icon button is embedded in the right edge of the Passphrase field. It
toggles the field between masked (type="password", contents displayed as
bullets) and plain-text (type="text", contents visible) display.

Masked state:   button aria-label is "Show passphrase"
Plain-text state: button aria-label is "Hide passphrase"

The toggle state is not persisted between sessions. The field always begins in
the masked state when the page loads or when Clear All is used.

Use the plain-text state only to verify that the passphrase was entered
correctly, and return to masked state before any period in which the screen
may be visible to others. Minimize the duration for which the passphrase is
displayed in plain text.
```

Behavioral notes:

```
The Passphrase field does not apply any normalization to its displayed value.
The NFKD normalization described above is applied to the string at the point
it is consumed by the derivation pipeline (mnemonicToSeed and related
functions), not at the point of entry. The field displays the raw typed string.

The Passphrase field is cleared by Clear All. It is not cleared by switching
the coin or derivation path. It persists in the field until explicitly cleared
or until the page is closed.
```

```
  The Passphrase field does not participate in the hidden-seed workflow. It is
  always readable in the DOM when in plain-text toggle state. On a machine where
  screen capture or DOM inspection is possible, the passphrase is readable in
  plain text if the visibility toggle is enabled.

  The inactivity auto-clear timer (5-minute idle timeout, introduced in
  v1.4.0) clears the Passphrase field as part of its full-session wipe. This
  provides an automatic recovery from a session that was left unattended.
```

Common Passphrase Errors and Their Consequences

The following errors are silent in all cases. They produce no error message, no warning, and no computationally detectable signal other than a root fingerprint mismatch or failure to locate known addresses.

```
Error: Entering a passphrase when none was used at wallet setup.
  Consequence: The derived key tree is completely different from the wallet that
    holds funds. All derived addresses and private keys are for an empty wallet.
    The tool produces no error. The root fingerprint will differ from any
    previously recorded value.
  Detection: Root fingerprint mismatch, or failure to locate any known funded
    address in the derived output.
  Resolution: Clear the Passphrase field and re-derive. Compare the root
    fingerprint against the recorded value.

Error: Omitting the passphrase when one was used at wallet setup.
  Consequence: Identical to the above. The empty-passphrase derivation produces
    a completely different key tree. The tool produces no error.
  Detection: Root fingerprint mismatch, or failure to locate any known funded
    address in the derived output.
  Resolution: Re-enter the passphrase and re-derive. If the passphrase is unknown,
    refer to the passphrase recovery procedure in Recovery.txt.

Error: Case error in a passphrase composed of mixed-case letters.
  Consequence: A single character in the wrong case produces a completely
    different 512-bit seed. All downstream key material is unrelated to the
    wallet that holds funds. No error is produced.
  Detection: Root fingerprint mismatch.
  Resolution: Systematically try each candidate capitalization. If the passphrase
    is short and the location of the error is suspected, the search space may be
    manageable. For a long passphrase where the error location is unknown, the
    search space grows exponentially with passphrase length.

Error: Extra or missing whitespace in the passphrase.
  Consequence: A trailing space, leading space, or doubled space produces a
    completely different seed. Whitespace is not normalized. No error is produced.
  Detection: Root fingerprint mismatch.
  Resolution: Try the passphrase with and without leading/trailing whitespace.
    If the passphrase was copied from a source that may have added whitespace,
    check the source carefully.

Error: Passphrase entered with a different keyboard layout than at setup time.
  Consequence: Characters that map to different code points on different keyboard
    layouts produce different byte sequences. A passphrase containing characters
    outside the invariant ASCII set (i.e., characters other than A-Z, a-z, 0-9,
    and common punctuation) may silently produce different byte sequences on
    different keyboards, regardless of visible appearance.
  Detection: Root fingerprint mismatch.
  Resolution: Identify the keyboard layout used at setup time. If the setup
    machine is still available, re-enter the passphrase on it and record the
    root fingerprint. Use ASCII-only passphrases for all future wallets to
    eliminate keyboard layout dependency.

Error: Passphrase forgotten entirely.
  Consequence: Permanent loss of access to all funds in the passphrase-derived
    key tree. There is no recovery path from a forgotten passphrase. The
    passphrase is not stored in the .arc file, is not derivable from the
    mnemonic, and cannot be reconstructed from the derived addresses or keys.
  Detection: The passphrase is not known. Root fingerprint cannot be matched.
  Resolution: None available through the tool. If the passphrase was a short,
    low-entropy string and a partial description of it is remembered, a manual
    search may be feasible. If the passphrase was a long, random, high-entropy
    string and no backup exists, the funds are permanently inaccessible.
```

Passphrase Backup Requirements

The passphrase must be backed up independently of the mnemonic. A mnemonic backup without the passphrase recovers only the empty-passphrase key tree. If all funded addresses are in a passphrase-derived tree, a mnemonic-only backup provides no access to funds.

The following backup architecture is the minimum acceptable for a wallet that uses a passphrase:

```
Backup 1 - Mnemonic (primary seed backup):
  The mnemonic, recorded in permanent ink on archival paper or engraved on
  metal. Stored in a fire-resistant container at a separate physical location
  from the hardware wallet. This backup provides access to the empty-passphrase
  key tree only. It does not provide access to funded addresses in the
  passphrase-derived tree without the passphrase.

Backup 2 - Passphrase (separate from mnemonic):
  The passphrase recorded in the same physical form as the mnemonic backup, but
  stored at a different physical location from both the mnemonic backup and the
  hardware wallet. No single physical location should hold both the mnemonic
  and the passphrase. An adversary who obtains both can reconstruct the wallet
  without any other material.

Backup 3 - Root fingerprint (stored alongside both backups):
  The root fingerprint for the mnemonic and passphrase combination. The
  fingerprint is not secret; it cannot be used to reconstruct any key material.
  It serves as a verification checksum to confirm that the mnemonic and
  passphrase have been correctly recorded and recalled. Store it in both backup
  locations: alongside the mnemonic backup and alongside the passphrase backup.

Backup 4 - .arc file (optional, encrypted digital backup of mnemonic):
  A .arc file stores and protects the mnemonic only. If a .arc backup is used,
  the passphrase backup (Backup 2) remains independently required. Recovering
  from the .arc file and omitting the passphrase at derivation time recovers
  only the empty-passphrase tree.
```

The passphrase backup must be verified at creation time using the same round-trip procedure defined for .arc files in OpSec.txt: enter the mnemonic and the passphrase, confirm the root fingerprint matches the recorded value, and confirm at least one known address appears in the derivation output. A passphrase backup that has not been verified by this procedure may contain a transcription error that renders it useless at recovery time.

The passphrase backup should be verified periodically (annually at minimum) using the same procedure, for the same reason that .arc files are re-verified: to confirm the backup is readable and the passphrase is still correctly recalled. A passphrase that was correct five years ago and has since been partially misremembered is indistinguishable from a wrong passphrase at recovery time.

Entropy Considerations for Passphrase Choice

The BIP39 passphrase provides two security properties that are often conflated:

```
Property 1 - Wallet segregation (plausible deniability):
  A single mnemonic can be used with multiple passphrases to create independent
  wallets. An adversary who obtains the mnemonic under coercion has no way to
  determine how many passphrases exist or which one controls the primary funds.
  A small amount of funds can be kept in the empty-passphrase wallet as a
  decoy; the primary funds are in a passphrase-derived wallet unknown to the
  adversary.

  For this use case, the passphrase does not need to be high-entropy. It needs
  to be unknown to the adversary and unknown from the mnemonic alone. Even a
  short passphrase that is not publicly known provides effective deniability,
  because the adversary cannot distinguish an empty wallet from a wallet that
  simply has no funds yet.

Property 2 - Cryptographic hardening against mnemonic compromise:
  If the mnemonic is obtained by an adversary - through theft of the paper
  backup, hardware wallet compromise, or other means - the passphrase prevents
  the adversary from accessing funds in the passphrase-derived tree until the
  passphrase is also obtained. The strength of this protection is a direct
  function of passphrase entropy.

  For this use case, the passphrase must be high-entropy. An adversary who
  obtains the mnemonic can attempt to brute-force the passphrase offline, with
  no rate limiting and no detection mechanism, because PBKDF2-HMAC-SHA512 at
  only 2,048 iterations - the BIP39 KDF parameter - provides negligible
  resistance to GPU-accelerated guessing. A short, low-entropy passphrase
```

```
(a single word, a name, a date, a common phrase) provides no meaningful
protection against a dedicated adversary.

The entropy threshold for effective hardening against offline brute-force is
substantially higher than what BIP39's 2,048-iteration KDF protects. A
passphrase of five or more randomly selected words from a large wordlist, or
a randomly generated alphanumeric string of 16 or more characters, provides
a search space large enough to make exhaustive attack infeasible regardless
of GPU availability.
```

The two use cases have conflicting backup risk profiles. A high-entropy passphrase that is difficult to brute-force is also more difficult to remember and more likely to be forgotten or transcribed incorrectly. A short, memorable passphrase that is easy to recall provides deniability but limited brute-force resistance. The choice between these trade-offs is a user decision that depends on the threat model. This document does not prescribe a specific choice; it documents the consequences of each approach so the decision can be made with full information.

In both cases, the backup requirement is identical: the passphrase must be recorded in a durable, physically secured backup that is stored separately from the mnemonic, and the backup must be verified against a known root fingerprint before it is relied upon for recovery.

Arculus Recovery v1.5.0 ? File Format Reference

Scope

This document is the authoritative structural reference for every file format produced or consumed by Arculus Recovery v1.5.0. It covers the .arc encrypted seed file in all supported versions, the JSON derived-output format and its nested schema in full, the CSV and TXT flat export formats, the PDF export layout, and the plain-text export file naming conventions.

This document is complementary to Encryption.txt, which specifies the cryptographic construction inside the .arc envelope at the byte level, and to Derivation.txt, which specifies the key material that populates the derived- output formats. Readers needing the cryptographic rationale for .arc encryption parameters, KDF selection, or MAC transcript construction should consult Encryption.txt. Readers needing derivation path semantics, script type resolution, or address encoding rules should consult Derivation.txt. This document is limited to the structural, schema, and serialization definitions of each format.

All formats except PDF are UTF-8 encoded. No byte-order mark is written or expected. Line endings in plain-text formats are platform-dependent in Python output and LF-only in HTML output; importers should accept both. All formats are self-contained single files. No multi-file bundles or external schema references are required to read or write any format defined here.

.arc Encrypted Seed Format

A .arc file stores a single BIP39 mnemonic phrase in an encrypted, integrity- authenticated envelope. The format exists in three generations. Only the current armored V2 format is produced by new exports. All three generations are accepted on import. The format is identical whether produced by the HTML or Python implementation.

Version identifier summary:

```
Current export format:     ARCULUS-ARC-V2 (armored)
Legacy import ? JSON V2:   arculus-encrypted-seed-v2 (raw JSON)
Legacy import ? JSON V1:   arculus-encrypted-seed-v1 (HTML only)
Legacy import ? Python V1: arculus-encrypted-seed-python-v1
```

The version is detected automatically during import from structural indicators described in the format detection section below. No user selection is required.

Current Format ? ARCULUS-ARC-V2 (Armored)

The armored V2 format is a three-line UTF-8 text file:

```
Line 1: "ARCULUS-ARC-V2"
Line 2: <base64-encoded compact JSON bundle>
Line 3: (empty, trailing newline)
```

Line 1 is the armor header. It is a fixed ASCII string. Its presence as the first line is the sole trigger for armored V2 detection. No other format begins with this string.

Line 2 is a standard base64 encoding (RFC 4648, alphabet A-Z/a-z/0-9/+/, with = padding) of the UTF-8 compact JSON bundle. The JSON is serialized with no indentation and no extraneous whitespace. Key ordering follows the field sequence documented in the bundle schema below.

Line 3 is a trailing newline. Importers must accept files with or without the trailing newline.

The base64 encoding on line 2 is not a security control. It provides a consistent single-line serialization and prevents casual inspection or accidental editing of the binary-embedded fields (salt, nonce, ciphertext, MAC). The security boundary is the MAC verification gate in the decryption procedure, not the base64 armor.

V2 Bundle Schema (Internal JSON)

After base64 decoding, the V2 bundle is a JSON object with the following fields. All fields are required. An importer that encounters a bundle missing any of these fields must reject the file with a structural validation error before attempting decryption.

```
Field           Type      Description
--------------- --------- ------------------------------------------------------
magic           string    Fixed value "ARCULUS-ARC". Identifies the file as an
                          Arculus seed bundle. Not version-specific.
format          string    Fixed value "arculus-encrypted-seed-v2". Identifies
                          the internal format variant.
```

version	integer	Fixed value 2. Integer, not string. Used to gate legacy importer dispatch.
created_at	string	UTC ISO 8601 timestamp of the export event, e.g. "2026-06-05T14:32:00.000Z". This field is a MAC-bound metadata commitment, not a user-editable annotation. Any modification to this field invalidates the MAC.
cipher	object	Cipher parameter block. Fields defined below.
kdf	object	KDF parameter block. Fields defined below.
ciphertext_b64	string	Base64-encoded ciphertext bytes. The ciphertext is the HMAC-SHA512-CTR keystream XOR of the UTF-8 encoded plaintext JSON payload.
mac_b64	string	Base64-encoded 64-byte HMAC-SHA512 MAC over the versioned metadata transcript. See Encryption.txt for the exact transcript construction.

The cipher sub-object:

Field	Type	Description
name	string	Fixed value "HMAC-SHA512-CTR". Stream cipher identifier.
nonce_b64	string	Base64-encoded 24-byte cryptographically random nonce, generated fresh by CSPRNG on every export.

The kdf sub-object:

Field	Type	Description
name	string	Fixed value "PBKDF2".
hash	string	Fixed value "SHA-512".
iterations	integer	KDF iteration count. 1,000,000 for new exports. Imports accept any value >= 600,000. Values below 600,000 are rejected as below the minimum hardening floor.
salt_b64	string	Base64-encoded 32-byte cryptographically random salt, generated fresh by CSPRNG on every export.

V2 Plaintext Payload Schema

After successful MAC verification and decryption, the recovered plaintext is a UTF-8 encoded JSON object. The current schema:

Field	Type	Required	Description
mnemonic	string	yes	Space-separated BIP39 mnemonic words, Unicode NFKD-normalized. 12 or 24 words.
word_count	integer	yes	Integer word count. Must equal the number of space-delimited tokens in the mnemonic field.
created_at	string	yes	UTC ISO 8601 timestamp. Matches the outer bundle created_at field; included in the plaintext as a convenience cross-reference.

Importers must ignore unknown fields at the top level of the plaintext payload. Future versions may add fields without changing the format version. An importer that rejects unknown plaintext fields will fail on files produced by future versions that add optional metadata.

The mnemonic field value is Unicode NFKD-normalized before encryption. On decryption and import, the recovered string is used directly for BIP39 seed derivation without any additional normalization step by the importer.

File Format Detection and Importer Dispatch

The import pipeline applies the following detection sequence to determine which decryption path to invoke. Detection is structural and requires no password.

```
Step 1 ? Check for armor header:
  If the file text, after stripping leading and trailing whitespace, begins
  with the exact string "ARCULUS-ARC-V2\n" or "ARCULUS-ARC-V2\r\n", dispatch
  to the armored V2 import path. This is the current production format.

Step 2 ? Attempt raw JSON parse:
  If no armor header is detected, attempt to parse the file text as JSON.
  If parsing fails, reject the file as unrecognized format.

Step 3 ? Inspect parsed JSON for magic and version:
  If the JSON object contains the field magic == "ARCULUS-ARC" and
  version == 2, dispatch to the raw JSON V2 import path. This path is
  identical to the armored V2 path after the armor-strip and base64-decode
  steps; the bundle schema is the same.
```

```
  Step 4 ? Inspect for format field (legacy V1, HTML only):
    If the JSON object contains format == "arculus-encrypted-seed-v1",
    dispatch to the legacy V1 AES-GCM import path. This path is implemented
    in the HTML version only. The Python implementation does not support V1.

  Step 5 ? Inspect for Python V1 sentinel field:
    If the JSON object contains format == "arculus-encrypted-seed-python-v1"
    or an equivalent Python-V1 sentinel, dispatch to the Python V1 legacy
    import path.

  Step 6 ? Attempt legacy headerless import:
    If none of the above conditions match, attempt decryption as a legacy
    headerless PBKDF2-SHA256 + XOR-HMAC file. This covers the earliest export
    format predating the format field and magic marker.

  If all steps fail, the file is rejected with a format error before any
  password is requested or any cryptographic operation is attempted.
```

Legacy Formats ? Import-Only

The following formats are accepted on import but are never produced by new exports. Users holding files in these formats should re-export to the current armored V2 format to apply current KDF hardening and standardize the storage representation.

```
Legacy V2 raw JSON:
  Structurally identical to the V2 bundle schema above but stored as a plain
  JSON file rather than armored text. Detected by magic == "ARCULUS-ARC" and
  version == 2 without the armor header. Produced by earlier builds before
  the armored text envelope was introduced. Accepted without restriction
  subject to the minimum KDF iterations floor of 600,000.

Legacy V1 AES-GCM (HTML only):
  An earlier HTML-only format that used the browser SubtleCrypto AES-GCM
  primitive directly. Format field value: "arculus-encrypted-seed-v1".
  Not supported by the Python implementation. Import is supported in the
  HTML version for backward compatibility. Re-export to armored V2 is strongly
  recommended because this format uses a different cipher construction with
  shorter authentication tags and does not commit the KDF iteration count
  into the MAC transcript.

Legacy Python V1:
  The earliest Python export format. Format field value:
  "arculus-encrypted-seed-python-v1". Uses PBKDF2-HMAC-SHA256 and a
  simpler XOR-HMAC authentication scheme with a different transcript
  structure. Accepted for import only. Re-export to armored V2 applies the
  current PBKDF2-HMAC-SHA512 construction and the full metadata MAC commitment.

Legacy headerless format:
  The oldest format, predating explicit format and magic fields. Detected as
  a fallback after all named-format checks fail. Uses PBKDF2-HMAC-SHA256 with
  a 32-byte salt and a simple XOR stream keyed on the full KDF output. The
  authentication tag covers only the ciphertext, not KDF parameters. Accepted
  for import only.
```

In all legacy import paths, the decrypted mnemonic is surfaced in the same hidden-seed workflow as a current-format import. The import path is transparent to the user; the same password entry and root fingerprint verification procedure applies regardless of the underlying format.

.arc File Naming

The tool produces .arc files with the following default filename pattern:

```
  arculus_seed_<YYYYMMDD_HHMMSS>.arc
```

The timestamp component is derived from the export creation time in UTC. The filename is not security-relevant. The file may be renamed freely without affecting decryptability. The format is identified entirely by its internal structure and armor header, not by its filename or extension.

The .arc extension is a project convention. The file is plain UTF-8 text and can be opened, copied, and transmitted like any other text file. The extension serves only as a visual indicator of the file type.

Derived-Output JSON Format

JSON is the primary export format for derived keys and addresses. It is the default output format in all CLI invocations that do not specify --output-format. It is one of four format options in the HTML and Python GUI export selector.

The JSON output is a single top-level object with the following structure:

```
Top-level object:

  Field             Type    Description
  ----------------- ------- ------------------------------------------------
  coin              string  Coin identifier: "bitcoin", "litecoin",
                            "dogecoin", "ethereum", or "xrp".
  network           string  Network identifier: "mainnet" or "testnet".
  mnemonic          string  The BIP39 mnemonic used for this derivation run.
                            Present in JSON output. Treat as operationally
                            sensitive; see security note at end of section.
  word_count        integer Number of mnemonic words: 12 or 24.
  derivation        string  The account-level derivation path used.
  account_script_type_used  string  Resolved script type after auto-inference,
                            if applicable. One of: p2pkh, p2wpkh-p2sh,
                            p2wpkh, p2tr, xrp, ethereum.
  root_fingerprint  string  4-byte hex fingerprint of the BIP32 master node.
                            Derived from HASH160 of the master public key.
  accounts          array   Array of one or more account objects. See below.

Account object (one entry per account path derived):

  Field             Type    Description
  ----------------- ------- ------------------------------------------------
  account_path      string  Full BIP32 account path including hardened indices.
  script_type       string  Script type for this account entry.
  account_xprv      string  Account-level extended private key in base58check.
  account_xpub      string  Account-level extended public key in base58check.
  account_zprv      string  P2WPKH extended private key (zprvA... form).
                            Present only when script type is p2wpkh or auto
                            with m/84' purpose.
  account_zpub      string  P2WPKH extended public key (zpub... form).
  account_yprv      string  P2WPKH-P2SH extended private key (yprvA... form).
                            Present only when script type is p2wpkh-p2sh or
                            auto with m/49' purpose.
  account_ypub      string  P2WPKH-P2SH extended public key (ypub... form).
  account_trprv     string  Taproot extended private key (trprvA... form).
                            Present only when script type is p2tr or auto
                            with m/86' purpose.
  account_trpub     string  Taproot extended public key (trpub... form).
  root_xprv         string  Root-level master extended private key (xprv).
  root_xpub         string  Root-level master extended public key (xpub).
  root_zprv         string  Root-level P2WPKH extended private key.
  root_zpub         string  Root-level P2WPKH extended public key.
  root_yprv         string  Root-level P2WPKH-P2SH extended private key.
  root_ypub         string  Root-level P2WPKH-P2SH extended public key.
  root_trprv        string  Root-level Taproot extended private key.
  root_trpub        string  Root-level Taproot extended public key.
  receiving         array   Address records for branch index 0.
  change            array   Address records for branch index 1.

Extended key fields that are not applicable to a given coin or script type
are omitted from the output rather than set to null. An importer that
requires a field not present for a given combination should treat absence as
equivalent to null.

Address record ? UTXO coins (P2PKH, P2WPKH-P2SH, P2WPKH):

  Field             Type    Description
  ----------------- ------- ------------------------------------------------
  path              string  Full child derivation path, e.g. m/84'/0'/0'/0/3
  branch            integer Branch index: 0 for receiving, 1 for change.
  address           string  Encoded address in the canonical format for the
                            script type and coin.
  public_key_hex    string  33-byte compressed public key as lowercase hex.
  private_key_hex   string  32-byte private scalar as lowercase hex.
  private_key_wif   string  Private key in Wallet Import Format (WIF).
                            Base58check encoded with the coin-specific version
                            byte and compression flag.

Address record ? P2TR (Taproot), augmented:

  All fields above, plus:

  Field                       Type    Description
  --------------------------- ------- --------------------------------------
  taproot_internal_public_key_hex   string  32-byte x-only internal public key hex.
  taproot_internal_private_key_hex  string  32-byte internal private scalar hex.
```

```
  taproot_tweak_hex                string  32-byte TapTweak scalar hex.
  taproot_output_public_key_hex    string  32-byte x-only output public key hex.
  taproot_output_private_key_hex   string  32-byte tweaked output private scalar hex.
  taproot_output_private_key_wif   string  Tweaked output private key in WIF.
  taproot_output_key_parity        integer Y-parity of the output public key: 0 or 1.

Address record ? Ethereum:

  Field                   Type    Description
  ----------------------- ------- ------------------------------------------
  path                    string  Full child derivation path.
  branch                  integer Branch index: 0 or 1.
  address                 string  EIP-55 checksummed Ethereum address (0x...).
  ethereum_public_key_hex string  33-byte compressed public key as hex.
  private_key_hex         string  32-byte private scalar as hex.
  erc20_address           string  Same value as address. ERC-20 tokens on
                                  Ethereum share the same address space.
  erc20_note              string  Advisory string confirming ERC-20 token
                                  compatibility. Content is informational only.

Address record ? XRP:

  Field               Type    Description
  ------------------- ------- ------------------------------------------------
  path                string  Full child derivation path.
  branch              integer Branch index: 0 or 1.
  xrp_classic_address string  XRPL classic address (r...).
  public_key_hex      string  33-byte compressed public key as hex.
  private_key_hex     string  32-byte private scalar as hex.
  xrp_note            string  Advisory string regarding destination tags.
                              Content is informational only.
```

Security note: The JSON export contains the source mnemonic in the mnemonic field and all derived private keys in their respective private_key_hex and private_key_wif fields. The JSON file is operationally equivalent in sensitivity to the mnemonic itself. It must be treated as Category B key material as defined in OpSec.txt and handled under the same storage and transfer controls. It must never be written to a networked filesystem or transmitted over any channel.

Derived-Output CSV Format

The CSV export is a flat, row-per-address representation of the same data contained in the JSON export. It is intended for tabular inspection in a spreadsheet application or for programmatic filtering with standard text tools.

Structure:

```
Row 1 is a typed header row. Column names match the field names in the JSON
address record schema. Account-level fields (derivation path, script type,
account xpub, account xprv, root xpub, root xprv, and all slip-0132
variants) are repeated on every row belonging to that account.

Rows 2 through N are one address record per row. Receiving and change branch
entries appear in derivation index order within each branch, with receiving
branch entries preceding change branch entries within each account block.

Field values containing commas, double-quote characters, or newlines are
wrapped in double quotes and internal double-quote characters are escaped as
two consecutive double-quote characters, per RFC 4180.

Null or absent fields for a given coin and script type are represented as
empty cells.
```

The CSV export contains the same private key material as the JSON export and carries the same operational sensitivity classification. The same handling requirements apply.

Derived-Output TXT Format

The TXT export is a human-readable, labeled-section document intended for offline visual inspection or printing. It does not preserve the nested JSON hierarchy. Instead, it organizes output into titled sections and subsections separated by blank lines.

Structure:

```
A header block at the top of the file contains the coin name, network,
derivation path, script type, and root fingerprint. Each field appears on
its own labeled line with consistent left-column alignment.

Below the header, one titled section per account appears. Within each account
section, receiving and change addresses are grouped into labeled subsections.
```

```
Within each subsection, one address block per derived index appears, with each
field on its own labeled line.

Extended keys (xprv, xpub, zprv, zpub, yprv, ypub, trprv, trpub at both
root and account level) appear in a dedicated block at the start of each
account section before the address subsections.

Field labels are left-aligned and right-padded to a consistent column width.
Values begin at a fixed column offset. Long values (extended keys, hex strings)
are not wrapped or truncated; they appear on a single line.
```

The TXT export contains the same private key material and carries the same operational sensitivity classification as the JSON and CSV exports.

PDF Export Format

The PDF export is produced by the jsPDF library. In the standalone HTML implementation, jsPDF is statically embedded inside Arculus_Recovery.html so PDF export works from a raw local file with no network access, no cdnjs request, no vendor directory at runtime, and no package installation. The PySide6 desktop GUI renders the packaged HTML copy and may retain a local vendored jsPDF injection fallback for older HTML assets, but current HTML assets already carry the PDF library inline. The Python CLI does not produce PDF output. The PDF is intended as a printable hard-copy record. It is produced in landscape A4 orientation.

Page structure:

```
Every page begins with a title block containing:
  - Title: "Arculus Key Export"
  - Meta line: coin name, app version string, UTC export timestamp, and root
    fingerprint, separated by center-dot (?) delimiters.

Page 1 contains, after the title block:
  - An Extended Keys section listing all present root and account extended
    keys vertically, one per line. Each entry has a left-column label and
    the full key value spanning the remaining page width.
  - A horizontal rule separating the extended keys section from the address
    table.
  - The address table header and as many rows as fit on the first page.

Continuation pages contain, after the title block:
  - The address table header repeated.
  - Address table rows continuing from the previous page.

The Extended Keys section does not repeat on continuation pages.
```

Address table column layout:

```
UTXO coins (Bitcoin, Litecoin, Dogecoin):
  Branch | Path | Address | Private Key WIF

Ethereum and XRP:
  Branch | Path | Address | Private Key Hex

The Branch column uses color-coded text: green for receiving (branch index 0),
gray for change (branch index 1).

Rows use alternating background shading: white and a light gray, applied per
address row.

Column widths are fixed. No column is truncated. Extended keys in the Extended
Keys section wrap at the right page margin if the value exceeds the available
width, which does not occur for standard base58check-encoded keys at landscape
A4 dimensions.
```

The PDF is saved to disk with the filename:

```
Arculus_Key_Export_<Coin>.pdf
```

where <Coin> is the display-capitalized coin name: Bitcoin, Litecoin, Dogecoin, Ethereum, or XRP.

The PDF export contains the same private key material as the other export formats. A printed PDF is a physical key document. It must be stored with the same physical security as a paper mnemonic backup and must not be printed on a networked printer. The PDF file itself, before printing, carries the same Category B sensitivity classification as JSON, CSV, and TXT exports.

Export File Naming Conventions

The default filenames produced by the HTML export controls:

```
JSON:   arculus_keys_<coin>.json
CSV:    arculus_keys_<coin>.csv
TXT:    arculus_keys_<coin>.txt
PDF:    Arculus_Key_Export_<Coin>.pdf
```

where <coin> is lowercase (bitcoin, litecoin, dogecoin, ethereum, xrp) for JSON, CSV, and TXT, and display-capitalized for PDF.

The Python CLI writes to stdout by default. When output is redirected to a file, the filename is at the user's discretion. No default filename is imposed by the CLI. The content and schema of the output are identical to the HTML export for the same format selector.

The PySide6 GUI uses a native save dialog for GUI exports. PDF exports default to the PDF file filter and append .pdf when the extension is omitted. JSON, CSV, TXT, encrypted seed, and QR PNG exports retain their format-appropriate default extensions.

The Tauri wrapper packages the same HTML export serializers and includes a Rust save_export command that writes browser-generated Blob bytes to the user's Downloads directory. The packaged WebView receives an injected window.arculusTauriSaveExport bridge during initialization and uses that bridge before any browser-style download path. The Tauri v2 bridge is represented by src-tauri/capabilities/default.json and src-tauri/permissions/export.toml. These files are required for packaged desktop exports and must remain present when building Tauri artifacts.

Filenames are not structural identifiers. The format of a derived-output file is identified by its content, not its extension or name. A JSON export renamed to .txt is valid JSON. An importer that relies on the file extension rather than the content structure is non-conformant with this specification.

Version History of Supported Formats

This table maps tool version to the set of formats that version can produce (write) or consume (read) on import.

Tool version	Write	Read (import)
v1.5.0	armored V2	armored V2, raw JSON V2, V1 (HTML), Python V1, legacy headerless
v1.4.0-production	armored V2	armored V2, raw JSON V2, V1 (HTML), Python V1, legacy headerless
v1.3.0-production	armored V2	armored V2, raw JSON V2, V1 (HTML), Python V1, legacy headerless
v1.2.0-production	armored V2	armored V2, raw JSON V2, V1 (HTML), Python V1, legacy headerless
v1.1.0-production	armored V2	armored V2, raw JSON V2, V1 (HTML), Python V1, legacy headerless
Pre-v1.1.0	raw JSON V2 or V1	raw JSON V2, V1 (HTML), Python V1, legacy headerless

The armored V2 format was introduced in v1.1.0-production as the sole new-export format. All builds from v1.1.0-production onward write armored V2 exclusively and read all prior formats for backward compatibility. The -production channel suffix was dropped from the version string beginning with v1.5.0; the file format itself is unchanged between v1.4.0-production and v1.5.0.

A .arc file produced by v1.5.0 is readable by any tool version at or above v1.1.0-production, provided the tool version implements the armored V2 import path and the decryption password is known. There are no version-specific extensions to the armored V2 bundle schema between v1.1.0-production and v1.5.0. The format has been stable since its introduction.

Arculus Recovery v1.5.0 - Encryption Subsystem Internals

Scope

This document is the authoritative byte-level specification of the encrypted seed storage subsystem as implemented in Arculus Recovery v1.5.0. It defines the .arc file format in complete structural detail, the PBKDF2-based key schedule, domain-separated key derivation, the HMAC-SHA512 counter-mode keystream construction, the authenticated metadata transcript, the full export and import procedures, the constant-time MAC comparison requirement, the compatibility matrix for all supported legacy formats, iteration count rationale, and the precise boundary of what the .arc format protects and does not protect.

This document is an offline reference. All cryptographic constructions are expressed as byte-level procedures and pseudocode. No external dependencies, diagrams, or supplementary materials are required to implement or verify compliance.

Design Objectives

The .arc format was designed under three non-negotiable engineering constraints that admit no exceptions:

1. Zero external cryptographic runtime dependencies.
   Python uses exclusively hashlib, hmac, os.urandom, json, and base64 from the Python standard library. The HTML version uses browser built-ins exclusively: SubtleCrypto (PBKDF2, HMAC, SHA-512), crypto.getRandomValues, TextEncoder, TextDecoder, btoa, and atob. No third-party cryptographic package, compiled extension, or WebAssembly module is introduced for .arc encryption under any circumstances. The constraint is absolute for seed encryption and applies retroactively: any proposed change that introduces an external cryptographic dependency is rejected by design, not by policy.

2. Cross-platform byte-level compatibility.
   A .arc file produced by the HTML version must be decryptable by the Python version and vice versa without any intermediate conversion step. Both implementations derive byte-identical key material and produce byte-identical ciphertext for the same (plaintext, password, salt, nonce) tuple. The only implementation differences lie in WebCrypto versus hashlib API surface; the underlying cryptographic operations are specification-identical.

3. Encrypt-then-MAC with fully authenticated metadata.
   The authentication tag covers not only the ciphertext but all versioned metadata, KDF parameters, cipher parameters, nonce, salt, and creation timestamp as raw bytes. An adversary who obtains a .arc file cannot alter any field - including the KDF iteration count - without producing a MAC verification failure. This property holds unconditionally: it does not require the adversary's modification to be detectable by inspection of the file structure.

The scheme in a single-line summary:

```
password -> PBKDF2-HMAC-SHA512(1,000,000 iterations, 32-byte salt)
         -> 64-byte master key
         -> HMAC-SHA512 domain separation -> enc_key (64 bytes), mac_key (64 bytes)
         -> HMAC-SHA512-CTR stream cipher over UTF-8 plaintext JSON
         -> encrypt-then-MAC over fully versioned metadata transcript
         -> compact JSON bundle -> base64 armor -> ARCULUS-ARC-V2 text file
```

Why HMAC-SHA512-CTR and Not AES-GCM

The decision to construct a counter-mode stream from HMAC-SHA512 rather than use AES-GCM is architectural, not a cryptographic compromise.

Python's standard library does not expose AES-GCM. Introducing AES-GCM would require one of: the `cryptography` package (a compiled C extension with an OpenSSL binding and an external trust surface), PyCryptodome, or a pure-Python AES implementation. Each option introduces packaging complexity, cross-platform build variability, and a divergence between the Python and HTML implementations that would require a compatibility shim or a distinct code path.

HMAC-SHA512 is natively available in both Python's hashlib and the browser's SubtleCrypto.sign API. Building the stream cipher from HMAC-SHA512 preserves complete dependency freedom across both runtimes, permits a single unified algorithm specification with no platform-conditional branches, and delegates security entirely to HMAC-SHA512's well-analyzed pseudorandom function properties under the standard assumption that SHA-512 is a secure hash function.

The construction is encrypt-then-MAC: authentication is computed over the ciphertext, never over the plaintext. MAC-then-encrypt exposes padding and decryption oracle vulnerabilities in certain cipher configurations. The authenticate-then-decrypt order in the import procedure is non-negotiable and is enforced unconditionally: no decryption step is attempted before MAC verification succeeds.

The MAC transcript commits to all metadata fields, making the scheme a committed authenticated encryption construction without the overhead of a dedicated AEAD library. KDF downgrade attacks - in which an adversary modifies the stored iteration count to reduce brute-force cost - are defeated because the iteration count is a bound field in the MAC commitment.

AES-GCM would be a technically appropriate alternative if external dependencies were acceptable. They are not for this tool, and this design decision is final.

Executive Summary of Current Format

```
Armor header:             ARCULUS-ARC-V2
Internal format ID:       arculus-encrypted-seed-v2
Internal version integer: 2
Password normalization:   Unicode NFKD, then UTF-8 encoded
KDF algorithm:            PBKDF2-HMAC-SHA512
KDF output length:        64 bytes (512 bits)
KDF iterations (new):     1,000,000
KDF iterations (minimum): 600,000 (legacy v2 import acceptance floor)
Salt:                     32 bytes, CSPRNG-generated per export
Nonce:                    24 bytes, CSPRNG-generated per export
Stream cipher:            HMAC-SHA512 counter mode (HMAC-SHA512-CTR)
Authentication:           HMAC-SHA512 over versioned metadata transcript
MAC length:               64 bytes (512 bits)
Key separation:           domain-specific HMAC-SHA512 labels
Plaintext payload:        normalized mnemonic, word count, UTC ISO 8601 timestamp
```

Implementation Constants

These constants are defined identically in both the Python and HTML implementations. Any deviation between implementations - even in whitespace, capitalization, or encoding - produces incompatible key material or MAC values.

```
ARC_MAGIC                 = "ARCULUS-ARC"
ARC_ARMOR_HEADER          = "ARCULUS-ARC-V2"
ARC_V2_FORMAT             = "arculus-encrypted-seed-v2"
ARC_V2_VERSION            = 2
ARC_V2_KDF_NAME           = "PBKDF2"
ARC_V2_KDF_HASH           = "SHA-512"
ARC_V2_KDF_ITERATIONS     = 1_000_000
ARC_V2_MIN_KDF_ITERATIONS = 600_000
ARC_V2_SALT_BYTES         = 32
ARC_V2_NONCE_BYTES        = 24
ARC_V2_MAC_BYTES          = 64
ARC_V2_MASTER_KEY_BYTES   = 64
ARC_V2_CIPHER_NAME        = "HMAC-SHA512-CTR"
```

Domain-separation label constants (byte literals):

```
ENC_LABEL    = b"Arculus ARC v2 encryption key"      -- 29 bytes
MAC_LABEL    = b"Arculus ARC v2 authentication key"  -- 33 bytes
STREAM_LABEL = b"Arculus ARC v2 stream\x00"          -- 22 bytes
MAC_PREFIX   = b"Arculus ARC v2 MAC"                 -- 18 bytes
```

The null byte (0x00) terminating STREAM_LABEL provides a hard boundary between the label and the variable-length nonce || counter suffix that follows it, preventing label extension collisions with any input that begins with a partial match of the label string.

File Format: External Structure

A .arc file is UTF-8 text with the following three-line structure:

```
Line 1: "ARCULUS-ARC-V2"                    -- armor header, exact string
Line 2: base64( compact_json(bundle) )       -- RFC 4648 standard base64
Line 3: empty (trailing LF newline)
```

The armor header on line 1 is the format discriminator. If line 1 is not exactly "ARCULUS-ARC-V2", the importer falls through to the legacy detection path without raising an error.

Line 2 is standard RFC 4648 base64 (not base64url). It encodes a compact JSON object as specified in the Internal Bundle Structure section below. The resulting file is an opaque envelope: opening the file reveals no human-readable metadata about the key derivation parameters, the mnemonic length, or the creation date.

This opaque presentation is an intentional property. The armoring does not provide confidentiality - the MAC and keystream are the security boundary - but it eliminates casual inspection of sensitive operational parameters and discourages manual file editing that could produce subtly incorrect inputs to the KDF or cipher without triggering an immediate parsing error.

File Format: Internal Bundle Structure

After RFC 4648 base64 decoding of line 2, the result is a compact JSON object. The Python serializer produces it with separators=(",", ":") and sort_keys=True. The HTML serializer produces compact JSON via JSON.stringify. JSON key ordering is not a security property; the MAC transcript is constructed from explicit field extractions over raw bytes, not over the JSON string itself.

Full internal bundle schema (all fields present in every current export):

```
{
  "cipher": {
    "name":      "HMAC-SHA512-CTR",
    "nonce_b64": "<base64(24 bytes)>"
  },
  "ciphertext_b64": "<base64(N bytes, N = len(plaintext_utf8))>",
  "created_at":     "<ISO 8601 UTC timestamp, e.g. 2026-05-04T12:00:00.000Z>",
  "format":         "arculus-encrypted-seed-v2",
  "kdf": {
    "hash":       "SHA-512",
    "iterations": 1000000,
    "name":       "PBKDF2",
    "salt_b64":   "<base64(32 bytes)>"
  },
  "mac_b64":  "<base64(64 bytes)>",
  "magic":    "ARCULUS-ARC",
  "version":  2
}
```

All binary fields (salt, nonce, ciphertext, MAC) are RFC 4648 base64-encoded within this JSON object. The JSON bundle itself is then base64-encoded again for the armored file line. This intentional double-encoding separates concerns: the outer base64 layer produces the printable armor body; the inner base64 fields keep binary data safe within JSON string values without requiring binary JSON extensions.

Plaintext Payload

The content encrypted by this subsystem is intentionally minimal:

```
{
  "mnemonic":   "word1 word2 ... wordN",
  "word_count": 12,
  "created_at": "2026-05-04T12:00:00.000Z"
}
```

Only the BIP39 seed mnemonic is encrypted. The following are explicitly outside the scope of .arc encryption:

- The BIP39 wallet passphrase (a separate wallet seed derivation parameter)
- Derived private keys, WIF keys, private_key_hex fields
- Extended private keys (xprv, yprv, zprv, tprv)
- Extended public keys (xpub, ypub, zpub, tpub)
- Derived addresses (P2PKH, P2WPKH-P2SH, P2WPKH, P2TR, Ethereum, XRP)
- Ethereum account state, ERC-20 token metadata, allowances, or balances
- XRP classic addresses, X-addresses, destination tags, or sequence numbers
- JSON/CSV/TXT derived-output export files (these are always plaintext)

The .arc encryption password is operationally and cryptographically distinct from the BIP39 wallet passphrase. The system never conflates them. The BIP39 passphrase is a wallet-tree derivation parameter processed through a 2048- iteration PBKDF2 over "mnemonic" + passphrase. The .arc password is a file-at- rest protection parameter processed through a

1,000,000-iteration PBKDF2 over a fresh 32-byte random salt. Their key schedules, salts, iteration counts, and derived key material are entirely independent.

Importers must silently ignore unknown fields in the plaintext JSON object to preserve forward compatibility. Future format versions may add payload fields without breaking existing readers.

Key Schedule

The key schedule derives all per-operation keys from a single PBKDF2 master key and then separates them by cryptographic domain. No key material is shared or reused across domains. The schedule has four sequential steps.

Step 1 - Password normalization:

```
password_nfkd  = NFKD(password_string)
password_bytes = password_nfkd.encode("utf-8")

Unicode NFKD normalization (Compatibility Decomposition) ensures that different
Unicode representations of the same logical password string - e.g., composed
vs. decomposed diacritics, full-width vs. half-width characters, ligature vs.
component forms - produce the same byte sequence regardless of input platform
or keyboard method. Without this normalization, the same passphrase typed on
two different operating systems may produce different byte sequences, resulting
in a permanently inaccessible .arc file.
```

Step 2 - Master key derivation via PBKDF2-HMAC-SHA512:

```
master_key = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = salt,         -- 32 bytes from CSPRNG (fresh per export)
    iterations = 1_000_000,
    dklen      = 64            -- 512-bit output
)

The 64-byte master key is never used directly for any cryptographic operation.
It serves exclusively as a key derivation source. This intermediate indirection
ensures that compromise of any single derived operation key does not yield the
master key, and therefore does not yield the other derived keys.
```

Step 3 - Encryption key derivation:

```
enc_key = HMAC-SHA512(
    key = master_key,
    msg = b"Arculus ARC v2 encryption key"    -- ENC_LABEL
)

Output: 64 bytes. Used exclusively as the HMAC key in the HMAC-SHA512-CTR
keystream construction. This key is never reused outside that context.
```

Step 4 - Authentication key derivation:

```
mac_key = HMAC-SHA512(
    key = master_key,
    msg = b"Arculus ARC v2 authentication key"  -- MAC_LABEL
)

Output: 64 bytes. Used exclusively as the HMAC key in the MAC transcript
computation. This key is never reused outside that context.
```

enc_key and mac_key are cryptographically independent under the standard PRF assumption on HMAC-SHA512. Knowledge of enc_key yields no information about mac_key or master_key, and vice versa. The domain-separation labels ensure that even if an adversary constructs an input that produces a target HMAC output for one label, that output is computationally unrelated to the HMAC output under a different label.

HMAC-SHA512 Counter-Mode Keystream

The keystream is generated from enc_key and the 24-byte per-export random nonce using an indexed HMAC-SHA512 construction. For counter value i = 0, 1, 2, ...:

```
block_i = HMAC-SHA512(
    key = enc_key,
    msg = STREAM_LABEL || nonce || uint64_be(i)
)
```

Where:

```
STREAM_LABEL = b"Arculus ARC v2 stream\x00"   -- 22 bytes including null terminator
nonce        = 24 bytes (CSPRNG-generated per export)
uint64_be(i) = counter i encoded as an 8-byte big-endian unsigned integer
```

Each block_i is 64 bytes (512 bits). The keystream is the sequential concatenation of as many counter blocks as required to cover the full plaintext byte length:

```
keystream = block_0 || block_1 || block_2 || ... || block_{ceil(N/64)-1}
```

Only the first N bytes of the keystream are consumed, where N = len(plaintext).

Encryption and decryption are identical XOR operations over the keystream:

```
ciphertext[j] = plaintext[j]  XOR keystream[j]
plaintext[j]  = ciphertext[j] XOR keystream[j]
```

Security analysis of the construction:

```
- Each counter block input is uniquely bound to (STREAM_LABEL, nonce, i),
  preventing block reordering attacks and ensuring independent per-position
  pseudorandomness.
- The 24-byte nonce provides 192 bits of nonce space, making per-export nonce
  collision probability negligible under reasonable export volumes.
- The nonce binds the keystream to a specific export event; two exports of the
  same mnemonic under the same password but with different nonces produce
  completely independent ciphertexts.
- The keystream is never consumed without prior or subsequent MAC verification;
  ciphertext is not accepted as decryption input before MAC verification
  succeeds unconditionally.
```

MAC Transcript Construction

The MAC binds the ciphertext to all versioned file metadata. It is computed over an explicitly constructed byte transcript, not over the raw JSON bytes of the bundle. This design prevents serialization-order attacks and makes the commitment explicit and auditable.

The transcript is constructed as the NUL-byte-separated (0x00) concatenation of the following fields in the following order:

```
parts = [
    MAC_PREFIX,                       -- b"Arculus ARC v2 MAC"     (18 bytes)
    bundle["magic"].encode("utf-8"),  -- b"ARCULUS-ARC"            (11 bytes)
    bundle["format"].encode("utf-8"), -- b"arculus-encrypted-seed-v2"
    str(bundle["version"]).encode(),  -- b"2"
    bundle["created_at"].encode(),    -- ISO 8601 timestamp bytes
    bundle["kdf"]["name"].encode(),   -- b"PBKDF2"
    bundle["kdf"]["hash"].encode(),   -- b"SHA-512"
    str(bundle["kdf"]["iterations"]).encode(),  -- b"1000000"
    bundle["cipher"]["name"].encode(), -- b"HMAC-SHA512-CTR"
    salt,                             -- raw 32 bytes (not base64)
    nonce,                            -- raw 24 bytes (not base64)
    ciphertext                        -- raw ciphertext bytes (not base64)
]

transcript = b"\x00".join(parts)

mac = HMAC-SHA512(
    key = mac_key,
    msg = transcript
)
```

The 0x00 separator between adjacent fields prevents concatenation collision attacks in which bytes are moved between adjacent fields to produce an identical transcript byte sequence. An adversary cannot shift bytes between the magic and format fields, or between the nonce and ciphertext, without introducing a 0x00 boundary violation that changes the transcript value.

The fields committed by this MAC and the attacks they defeat:

```
MAC_PREFIX        -- domain-separation; prevents cross-context MAC reuse
magic             -- prevents format confusion with other arc-family formats
format            -- version-specific format string commitment
version           -- numeric version commitment
created_at        -- binds the MAC to the specific export event timestamp
kdf.name          -- prevents substitution of an alternative KDF algorithm
kdf.hash          -- prevents substitution of an alternative hash function
kdf.iterations    -- defeats KDF iteration-count downgrade attacks
cipher.name       -- prevents substitution of an alternative cipher
salt (raw bytes)  -- prevents base64 re-encoding of a modified salt value
nonce (raw bytes) -- prevents nonce substitution or reuse attacks
```

```
  ciphertext (raw)  -- the primary ciphertext authentication commitment
```

Full Export Procedure

Input: mnemonic (string), password (string)

```
  -- 1. Normalize and validate the mnemonic
  words = normalize_mnemonic_words(mnemonic)
  assert len(words) in {12, 24}

  -- 2. Generate fresh random parameters
  salt       = CSPRNG(32)                    -- os.urandom(32) / getRandomValues(32)
  nonce      = CSPRNG(24)                    -- os.urandom(24) / getRandomValues(24)
  created_at = current_utc_timestamp_iso8601()

  -- 3. Normalize password and derive keys
  password_bytes = NFKD(password).encode("utf-8")
  master_key     = PBKDF2-HMAC-SHA512(password_bytes, salt, 1_000_000, 64)
  enc_key        = HMAC-SHA512(master_key, ENC_LABEL)
  mac_key        = HMAC-SHA512(master_key, MAC_LABEL)

  -- 4. Construct and encrypt plaintext
  plaintext_obj  = {
      "mnemonic":   " ".join(words),
      "word_count": len(words),
      "created_at": created_at
  }
  plaintext      = compact_json(plaintext_obj).encode("utf-8")
  ciphertext     = plaintext XOR hmac_sha512_ctr_stream(enc_key, nonce, len(plaintext))

  -- 5. Assemble bundle and compute MAC
  bundle = {
      "magic":          "ARCULUS-ARC",
      "format":         "arculus-encrypted-seed-v2",
      "version":        2,
      "created_at":     created_at,
      "kdf":   { "name": "PBKDF2", "hash": "SHA-512",
                 "iterations": 1_000_000, "salt_b64": base64(salt) },
      "cipher": { "name": "HMAC-SHA512-CTR", "nonce_b64": base64(nonce) },
      "ciphertext_b64": base64(ciphertext),
      "mac_b64":        base64( HMAC-SHA512(mac_key, transcript(bundle, salt, nonce,
ciphertext)) )
  }

  -- 6. Serialize to armored text
  file_text = "ARCULUS-ARC-V2\n" + base64(compact_json(bundle)) + "\n"
```

Every invocation of the export procedure produces a distinct .arc file for the same (mnemonic, password) pair because salt and nonce are independently sampled from the CSPRNG on each call. Two .arc files exported from the same mnemonic with the same password are computationally indistinguishable from two .arc files with entirely different mnemonics. This is the correct and expected behavior.

Full Import Procedure

Input: file_text (string), password (string)

```
  -- 1. Detect and decode format
  if file_text.startswith("ARCULUS-ARC-V2"):
      body   = file_text.split("\n")[1]
      bundle = json_parse(base64_decode(body))
  else:
      bundle = json_parse(file_text)           -- legacy path: raw JSON body

  -- 2. Dispatch on version
  if bundle.get("magic") == "ARCULUS-ARC" or bundle.get("version") == 2:

      -- 3. Structural field validation
      assert bundle["format"]            == "arculus-encrypted-seed-v2"
      assert bundle["kdf"]["name"]       == "PBKDF2"
      assert bundle["kdf"]["hash"]       == "SHA-512"
      assert bundle["kdf"]["iterations"] >= ARC_V2_MIN_KDF_ITERATIONS
      assert bundle["cipher"]["name"]    == "HMAC-SHA512-CTR"

      -- 4. Decode binary fields
      salt       = base64_decode(bundle["kdf"]["salt_b64"])
      nonce      = base64_decode(bundle["cipher"]["nonce_b64"])
      ciphertext = base64_decode(bundle["ciphertext_b64"])
```

```
      mac_stored = base64_decode(bundle["mac_b64"])

      -- 5. Validate binary field lengths
      assert len(salt)       == 32
      assert len(nonce)      == 24
      assert len(mac_stored) == 64

      -- 6. Derive keys
      password_bytes = NFKD(password).encode("utf-8")
      iterations     = bundle["kdf"]["iterations"]
      master_key     = PBKDF2-HMAC-SHA512(password_bytes, salt, iterations, 64)
      enc_key        = HMAC-SHA512(master_key, ENC_LABEL)
      mac_key        = HMAC-SHA512(master_key, MAC_LABEL)

      -- 7. Verify MAC before any decryption
      mac_computed = HMAC-SHA512(mac_key, transcript(bundle, salt, nonce, ciphertext))
      if not constant_time_compare(mac_computed, mac_stored):
          raise DecryptionError("MAC verification failed")
                           -- error is intentionally non-specific

      -- 8. Decrypt (only reached after MAC verification succeeds)
      plaintext_bytes = ciphertext XOR hmac_sha512_ctr_stream(enc_key, nonce, len(
ciphertext))
      payload         = json_parse(plaintext_bytes.decode("utf-8"))

      -- 9. Extract and validate mnemonic
      mnemonic    = payload["mnemonic"]
      word_count  = payload.get("word_count")

      normalize and validate mnemonic words
      if word_count is present: assert len(words) == word_count

      return mnemonic

  else:
      -- Fall through to legacy import path
```

Step 7 is the authenticate-before-decrypt gate. This ordering is unconditional and cannot be bypassed. Decryption is not attempted, and no keystream bytes are consumed, before MAC verification returns a successful constant-time comparison. A MAC failure is reported as a generic authentication error. The error message does not distinguish between wrong password, corrupted file, truncated file, or tampered metadata. This information concealment is intentional.

Constant-Time MAC Comparison

MAC comparison must be performed in constant time to prevent timing side-channel attacks. A conditional short-circuit comparison - one that returns false on the first mismatched byte - leaks the length of the common prefix between the computed and stored MACs. Against a remote timing oracle this leakage can be exploited to recover the expected MAC byte by byte.

Python uses hmac.compare_digest(mac_computed, mac_stored) for constant-time comparison. compare_digest is implemented in C with explicit resistance to compiler optimization that could reintroduce early exit behavior.

The HTML version uses a manual XOR-accumulation loop over the full 64-byte MAC arrays:

```
  let diff = 0;
  for (let i = 0; i < 64; i++) diff |= mac_computed[i] ^ mac_stored[i];
  const match = (diff === 0);
```

The XOR-OR accumulation pattern ensures that every byte of both arrays is read regardless of any mismatch at any position. The SubtleCrypto API does not expose a constant-time comparison primitive for raw byte arrays, making this explicit loop the correct implementation choice.

Compatibility Matrix

```
Format description                            Export   Import    Notes
--------------------------------------------  -------  --------  --------------------------
------
ARCULUS-ARC-V2 armored v2                     yes      yes       Current format; all new
exports
JSON arculus-encrypted-seed-v2,               no       yes       Compatibility format;
  magic=ARCULUS-ARC, version=2                                   >=600,000 iterations
accepted
JSON v2, PBKDF2-SHA256 + XOR-HMAC             no       yes       Legacy Python-style format
arculus-encrypted-seed-python-v1              no       yes       Legacy Python pre-release
format
arculus-encrypted-seed-v1 (AES-GCM)          no       browser   HTML only; AES-GCM via
                                                       only      SubtleCrypto; Python
cannot read
```

All new exports use the ARCULUS-ARC-V2 armored format unconditionally, regardless of the format of the imported source file. If a legacy .arc file is imported and the user subsequently exports, the output is a fresh ARCULUS-ARC-V2 file with a newly sampled salt, a newly sampled nonce, and exactly 1,000,000 KDF iterations. This is the correct and recommended migration path for all pre-existing .arc files produced under any prior format version.

The minimum accepted KDF iteration count for v2 imports is exactly 600,000. Files with a kdf.iterations value below 600,000 are rejected even if MAC verification would otherwise succeed. This floor prevents silent acceptance of artificially weakened files without requiring the importer to know in advance what iteration count was used at export time.

KDF Iteration Count Rationale

PBKDF2-HMAC-SHA512 is an iterated pseudorandom function. Its resistance to offline password guessing is linear in the iteration count: an adversary who can compute T PBKDF2 evaluations per second can make T / iterations guesses per second against a captured .arc file.

The current export count of 1,000,000 iterations over SHA-512 is selected to impose meaningful computational cost on offline guessing while remaining tolerable for the import operation on low-power, software-only hardware. On a contemporary CPU executing PBKDF2-HMAC-SHA512 in software, 1,000,000 iterations completes on the order of several seconds. The deliberate import latency is the intended user experience: it imposes the same per-guess cost on an attacker.

Critical limitations:

```
- GPU-accelerated PBKDF2 achieves substantially higher throughput than
  software-only computation. The iteration count does not make a short,
  low-entropy, or dictionary-derivable password safe against a GPU-equipped
  adversary with a captured .arc file.

- Against a password of sufficient entropy - a randomly generated sequence
  of five or more words from a large vocabulary, or a random alphanumeric
  string of 16 or more characters - the iteration count provides comfortable
  resistance to all known offline guessing approaches.

- Against a PIN, a single dictionary word, a short common phrase, or any
  password derivable from personal information, no iteration count is a
  substitute for password entropy. The computational barrier is linear in
  iterations, but the search space is exponential in password length and
  alphabet size. A weak password with 1,000,000 iterations is less secure
  than a strong password with 10,000 iterations.
```

Memory-hard key derivation functions such as Argon2id or scrypt provide substantially stronger resistance to GPU and ASIC acceleration by requiring large amounts of RAM per evaluation, which limits parallelism. Neither can be used without external dependencies, which are excluded by design constraint 1. The iteration count is the only available tuning lever within the dependency boundary, and it has been set to the maximum value consistent with acceptable interactive latency on target hardware.

What the .arc Format Protects and Does Not Protect

Protected by the .arc format at rest:

```
- The mnemonic phrase stored inside the encrypted file, against an adversary
  who obtains the file without the password
- The integrity of all metadata fields - magic, format, version, created_at,
  KDF parameters, cipher parameters, nonce - through the MAC transcript
  commitment
```

- The KDF iteration count against downgrade attacks, because iterations is a MAC-bound field
- The creation timestamp as a non-repudiable bound element in the MAC

Not protected by the .arc format:

- The mnemonic while loaded into application memory after decryption
- The mnemonic while displayed on screen or in the word grid
- The decryption password while being typed or held in process memory
- Clipboard contents at any point in the workflow
- Derived private keys, extended private keys, and WIF keys, which are not encrypted by this format
- JSON/CSV/TXT derived-output export files, which are always plaintext
- Any on-chain state: Ethereum balances, ERC-20 token metadata, XRP ledger data, destination tags, or transaction history (the tool does not fetch, store, or process chain state in any form)
- The BIP39 wallet passphrase, which is a separate derivation parameter outside the scope of .arc protection
- The file against an adversary who knows or can correctly guess the password; a correct password always produces a successful MAC verification and decryption

The armored envelope prevents casual JSON inspection of sensitive operational parameters and discourages manual editing. It is not a security boundary. The security boundary is the MAC verification gate backed by the password-derived mac_key. The armoring is an operational defense against inadvertent information disclosure; the MAC is the cryptographic defense against unauthorized access.

Security Considerations for .arc File Storage

- Store .arc files exclusively on encrypted removable media. Storing a .arc file on the general filesystem of a daily-use machine reduces its security to that of the machine's full-disk encryption and OS access controls.

- Never synchronize .arc files to cloud storage services, email attachments, or any system with network connectivity. The file is designed for offline backup. Placing it on a networked system expands the attack surface from local to remote.

- The encryption is bounded by password entropy. A randomly generated passphrase of five or more words from a large wordlist provides a reasonable entropy baseline against offline attack. A passphrase derived from personal information, a PIN, or a common phrase provides substantially weaker protection regardless of the KDF iteration count.

- The .arc encryption password and the BIP39 wallet passphrase may differ. Neither can be derived from the other by any means. Both must be preserved independently.

- Re-export any .arc files created with fewer than 1,000,000 KDF iterations to apply the current hardening level. The compatibility floor of 600,000 accepts older files for import; re-exporting upgrades them.

- Verify that a newly exported .arc file imports correctly before relying on it as the sole backup of a mnemonic. Export verification - import under the export password and confirm the mnemonic matches - is a mandatory step in any operationally sound backup procedure.

Implementation Reference

Python function map:

```
arc_v2_keys(password, salt, iterations)
    Derives master_key via PBKDF2-HMAC-SHA512, then derives enc_key and
    mac_key via domain-labeled HMAC-SHA512. Returns (enc_key, mac_key).

arc_v2_stream_crypt(enc_key, nonce, data)
    Generates the HMAC-SHA512-CTR keystream and applies XOR to data. Used
    for both encryption (plaintext -> ciphertext) and decryption (ciphertext
    -> plaintext) since the operation is self-inverse.

arc_v2_mac_data(bundle_meta, salt, nonce, ciphertext)
    Constructs the NUL-byte-separated MAC transcript bytes from explicit
    field extractions. Returns the raw transcript byte sequence.

encrypt_v2(mnemonic, password)
    Full export pipeline: generates salt and nonce from os.urandom, derives
    all keys, encrypts plaintext, computes MAC, assembles bundle dict. Returns
```

```
    the internal bundle dict before serialization.

decrypt_v2(bundle, password)
    Full import pipeline: validates structural fields, derives keys, verifies
    MAC via hmac.compare_digest, decrypts on MAC success, validates mnemonic.
    Returns the plaintext mnemonic string.

serialize_seed_bundle(bundle)
    Produces the armored ARCULUS-ARC-V2 file text from an internal bundle
    dict: compact JSON serialization, then base64 encoding, then header line
    prepension.

parse_seed_bundle(file_text)
    Detects the file format (armored v2 vs. raw JSON legacy), decodes the
    armor if present, and returns the internal bundle dict for decryption.
```

HTML counterpart functions mirror the Python implementations in all cryptographically material respects:

```
serializeSeedBundle()     -- armor write path
parseSeedBundle()         -- format detection and armor decode
PBKDF2 via SubtleCrypto.deriveBits with { name: "PBKDF2", hash: "SHA-512" }
HMAC via SubtleCrypto.sign with { name: "HMAC", hash: "SHA-512" }
```

All ARC constants (labels, magic strings, format IDs, version integers, parameter sizes) are defined identically in both implementations. Any change to a constant value in either implementation must be propagated to the other with identical byte representation, or cross-implementation compatibility breaks silently - .arc files produced by the modified implementation will fail MAC verification when imported into the unmodified one, and vice versa.

Application Surface Notes

The standalone HTML application performs .arc export and import entirely with browser built-ins and embedded application code. The PySide6 GUI renders the same HTML application inside Qt WebEngine and therefore uses the same browser- side .arc implementation for GUI import/export. The Python CLI uses the Python implementation directly.

The Tauri wrapper also renders the canonical HTML application, but its desktop file-save path is routed through a native export bridge and Tauri v2 capability/permission files. This file-save bridge is outside the .arc cryptographic construction: a Tauri save failure does not imply a cryptographic format failure, but the final packaged Tauri export path must be verified before it is relied upon during an operational recovery session.

Arculus Recovery v1.5.0 ? Derivation Engine Internals

Scope

This document is the authoritative byte-level specification of the deterministic key derivation engine as implemented in Arculus Recovery v1.5.0. It defines the complete computational pipeline from raw CSPRNG entropy through BIP39 mnemonic generation and validation, Unicode normalization, PBKDF2 seed stretching, BIP32 hierarchical deterministic master node construction, recursive CKDpriv child key derivation, script-type resolution, per-coin address construction across five supported asset families, Taproot key-material processing under BIP340/BIP341/BIP86, extended key serialization, and structured multi-format output.

The current application has four user-facing surfaces over the same derivation engine: the standalone HTML application, the PySide6 desktop GUI, the Tauri desktop wrapper, and the Python CLI. The HTML application is the canonical GUI surface. The PySide6 and Tauri desktop variants render that same HTML application rather than maintaining a separate derivation UI. The CLI invokes the Python derivation engine directly. All GUI surfaces feed identical mnemonic, passphrase, coin, path, script type, range, and count values into the same deterministic pipeline.

Earlier releases introduced the cryptographically random mnemonic generation subsystem (generateRandomMnemonic / generate_random_mnemonic) and the individual word-entry grid that replaced the previous single-textarea mnemonic input. Both surfaces feed identical downstream validation and derivation pipelines with no behavioral divergence. This document covers typed word-grid input, paste- distributed input, CSPRNG-generated input, and CLI-supplied mnemonic input.

This is an offline reference. All constructions are defined entirely by their mathematical specification and byte-level behavior. No external service, network oracle, balance query, or chain-state lookup of any kind is required or performed at any stage.

Architectural Invariants

The engine is stateless and purely deterministic. Given identical inputs ? mnemonic, BIP39 passphrase, coin identifier, derivation path, script type, and address count ? it produces byte-for-byte identical output on every invocation, on every conformant platform, with zero dependency on external state.

The following operations are explicitly outside the engine's domain and are never performed:

- Address discovery or balance queries against any blockchain
- UTXO set scanning or wallet gap-limit heuristics
- Ethereum account state or ERC-20 token enumeration
- XRP ledger queries, destination-tag resolution, or sequence number lookup
- Fee estimation, mempool inspection, or chain-tip awareness of any kind
- Network I/O of any form

The derivation graph has the following invariant computational structure:

```
CSPRNG entropy (optional, for generation) OR word-grid / paste input
     |
     | BIP39 word index mapping + checksum verification
     v
normalized mnemonic string
     |
     | Unicode NFKD normalization of mnemonic and passphrase
     v
PBKDF2-HMAC-SHA512 (2048 iterations, 64-byte output)
     |
     | 512-bit seed
     v
HMAC-SHA512 keyed on b"Bitcoin seed"
     |
     | split at byte boundary 32
     v
master private scalar (IL[0:32]) || master chain code (IR[32:64])
     |
     | CKDpriv traversal over each hardened and non-hardened path component
     v
account private node at configured path depth
     |
```

```
      +---> branch index 0 (receiving): CKDpriv(account, 0) -> CKDpriv(node, i)
      +---> branch index 1 (change):    CKDpriv(account, 1) -> CKDpriv(node, i)
      |
      v
  per-index child private scalar -> address encoder -> structured output record
```

The HTML implementation executes this pipeline asynchronously via the Web Crypto API (SubtleCrypto.deriveBits, SubtleCrypto.sign) and pure-JavaScript big-integer arithmetic. The Python implementation executes the identical pipeline synchronously using hashlib, hmac, and native arbitrary-precision integers. Both implementations produce byte-identical derivation output for all inputs.

Elliptic Curve Domain Parameters

All public-key operations are performed over the secp256k1 prime-field elliptic curve as specified in SEC 2, version 2.0, section 2.4.1. The curve has a Koblitz-form short Weierstrass equation over a prime field:

```
  y^2 ? x^3 + 7  (mod p)
```

Curve parameters in full hexadecimal:

```
  Field prime:
    p  = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F

  Group order:
    n  = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

  Generator x-coordinate:
    Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798

  Generator y-coordinate:
    Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

  Cofactor: h = 1
```

The valid private key domain is the open interval (0, n). Any scalar equal to zero or greater than or equal to n is cryptographically undefined and is rejected by the engine at every point where a private scalar is produced or consumed.

secp256k1 has prime group order, making every non-identity point a valid generator of the full group. The cofactor of 1 means there are no small-subgroup attack vectors from points on the curve.

The engine implements the following primitives natively without delegating to external elliptic curve libraries:

```
  - Affine-coordinate point addition with lambda computation
  - Double-and-add scalar multiplication (left-to-right binary method)
  - Compressed public key serialization: 0x02 prefix if y is even, 0x03 if odd,
    followed by the 32-byte big-endian x-coordinate
  - X-only (BIP340) public key extraction: the lower 32 bytes of a compressed
    key after Y-parity normalization
  - Field arithmetic modulo p
  - Scalar arithmetic modulo n
```

All arithmetic is performed in the native integer domain. No floating-point operations are involved at any stage.

CSPRNG-Based Mnemonic Generation

The mnemonic generation function derives its entropy exclusively from the platform cryptographic random number generator. This function is invoked when the user activates the Generate Random Seed control.

The generation algorithm for a target word count wc ? {12, 24}:

```
  CS          = wc / 3                          -- checksum bits (integer division)
  ENT         = wc * 11 - CS                    -- entropy bits: 128 for 12w, 256 for 24w
  entropy     = CSPRNG(ENT / 8)                 -- ENT/8 raw random bytes

  hash        = SHA256(entropy)                 -- 32-byte digest
  checksum    = MSB(hash, CS)                   -- first CS bits of the digest

  bit_string  = entropy_bits || checksum_bits   -- ENT + CS = wc * 11 total bits
  words[i]    = BIP39_WORDS[ bit_string[i*11 : i*11+11] ]  for i in 0..wc-1

  mnemonic    = words.join(' ')

  -- Mandatory self-validation:
  assert checkMnemonic(mnemonic).wordsOk && checkMnemonic(mnemonic).checksumOk
```

The entropy source:

```
HTML:   crypto.getRandomValues(new Uint8Array(ENT / 8))
Python: os.urandom(ENT / 8)
```

Both delegates are OS-level CSPRNGs. The Web Crypto API requirement for crypto.getRandomValues ensures the browser cannot substitute a non-cryptographic PRNG. The Python os.urandom call maps to the same kernel-level entropy pool as /dev/urandom on POSIX systems and CryptGenRandom on Windows.

The self-validation step is non-optional. A generated mnemonic that fails its own BIP39 checksum verification is discarded entirely and never surfaced to the user. This guards against any integer truncation or bit-packing error in the generation implementation.

After successful generation, the mnemonic is routed through the hidden-seed workflow: it is stored in importedSeedState with kind='generated', the word grid is populated with obfuscated placeholders, and the mnemonic is not accessible without an explicit press-and-hold of Show Seed. The root fingerprint display updates immediately after generation using an asynchronous fingerprint refresh triggered by applyGeneratedSeed.

Individual Word-Entry Grid

The word-entry grid is built dynamically by buildWordInputs(). The grid always renders 24 individual input cells; the active word count is controlled by the 12/24-word radio selector (getSeedLen / setSeedLen).

Grid behavioral specification:

```
Active cells:  indices 0..(getSeedLen()-1)
Inactive cells: indices getSeedLen()..23; disabled and cleared on selector change

Paste behavior: pasting a space-separated string into any cell triggers
  distributeWords(), which tokenizes the input on whitespace, normalizes each
  token via normalizeNFKD, and distributes words sequentially into active
  cells starting at index 0. Distributing more than 12 words auto-promotes
  the selector to 24; distributing more than 0 words that sum to 12 sets the
  selector to 12. This allows a full mnemonic string to be pasted into any
  cell in the grid.

Input normalization: each cell applies normalizeNFKD() on input and blur
  events via handleWordInput, which also triggers styleWordInput() to apply
  inline per-word validation feedback.

Word extraction: getMnemonicFromGrid() iterates indices 0..(getSeedLen()-1),
  normalizes each cell value, and joins with single spaces to produce the
  canonical mnemonic string that is passed downstream.

Selector change behavior: switching from 24 to 12 clears and disables cells
  12..23. Switching from 12 to 24 re-enables cells 12..23. If an importedSeed
  State is active and not being revealed, selector changes have no effect on
  the word grid display.

Sync with legacy textarea: the hidden mnemonic textarea (id="mnemonic") is
  kept in sync with the word grid via handleMnemonicInput and handleWordInput
  event handlers through the syncBusy guard to prevent re-entrancy. The
  textarea remains the canonical internal mnemonic store for non-grid input
  paths (e.g., direct paste into the textarea).

Hidden-seed overlay: when importedSeedState is set and showingImportedSeed is
  false, refreshWordEnabled() disables all word cells. The cells display
  obfuscated placeholders set by setObfuscatedSeedInputs(). No word from the
  hidden mnemonic is readable from the DOM.
```

BIP39 Validation

The engine embeds the canonical BIP39 English word list in full (2048 entries, indices 0x000..0x7FF). Word lookup is O(1) via a pre-constructed reverse index map. The engine accepts exclusively 12-word and 24-word inputs; all other word counts are rejected with an explicit error rather than silently mishandled.

Validation procedure in formal notation:

```
words   = normalize_mnemonic_words(raw_input)
wc      = |words|
require wc in {12, 24}
require every w in words: w in BIP39_WORD_INDEX

B       = concat( index_11bit(words[i]) for i in 0..wc-1 )
ENT     = floor(wc * 11 * 32 / 33)
CS      = wc * 11 - ENT
```

```
B_ent   = B[0 : ENT]
B_cs    = B[ENT : ENT + CS]

entropy         = bits_to_bytes(B_ent)           -- ENT/8 bytes
checksum_source = SHA256(entropy)
B_cs_expected   = msb(checksum_source, CS)        -- first CS bits of hash

valid iff B_cs == B_cs_expected
```

Word count parameters:

```
wc    total_bits   ENT    CS
12    132           128     4
24    264           256     8
```

A mnemonic whose words are all valid BIP39 entries but whose checksum fails is classified as BIP39-like with invalid checksum. This distinction is operationally critical during recovery: a single transposed or misremembered word produces a structurally valid-looking derivation from a completely incorrect seed. The checksum failure is a mandatory recovery signal.

BIP39 Seed Derivation

Both the mnemonic and the optional BIP39 passphrase undergo Unicode NFKD normalization before byte-encoding. NFKD decomposition is specified in BIP39 and is mandatory ? it ensures that visually or semantically equivalent Unicode strings produce identical byte sequences regardless of composition form or platform input method. The implementation uses normalizeNFKD() in the HTML version and unicodedata.normalize('NFKD', ...) in Python.

```
mnemonic_str     = " ".join(normalized_words)
passphrase_str   = NFKD(user_passphrase)   -- empty string if not supplied

password_bytes   = mnemonic_str.encode("utf-8")
salt_bytes       = ("mnemonic" + passphrase_str).encode("utf-8")

seed = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = salt_bytes,
    iterations = 2048,
    dklen      = 64
)
```

The output is a 512-bit (64-byte) deterministic seed. The BIP39 passphrase functions as a second-factor wallet derivation parameter. An empty passphrase and any non-empty passphrase each derive a completely disjoint key tree from the same mnemonic, with no error, no warning, and no computational distinguisher between them. There is no mechanism to identify which passphrase produced a given tree without already knowing it.

The .arc encryption password is operationally and cryptographically independent of the BIP39 passphrase. They may coincidentally share the same string value by user choice, but the engine never conflates them. They operate on separate key schedules with unrelated KDF parameters.

BIP32 Master Node Construction

The 64-byte seed is processed through a single HMAC-SHA512 invocation:

```
I   = HMAC-SHA512(key = b"Bitcoin seed", msg = seed)
IL  = I[0:32]       -- 256-bit candidate private scalar
IR  = I[32:64]      -- 256-bit master chain code

master_private_key = parse256(IL)
master_chain_code  = IR
```

The private scalar is valid if and only if parse256(IL) ? (0, n). The probability of failure on a uniformly random seed is approximately 1/2^128, which is negligible for any practical purpose. The engine raises a hard error on this edge case rather than silently yielding an invalid or zero key.

The root fingerprint displayed in the UI toolbar and updated immediately after mnemonic generation or import is computed as:

```
root_pubkey       = serP(master_private_key ? G)   -- compressed, 33 bytes
root_fingerprint  = HASH160(root_pubkey)[0:4]
```

Where HASH160 is the standard Bitcoin double-hash function:

```
HASH160(x) = RIPEMD160(SHA256(x))
```

The fingerprint is serialized as 4 bytes of lowercase hexadecimal. It provides a reproducible, non-secret short identifier for the root key without exposing any recoverable key material. The fingerprint is recomputed asynchronously after every mnemonic state change (typed input, paste, import, or generation) using a cancellation token (rootFingerprintRefreshToken) to discard stale results from superseded inputs.

CKDpriv: Private Child Key Derivation

Child derivation follows BIP32 CKDpriv exactly. Let the parent node be defined by private scalar k_par, chain code c_par, and derived public key K_par = k_par ? G.

For a child at index i (0-indexed, 32-bit unsigned integer):

```
Hardened child (i ? 0x80000000):
  data = 0x00 || ser256(k_par) || ser32(i)

Normal child (i < 0x80000000):
  data = serP(K_par) || ser32(i)

I   = HMAC-SHA512(key = c_par, msg = data)
IL  = I[0:32]
IR  = I[32:64]

child_private_key   = (parse256(IL) + k_par) mod n
child_chain_code    = IR
```

Hard invalidity conditions, both of which require skipping to child index i+1 per the BIP32 specification (this engine raises an explicit error instead):

```
parse256(IL) ? n
child_private_key == 0
```

Hardened derivation folds the raw private scalar into the HMAC input, making child keys non-derivable from the parent public key alone. All account-level path components ? purpose', coin_type', account' ? use hardened indices. Branch indices (0, 1) and address indices (0..count-1) are non-hardened.

Each derived node carries the following fields:

```
private_scalar       32-byte big-endian integer
chain_code           32 bytes
depth                1 byte (path depth from master node)
parent_fingerprint   4 bytes (HASH160(parent_pubkey)[0:4])
child_number         4 bytes (ser32(i))
```

Derivation Path Parser

The path parser accepts all three conventional notations for hardened path components:

```
Apostrophe:   m/84'/0'/0'
Lowercase h:  m/84h/0h/0h
Uppercase H:  m/84H/0H/0H
```

Hardening applies the BIP32 hardening offset:

```
HARDENED = 0x80000000
```

Purpose field 84' is stored internally as index 0x80000054 (decimal 2147483732).

The canonical output form uses apostrophe notation exclusively:

```
m/84'/0'/0'
```

All path strings are emitted in this canonical form regardless of input notation. The parser rejects non-numeric path components, indices exceeding the 32-bit unsigned maximum, and malformed separator sequences as hard errors.

Account and Branch Model

The configured derivation path is treated as an account-level node. The engine derives the account node and then generates two standard BIP44-style branches:

```
Receiving branch:   <account_path>/0/<index>
Change branch:      <account_path>/1/<index>
```

Both the branch index (0 or 1) and the address index (0..count-1) are non-hardened derivations. The count parameter controls how many address indices are generated per branch.

Example for m/84'/0'/0' with count = 3:

```
Account:            m/84'/0'/0'
Receiving index 0:  m/84'/0'/0'/0/0
Receiving index 1:  m/84'/0'/0'/0/1
Receiving index 2:  m/84'/0'/0'/0/2
Change index 0:     m/84'/0'/0'/1/0
Change index 1:     m/84'/0'/0'/1/1
Change index 2:     m/84'/0'/0'/1/2
```

This branching structure is mandatory for BIP44/49/84/86 compliant derivation. The change branch is always generated in the output; the caller is responsible for selecting which branch is relevant to their use case.

SLIP44 Coin Type Registry and Default Paths

SLIP44 coin type assignments used by this engine:

```
bitcoin    coin_type = 0
litecoin   coin_type = 2
dogecoin   coin_type = 3
ethereum   coin_type = 60
xrp        coin_type = 144
```

Default account paths surfaced in the UI and GUI:

```
bitcoin     m/0'              (simplified recovery default)
litecoin    m/84'/2'/0'
dogecoin    m/44'/3'/0'
ethereum    m/44'/60'/0'
xrp         m/44'/144'/0'
```

Standard BIP purpose path matrix for UTXO coins:

```
BIP44 (P2PKH legacy)        m/44'/coin_type'/0'
BIP49 (P2WPKH-P2SH)         m/49'/coin_type'/0'
BIP84 (P2WPKH native SegWit) m/84'/coin_type'/0'
BIP86 (P2TR Taproot)        m/86'/coin_type'/0'
```

The --all-common CLI flag invokes all four purpose paths for UTXO coins in a single derivation pass. For Ethereum and XRP, --all-common degenerates to the single canonical BIP44 path, as those coins have no defined SegWit or Taproot analogues under SLIP44 and operate exclusively as account-address families.

Script Type Resolution

Script type is either explicitly specified or inferred from the path via Auto mode.

Explicit values accepted:

```
p2pkh
p2wpkh-p2sh
p2wpkh
p2tr
```

Auto mode reads the purpose field (first path component) and maps it:

```
purpose 44'  ->  p2pkh
purpose 49'  ->  p2wpkh-p2sh
purpose 84'  ->  p2wpkh
purpose 86'  ->  p2tr
```

If the resolved script type has no corresponding version byte entry in the active network configuration object, the engine halts with a hard error rather than generating a malformed address. Producing an address for an undefined script type is treated as an unrecoverable configuration fault.

Ethereum and XRP bypass the UTXO script-type resolution system entirely. Their address families are set to 'ethereum' and 'xrp' respectively, and all UTXO script-type parameters are ignored. Auto is the correct and safe selection for both account-family coins.

Network Version Byte Constants

All values are hexadecimal. Extended key version bytes are 4-byte prefixes for Base58Check serialization.

Bitcoin mainnet:

```
Script type    xprv version    xpub version    WIF prefix
p2pkh          0488ADE4        0488B21E         80
p2wpkh-p2sh    049D7878        049D7CB2         80
p2wpkh         04B2430C        04B24746         80
p2tr           0488ADE4        0488B21E         80
P2PKH version byte: 00
P2SH version byte:  05
Bech32 HRP:         bc
```

Bitcoin testnet (--testnet flag, Python CLI):

```
Script type    xprv version    xpub version    WIF prefix
p2pkh          04358394        043587CF         EF
p2wpkh-p2sh    044A4E28        044A5262         EF
p2wpkh         045F18BC        045F1CF6         EF
p2tr           04358394        043587CF         EF
P2PKH version byte: 6F
P2SH version byte:  C4
Bech32 HRP:         tb
```

Litecoin mainnet:

```
Script type    xprv version    xpub version    WIF prefix
p2pkh          019D9CFE        019DA462         B0
p2wpkh-p2sh    01B26792        01B26EF6         B0
p2wpkh         04B2430C        04B24746         B0
p2tr           019D9CFE        019DA462         B0
P2PKH version byte: 30
P2SH version byte:  32
Bech32 HRP:         ltc
```

Dogecoin mainnet:

```
Script type    xprv version    xpub version    WIF prefix
p2pkh          02FAC398        02FACAFD         9E
p2wpkh-p2sh    02FAC398        02FACAFD         9E  (recovery inspection only)
p2wpkh         02FAC398        02FACAFD         9E  (recovery inspection only)
p2tr           UNSUPPORTED (explicitly disabled in the engine)
P2PKH version byte: 1E
P2SH version byte:  16
Bech32 HRP:         doge
```

Dogecoin SegWit-style script outputs are generated for recovery inspection purposes only. Production wallet support for Dogecoin SegWit is not standardized; m/44'/3'/0' P2PKH remains the only safe Dogecoin default.

Ethereum mainnet:

```
SLIP44 coin_type:    60
Canonical path:      m/44'/60'/0'
Address family:      ethereum
Address format:      EIP-55 checksummed 0x<40 hex chars>
xprv/xpub versions:  0488ADE4 / 0488B21E
WIF:                 undefined; private key exported as raw 32-byte hex
```

ERC-20 tokens operate on the same Ethereum account address. The engine does not query token contracts, read allowances, enumerate on-chain token balances, or interact with any chain state. The ERC-20 output field is the identical Ethereum address, annotated to clarify token compatibility.

XRP Ledger mainnet:

```
SLIP44 coin_type:    144
Canonical path:      m/44'/144'/0'
Address family:      xrp
Address format:      XRPL classic r<base58check>
xprv/xpub versions:  0488ADE4 / 0488B21E
WIF:                 undefined; private key exported as raw 32-byte hex
XRPL base58 alphabet:
  rpshnaf39wBUDNEGHJKLM4PQRST7VWXYZ2bcdeCg65jkm8oFqi1tuvAxyz
```

XRP destination tags are exchange-assigned routing metadata that are not derived from the seed and are not produced by this engine under any circumstance. If a custodian requires a destination tag, that value must be sourced from the custodian

directly.

Address Construction ? Per Script Type

All UTXO address construction uses the 33-byte compressed public key of the derived child node:

```
child_pub = serP(child_private ? G)   -- parity byte (0x02/0x03) || x-coordinate
```

Let pkh = HASH160(child_pub) = RIPEMD160(SHA256(child_pub)) -- 20 bytes

P2PKH (Pay-to-Public-Key-Hash):

```
payload = p2pkh_version_byte || pkh
address = Base58Check(payload)
```

P2WPKH-P2SH (Wrapped SegWit, P2SH-encapsulated witness program):

```
redeem_script = 0x00 || 0x14 || pkh    -- OP_0 PUSH20 <pkh>; 22 bytes total
script_hash   = HASH160(redeem_script)  -- 20 bytes
payload       = p2sh_version_byte || script_hash
address       = Base58Check(payload)

The opcode 0x14 is the minimal-push encoding for a 20-byte push (decimal 20 =
hex 0x14). This wraps the native SegWit witness program in a P2SH envelope for
backward compatibility with pre-SegWit infrastructure.
```

P2WPKH (Native SegWit, Bech32, BIP173):

```
witness_program  = convertbits(pkh, 8, 5)    -- 5-bit regrouping
address          = Bech32Encode(hrp, [0] || witness_program)
```

P2TR (Taproot, Bech32m, BIP341/BIP350): see Taproot Key Material section below.

Ethereum (EIP-55 checksummed account address):

```
uncompressed = x(P) || y(P)          -- 64 bytes, no prefix byte
account_id   = Keccak256(uncompressed)[12:32]   -- rightmost 20 bytes
address_hex  = lowercase_hex(account_id)         -- 40 hexadecimal characters

checksum     = Keccak256(ascii(address_hex))

For each hex character at position i in address_hex:
  if bit (4*i) of checksum == 1: uppercase the character
  else: lowercase the character

address = "0x" || EIP55_cased(address_hex)

NOTE: Ethereum uses Keccak-256, the pre-standardization Keccak variant, not
the NIST FIPS 202 SHA3-256 standard. These two functions produce different
output for the same input. The implementation uses the pre-standard Keccak
function throughout; substituting SHA3-256 produces incorrect addresses.
```

XRP (XRPL classic address via secp256k1):

```
account_id   = RIPEMD160(SHA256(child_pub))    -- child_pub is 33-byte compressed
payload      = 0x00 || account_id               -- account type prefix byte
checksum     = SHA256(SHA256(payload))[0:4]
address      = Base58Encode(payload || checksum, XRPL_ALPHABET)

The XRPL base58 alphabet is distinct from Bitcoin's. The encoding algorithm is
structurally identical to Bitcoin Base58Check but uses the XRPL character set.
Version byte 0x00 maps to the leading 'r' character in the XRPL alphabet,
producing the canonical r... classic address format.
```

Base58Check Encoding

Base58Check is used for P2PKH addresses, P2WPKH-P2SH addresses, WIF private keys, and extended keys across all applicable coins. The encoding procedure:

```
checksum   = SHA256(SHA256(payload))[0:4]
full_data  = payload || checksum
address    = Base58Encode(full_data)
```

Bitcoin Base58 alphabet (58 characters, no 0, O, I, l):

```
123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
```

Leading zero bytes in the payload are encoded as leading '1' characters; this property is essential for correct P2PKH address construction on certain address prefixes.

WIF (Wallet Import Format) for compressed-public-key wallets:

```
wif_payload = wif_prefix || ser256(private_scalar) || 0x01
wif         = Base58Check(wif_payload)
```

The 0x01 compression flag byte is mandatory. A WIF key without this suffix encodes an uncompressed public key association, which produces a different address and is not compatible with any address type generated by this engine.

Bech32 and Bech32m Encoding

Bech32 (BIP173) is used for P2WPKH native SegWit addresses (witness version 0). Bech32m (BIP350) is used for P2TR Taproot addresses (witness version 1).

The two encodings are identical except for the checksum polynomial constant:

```
Bech32:   generator constant = 1
Bech32m:  generator constant = 0x2BC830A3
```

Both use the same 32-character 5-bit data alphabet:

```
qpzry9x8gf2tvdw0s3jn54khce6mua7l
```

Witness program encoding procedure:

```
P2WPKH (witness version 0):
  data    = convertbits(pkh, 8, 5)           -- regroup 20-byte pkh into 5-bit groups
  payload = [0] || data                       -- prepend witness version byte
  address = bech32_encode(hrp, payload)

P2TR (witness version 1):
  data    = convertbits(output_x_only, 8, 5) -- regroup 32-byte x-only key
  payload = [1] || data
  address = bech32m_encode(hrp, payload)
```

The convertbits function (defined in BIP173) performs strict bit-group regrouping with end-padding. It does not reverse byte order.

Taproot Key Material (BIP340 / BIP341 / BIP86)

Taproot address derivation begins with the child private scalar at the derived path. The construction follows BIP340 x-only public key semantics, BIP341 key-path spending with a TapTweak commitment, and BIP86 empty-script-tree enforcement. All key-path-only Taproot outputs use an empty Merkle root, meaning the tweak commits solely to the internal key with no script alternatives.

Let d = child private scalar, P = d ? G (affine point on secp256k1).

Step 1 ? Y-parity normalization to produce the BIP340 internal key pair:

```
BIP340 requires the internal public key to have an even Y coordinate.

if y(P) is odd:
    internal_private = n - d        -- scalar negation: (n - d) mod n
    internal_public  = -P           -- point negation: (x, p - y)
else:
    internal_private = d
    internal_public  = P

internal_x = x(internal_public)    -- 32-byte x-only representation (big-endian)
```

Step 2 ? TapTweak tagged hash computation:

```
tagged_hash(tag, msg) = SHA256(SHA256(tag_bytes) || SHA256(tag_bytes) || msg)

For BIP86 key-path-only (empty script tree, no Merkle root):

  t = int(tagged_hash("TapTweak", internal_x)) mod n

Reject if t ? n (probability approximately 2^-128; treated as derivation error).

The double-SHA256 tag prefix is a BIP340 convention that domain-separates the
hash from all other SHA256 applications in the protocol, preventing cross-
context hash collisions.
```

Step 3 ? Output key derivation:

```
output_public  = internal_public + t ? G
output_private = (internal_private + t) mod n

Reject if output_public is the point at infinity (negligible probability).
Reject if output_private == 0 (negligible probability).
```

Step 4 ? X-only output key extraction and parity recording:

```
output_x      = x(output_public)            -- 32 bytes; used as witness program
output_parity = y(output_public) mod 2      -- 0 = even Y, 1 = odd Y
```

Step 5 ? Bech32m address construction:

```
address = Bech32m(hrp, [1] || convertbits(output_x, 8, 5))
```

Exported Taproot fields per derived address:

```
internal_public_key_hex    x-only internal key, 32 bytes hex
internal_private_key_hex   internal private scalar, 32 bytes hex
tweak_hex                  t as 32 bytes hex
output_public_key_hex      x-only output key, 32 bytes hex
output_private_key_hex     output private scalar, 32 bytes hex
output_private_key_wif     WIF encoding of output_private using coin WIF prefix
output_key_parity          integer 0 or 1
address                    Bech32m-encoded Taproot address
```

The output_private_key is the operative signing scalar for the Taproot output. The internal_private_key is an intermediate derivation value. Both are funds-controlling secrets and must be treated with the same operational security as the mnemonic itself.

Extended Key Serialization

Extended keys are serialized as 78-byte structures and then Base58Check-encoded.

Extended private key (xprv / yprv / zprv / tprv variants):

```
Offset   Length   Field
------   ------   -----
0        4        version bytes (script-type and network dependent)
4        1        depth (0 at master; increments by 1 per derivation step)
5        4        parent fingerprint (HASH160(parent_pubkey)[0:4]; 0x00000000 at master)
9        4        child number (ser32(i); 0x00000000 at master)
13       32       chain code
45       1        0x00 (private key marker prefix)
```

46 32 private scalar (ser256(k))

```
Total:   78 bytes before Base58Check double-SHA256 checksum
```

Extended public key (xpub / ypub / zpub / tpub variants):

```
Offset   Length   Field
------   ------   -----
0        4        version bytes
4        1        depth
5        4        parent fingerprint
9        4        child number
13       32       chain code
```

45 33 compressed public key (serP(K))

```
Total:   78 bytes before Base58Check double-SHA256 checksum
```

Ethereum and XRP use the standard xprv/xpub version bytes (0488ADE4 / 0488B21E) as no coin-specific extended key version byte allocations for those coins exist in SLIP132. WIF is undefined for account-family coins; their private key exports carry private_key_hex as a raw 32-byte hexadecimal string.

Output Record Schema

Top-level derivation result object:

```
{
  "coin":       <string>,
```

```
  "network":    <string>,
  "word_count": <integer>,
  "accounts":   [ <account>, ... ]
}
```

Account object:

```
{
  "derivation":                   <path string>,
  "account_script_type_used":     <string>,
  "account_extended_private_key": <xprv/yprv/zprv string>,
  "account_extended_public_key":  <xpub/ypub/zpub string>,
  "root_extended_private_key":    <xprv string>,
  "root_extended_public_key":     <xpub string>,
  "receiving":                    [ <address_record>, ... ],
  "change":                       [ <address_record>, ... ]
}
```

Address record (UTXO coins, non-Taproot):

```
{
  "path":             <full BIP32 path string>,
  "address":          <encoded address>,
  "public_key_hex":   <33-byte compressed hex>,
  "private_key_hex":  <32-byte hex>,
  "private_key_wif":  <WIF string>
}
```

Address record (P2TR Taproot, augmented):

```
As above, plus:
  "internal_public_key_hex":   <32-byte x-only hex>,
  "internal_private_key_hex":  <32-byte hex>,
  "tweak_hex":                 <32-byte hex>,
  "output_public_key_hex":     <32-byte x-only hex>,
  "output_private_key_hex":    <32-byte hex>,
  "output_private_key_wif":    <WIF string>,
  "output_key_parity":         <integer 0 or 1>
```

Address record (Ethereum):

```
{
  "path":                    <path string>,
  "address":                 <EIP-55 checksummed 0x... string>,
  "ethereum_public_key_hex": <33-byte compressed hex>,
  "private_key_hex":         <32-byte hex>,
  "erc20_address":           <same as address>,
  "erc20_note":              <token compatibility advisory string>
}
```

Address record (XRP):

```
{
  "path":               <path string>,
  "xrp_classic_address": <r... classic address>,
  "public_key_hex":     <33-byte compressed hex>,
  "private_key_hex":    <32-byte hex>,
  "xrp_note":           <destination tag advisory string>
}
```

Export Format Behavior

```
JSON:
  Preserves the full nested object hierarchy: top-level result -> accounts array
  -> per-account receiving and change arrays -> per-address records. All schema
  fields are present and ordered according to the internal schema definition.
  Suitable for programmatic inspection and cross-tool compatibility.

CSV:
  Flattened row-per-address representation. Account-level fields (derivation
  path, script type, account xpub/xprv) are repeated on every row belonging to
  that account. The first row is a typed header. Receiving and change branch
  entries are distinguished by a branch column value.

TXT:
  Human-readable labeled sections with consistent field-label alignment.
  Intended for offline visual inspection on paper or screen. Accounts are
  top-level sections; receiving and change branches are subsections separated
  by blank lines.
```

None of the three export formats apply encryption. Any file containing private_key_hex, private_key_wif, output_private_key_hex, or any extended private key field is operationally equivalent in sensitivity to the source mnemonic. Such exports must be stored on encrypted media and handled with the same operational security as the seed phrase itself.

Failure Modes and Hard Errors

The engine raises explicit, descriptive errors and produces no partial output on:

```
Input validation:
  - Mnemonic word count not in {12, 24}
  - Any word not present in the BIP39 English word list
  - BIP39 checksum mismatch

Configuration:
  - Unrecognized coin identifier
  - Testnet flag set for a coin without defined testnet network constants
  - Script type unsupported for the selected coin and network combination
  - Invalid path component: non-numeric, out of 32-bit unsigned range, malformed separator
  - Missing version byte constant for the requested script type on the target network

Derivation edge cases (negligible probability, explicit handling required):
  - Master key invalid: parse256(IL) ? n or IL == 0
  - Child key invalid: parse256(IL) ? n or child_key == 0
  - Taproot TapTweak scalar t ? n
  - Taproot output public key is the point at infinity
  - Taproot output private scalar == 0

Generation:
  - Generated mnemonic fails its own BIP39 checksum self-validation
```

The engine is intentionally intolerant of ambiguity. Deriving an address for the wrong path, wrong script type, or wrong coin is a worse outcome than refusing to derive. Every error condition surfaces a descriptive diagnostic and yields no output.

Practical Recovery Reference

Systematic recovery procedure when attempting to locate a wallet's addresses:

```
1. Validate the mnemonic and confirm the BIP39 checksum passes.
2. Determine definitively whether a BIP39 passphrase was used. An incorrect
   or absent passphrase silently produces a completely different key tree with
   no error or warning; all derived addresses will be cryptographically
   unrelated to the on-chain history.
3. Identify the coin and network.
4. Try each purpose-level path in descending order of statistical likelihood:
     m/44'/coin_type'  -- most common for legacy wallets and all account coins
     m/84'/coin_type'  -- most common for native SegWit Bitcoin / Litecoin
     m/49'/coin_type'  -- wrapped SegWit
     m/86'/coin_type'  -- Taproot
5. Inspect the receiving branch (branch index 0) before the change branch (1).
6. Use --all-common to derive all four purpose paths for UTXO coins in a single
   CLI invocation when the originating wallet software is unknown.
7. Increase --count if the target address lies beyond the default address window.
8. Treat every output record containing private key material as operationally
   sensitive regardless of current on-chain balance.
```

CLI Reference

Invocation:

```
python Arculus_Recovery.py \
  --mnemonic "word1 word2 ... word12" \
  --passphrase "optional BIP39 passphrase" \
  --derivation "m/84'/0'/0'" \
  --script-type p2wpkh \
  --count 5 \
  --coin bitcoin \
  --output-format txt
```

Flag reference:

```
--mnemonic        BIP39 mnemonic phrase (12 or 24 words, space-separated, quoted)
--passphrase      Optional BIP39 wallet passphrase
--derivation      Account-level derivation path in BIP32 path notation
--all-common      Derive m/44', m/49', m/84', m/86' for UTXO coins in one pass;
                  m/44' only for Ethereum and XRP
```

```
--script-type     auto | p2pkh | p2wpkh-p2sh | p2wpkh | p2tr
--count           Number of address indices per branch (default: 5)
--coin            bitcoin | litecoin | dogecoin | ethereum | xrp (default: bitcoin)
--testnet         Enable testnet network parameters (Bitcoin only)
--output-format   json | csv | txt (default: json)
--gui             Launch the PySide6 desktop GUI instead of CLI derivation
--version         Print version string (1.5.0) and exit
```

JSON is the default CLI output format. When --output-format is not specified, the output is written to stdout as a UTF-8 JSON document.

Implementation Reference

Core function map in derivation pipeline order:

```
Python                              HTML equivalent
----------------------------------  ----------------------------------
generate_random_mnemonic(wc)        generateRandomMnemonic(wordCount)
bip39_validate(words)               checkMnemonic(mnemonic, passphrase)
bip39_to_seed(mnemonic, passphrase) mnemonicToSeed(mnemonic, passphrase)
master_from_seed(seed)              (inline in derivation pipeline)
parse_path(path_str)                parsePath(path)
normalize_path(path_str)            (canonical form enforced at parse time)
ckd_priv(parent_node, index)        ckdPriv(node, index)
derive(root_node, path)             deriveNode(root, pathArr)
derive_account(seed, path, ...)     deriveAccount(seed, ...)
taproot_key_material(child_priv)    taprootKeyMaterial(childPriv)
pubkey_to_address(pubkey, ...)      (script-type-dispatched inline)
ext_prv_to_base58(node, ...)        extPrivKeyB58(node, ...)
ext_pub_to_base58(node, ...)        extPubKeyB58(node, ...)
run_derivation(mnemonic, ...)       runDerivation(mnemonic, ...)
format_derived_output(result, fmt)  formatDerivedOutput(result, fmt)
```

The HTML and Python implementations are specification-equivalent. For identical (mnemonic, passphrase, coin, path, script_type, count) inputs, both produce byte-identical derivation output, enabling cross-verification between the two implementations. The PySide6 and Tauri GUI surfaces inherit the HTML derivation behavior because they package and render the canonical HTML application. A .arc seed file exported from the HTML or PySide6 version imports correctly into the Python version and vice versa.

Arculus Recovery v1.5.0 ? QR Export Subsystem Reference

Scope

This document is the authoritative specification of the QR Export subsystem as implemented in Arculus Recovery v1.5.0. It defines the self-contained QR code encoder, the URI scheme auto-detection rules for all five supported coins, the XRP destination tag workflow, the rendering parameters, the two entry points into the QR modal, the output actions available after generation, and the complete capacity and version selection table.

This document covers only the QR Export subsystem. Address derivation, coin selection, and key material handling are defined in Derivation.txt. Operational security requirements that apply to any session in which key material or addresses are handled apply equally here and are defined in OpSec.txt. This document does not repeat those requirements except where they have direct consequences specific to QR code use.

The QR Export subsystem is implemented in the HTML application surface. It is not available in the Python CLI. It is available in GUI surfaces that render the canonical HTML application, including the standalone browser version and the PySide6 desktop GUI. The Tauri wrapper packages the same QR code generation logic, but its desktop save-to-file path is subject to the Tauri export bridge described later in this document and in FileFormat.txt.

Design Constraints

The QR encoder was designed under the same zero-dependency constraint that governs the rest of the tool. No external libraries, no CDN requests, and no network I/O of any kind are involved in generating or rendering a QR code. The encoder is a fully self-contained implementation embedded in the HTML file alongside the derivation and encryption subsystems.

This constraint is absolute. Any scanner or wallet app that reads a QR code generated by this tool receives data that was encoded entirely by local computation on the airgapped machine. No third-party server has seen the address or URI at any point in the process.

The encoder supports byte mode encoding for all input strings, with automatic version and error correction level selection from QR versions 1 through 10. It does not implement alphanumeric or numeric mode separately; all input is encoded as byte-mode data using the UTF-8 encoding of the input string. For ASCII-only address strings (all supported coin address formats), byte mode produces output identical in decoded content to alphanumeric mode, though byte mode may select a higher QR version for the same input length due to the longer mode indicator length.

QR Code Encoder ? Technical Specification

The encoder is implemented as the QRGen module, an immediately-invoked function expression returning two public methods: generate(text) and render(matrix, canvas, opts). Both methods are used internally by the QR UI; neither is exposed on the global scope outside the module.

Reed-Solomon arithmetic foundation:

```
The encoder uses GF(256) arithmetic over the primitive polynomial
x^8 + x^4 + x^3 + x^2 + 1 (0x11D) for Reed-Solomon error correction code
generation. GF(256) tables (EXP and LOG, each 256 entries) are computed once
at module initialization using a generator element of 2. EXP is extended to
512 entries by wrapping (EXP[i] = EXP[i - 255] for i in 255..511) to simplify
modular index arithmetic in the multiply operation. The Reed-Solomon encode
function (rsEncode) computes the remainder of polynomial long division over
GF(256) for a given data codeword sequence and error correction count.
```

Version and error correction level selection:

```
The encoder supports QR versions 1 through 10 at two error correction levels:
M (approximately 15% recovery capacity) and Q (approximately 25% recovery
capacity). Level M is attempted first; level Q is used only when the input
exceeds the level M capacity of version 10.

Version and level are selected by the following rule: iterate versions 1
through 10 in ascending order. For each version, if the required byte count
fits within the level M data capacity, select that version at level M and
stop. If it does not fit level M but fits level Q, select level Q and continue
to the next version to check if a lower version at M is possible. If version
10 is reached and only level Q fits, select version 10 at level Q.

Required byte count is computed as: ceil((4 + len_bits + input_bytes * 8) / 8),
where len_bits is 8 for versions 1-9 and 16 for version 10.
```

Capacity table (data bytes available per version and level):

Version	Level M	Level Q
1	16	13
2	28	22
3	44	32
4	64	48
5	86	64
6	108	84
7	124	93
8	154	122
9	182	154
10	216	180

Reed-Solomon error correction parameters per version and level:

Version	Level M (ec/block, blocks)	Level Q (ec/block, blocks)
1	10, 1	13, 1
2	16, 1	22, 1
3	26, 2	18, 2
4	18, 2	26, 2
5	24, 2	18, 4
6	16, 4	24, 4
7	18, 4	18, 6
8	22, 4	22, 6
9	22, 5	20, 7
10	26, 5	24, 8

The encoder handles block interleaving: when multiple blocks are present, data codewords are interleaved by taking one codeword from each block in turn across all blocks, and error correction codewords are interleaved separately by the same method.

Matrix construction:

The QR matrix is constructed by makeMatrix(ver), which places structural module patterns using integer values 1 (dark) and 0 (light), reserving boolean values (true/false) exclusively for data and error correction modules. This type distinction is load-bearing: the mask application function (applyMask) identifies data modules by checking typeof cell === 'boolean' and applies the mask function only to those cells. Structural modules are never masked. Data modules are placed by placeData() in the canonical data module placement order specified by the QR standard.

Structural patterns placed by makeMatrix:

- Three finder patterns (top-left, top-right, bottom-left), each 7x7 with a one-cell separator ring of light modules
- Timing strips: alternating dark/light modules on row 6 and column 6 between the finder patterns
- Alignment patterns at positions defined per version (none for version 1; a single alignment pattern for versions 2-6; multiple for versions 7-10)
- Dark module at fixed position (4*ver + 9, 8)
- Format information module positions reserved as 0 before format bits are written

Mask selection:

All eight QR mask patterns (0 through 7) are applied to candidate matrices in turn. For each candidate, the format information strips are written and a penalty score is computed by penaltyScore(). The mask with the lowest penalty score is selected as the final mask. The penalty scorer applies all four QR standard penalty rules: runs of five or more identical modules in a row or column, 2x2 blocks of identical modules, patterns resembling finder patterns, and proportion of dark modules outside the 45?55% band.

Format information:

writeFormat() places all 15 format information bits at their exact positions on both the top-left strip (column 8, rows 0-5, row 7, row 8; and row 8, columns 0-5, column 7, column 8) and the top-right and bottom-left copy strips. Format strings for all eight masks at both supported levels are stored as precomputed 15-bit integers in FMT_M (level M) and FMT_Q (level Q).

Rendering:

The render(matrix, canvas, opts) method draws the QR matrix to an HTML canvas element. The canvas dimensions are computed as: module_size = floor(dim / (matrix_size + quiet * 2)), then actual_dim = module_size * (matrix_size +

```
quiet * 2). The canvas is sized to actual_dim x actual_dim to avoid fractional
module scaling.

Rendering parameters for the QR Export modal:
  Canvas size:     256 pixels
  Quiet zone:      4 modules
  Foreground:      #000000 (black)
  Background:      #ffffff (white)

The foreground and background colors are fixed constants regardless of the
active theme. Dark, Dark+, and Light mode all render identical black-on-white
QR codes. This is a deliberate design decision: camera-based QR decoders in
wallet applications are calibrated for high-contrast black-on-white patterns.
Rendering with colored or dark backgrounds causes silent scan failures on many
mobile wallet apps. The enforcement of black-on-white rendering is
unconditional and overrides any theme preference.

The quiet zone of 4 modules matches the minimum specified by the QR standard
(ISO/IEC 18004). A quiet zone smaller than 4 modules causes decode failures
on scanners that crop near the canvas edge.
```

URI Scheme Auto-Detection (toBip21Uri)

Before encoding, the address string is passed through toBip21Uri(), which detects the coin type from the address format and constructs the appropriate URI. This produces the URI format that wallet apps expect to parse when scanning a payment QR code, rather than a bare address string which some wallets reject.

Detection is applied in the following order. The first rule that matches wins. All pattern tests are applied to the trimmed address string.

```
Rule 1 ? Pass-through for existing scheme:
  If the address already contains a URI scheme prefix (matches /^[a-z]+:/i),
  the address is returned unchanged. A manually typed URI such as
  "bitcoin:1abc..." is passed through without modification or re-prefixing.

Rule 2 ? Bitcoin:
  Pattern: /^(bc1|[13])[a-zA-HJ-NP-Z0-9]{25,}/
  Matches: bc1q... (P2WPKH native SegWit), bc1p... (P2TR Taproot), 1...
  (P2PKH legacy), 3... (P2SH wrapped SegWit).
  URI produced: "bitcoin:" + address

Rule 3 ? Litecoin:
  Pattern: /^(ltc1|[LM])[a-zA-HJ-NP-Z0-9]{25,}/
  Matches: ltc1q... (P2WPKH native SegWit), ltc1p... (Taproot-style),
  L... (P2PKH legacy), M... (P2SH wrapped SegWit).
  URI produced: "litecoin:" + address
  Note: The Litecoin pattern does not match addresses starting with "3".
  Bitcoin P2SH addresses begin with 3; Litecoin P2SH addresses begin with M.
  The absence of "3" in the Litecoin rule prevents ambiguity with the Bitcoin
  rule, which is evaluated first and matches 3... addresses as Bitcoin P2SH.

Rule 4 ? Dogecoin:
  Pattern: /^[DA9][a-zA-HJ-NP-Z0-9]{25,}/
  Matches: D... (P2PKH, the standard Dogecoin address format), A... and 9...
  (P2SH, used for wrapped SegWit; wallet support for these Dogecoin formats
  varies; see the derivation path table in Derivation.txt).
  URI produced: "dogecoin:" + address

Rule 5 ? Ethereum:
  Pattern: /^0x[0-9a-fA-F]{40}$/
  Matches: EIP-55 checksummed or unchecksummed Ethereum addresses of exactly
  42 characters (0x prefix plus 40 hex digits). ERC-20 token addresses use
  the same address format and match this rule.
  URI produced: "ethereum:" + address
  Note: The ethereum: scheme is defined in EIP-681 and is supported by
  MetaMask, Trust Wallet, and most major Ethereum mobile wallets for basic
  address scanning. Amount and token parameters defined in EIP-681 are not
  added by this tool; only the base address URI is produced.

Rule 6 ? XRP:
  Detection: isXrpAddress(a), which tests /^r[1-9A-HJ-NP-Z0-9]{24,}/
  Matches: XRPL classic r-addresses of 25 or more base58-alphabet characters
  beginning with r.
  URI produced (without destination tag): the bare address, no scheme prefix.
  URI produced (with destination tag):    address + "dt=" + tag
  The XRP rule does not prepend a scheme prefix. The xrp: URI scheme is not
  an IETF-registered standard, and xrpl: (XLS-7d) has inconsistent support
  across wallet software. The bare address format with the optional dt= query
```

```
    parameter for destination tags is the most broadly compatible representation
    across Coinbase, Robinhood, Binance, and XUMM/Xaman. See the XRP Destination
    Tag Workflow section below for the full two-step generation procedure.

  Rule 7 ? Fallback:
    If no rule above matches, the address is encoded as entered. No scheme is
    prepended. This covers custom or non-standard address formats not recognized
    by the detection rules.
```

The URI string produced by toBip21Uri() is both encoded into the QR matrix and displayed verbatim in the address label below the QR canvas, so the user can verify exactly what was encoded before scanning or saving.

XRP Destination Tag Workflow

XRP Ledger exchange accounts on platforms including Coinbase, Robinhood, Binance, and XUMM/Xaman require a destination tag to route an incoming deposit to the correct user account. The tag is a 32-bit unsigned integer associated with the exchange's shared deposit address. Sending XRP to an exchange deposit address without the required destination tag results in the funds arriving at the exchange but not being credited to the user's account; recovery requires contacting the exchange's support and is not guaranteed.

Because of this risk, the QR Export modal implements a two-step workflow for XRP addresses rather than rendering immediately on Generate.

```
Step 1 ? Address entry and detection:
  The user pastes or types an XRP address into the address input field and
  clicks Generate (or presses Enter). isXrpAddress() tests the input. If the
  input matches an XRP r-address, the QR canvas is not rendered yet. Instead,
  the destination tag prompt panel is shown and focus moves to the tag input.

Step 2a ? Generate with destination tag:
  If a destination tag is required, the user enters the numeric tag value into
  the tag input field and clicks Generate QR. The QR is encoded from the URI:

    rADDRESSdt=TAGNUMBER

  where TAGNUMBER is the destination tag value passed through
  encodeURIComponent(). For a purely numeric tag, encodeURIComponent produces
  no escaping. The dt= parameter format is recognized by Coinbase, Robinhood,
  Binance, and XUMM/Xaman. The address label below the canvas displays the full
  URI including the dt= parameter so the user can verify the tag was included.

Step 2b ? Generate without destination tag:
  If no destination tag is needed (for example, the address is a self-custodied
  XRP wallet rather than an exchange deposit address), the user clicks
  Skip ? No Tag. The QR is encoded from the bare address string with no scheme
  prefix and no dt= parameter.
```

The two-step flow is triggered for any input that matches the XRP address pattern, regardless of whether a destination tag is actually required for that specific address. The user makes the determination at step 2; the tool does not and cannot determine whether a given XRP address requires a destination tag.

The Edit button (described in the Output Actions section below) returns to the appropriate step: for XRP addresses it returns to the destination tag prompt with the tag input focused; for all other coins it returns to the address input. This allows the address or tag to be corrected without reopening the modal.

Entry Points into the QR Modal

The QR modal is accessible from two independent entry points. Both entry points invoke the same generation pipeline and produce identical output for the same inputs.

```
Entry point 1 ? QR Export button:
  The QR Export button in the action toolbar opens the modal in a blank state.
  The address input is empty and focused. The user must type or paste an address
  manually. This entry point is independent of any prior derivation run; it
  does not pre-populate from the current derivation output. It is the primary
  entry point for generating a receive QR code from any known address, whether
  or not it was derived in the current session.

Entry point 2 ? Per-row QR button in Table View:
  Each row in the Table View output panel includes a small QR trigger button
  (?) in the address column. Clicking it opens the QR modal with the address
  field pre-populated from that row's address value. For XRP rows, the
  destination tag prompt is shown immediately because the address is already
```

```
  known to be an XRP address; the user proceeds directly to step 2 of the
  destination tag workflow.
```

Both entry points share the same modal instance. Opening the modal via either entry point resets all prior output state: the canvas, the address label, the save row, and the memo prompt are all hidden and their values are cleared before the modal is shown.

Output Actions

After a QR code has been generated and rendered, three action buttons appear below the canvas.

```
Save PNG:
  Saves or downloads the canvas content as a PNG image file. In a normal browser
  this uses the standard Blob download path. In the PySide6 GUI it uses the
  WebEngine download/save path. In the Tauri wrapper it uses the injected
  window.arculusTauriSaveExport bridge and native save_export command before any
  browser-style download path. The filename is constructed as
  qr_<sanitized_address>.png, where the sanitized address is the address string
  with all non-alphanumeric characters replaced by underscores, truncated to 40
  characters. The image is the raw canvas at its computed pixel dimensions
  (at most 256 pixels wide, adjusted to the nearest multiple of the module size).
  The PNG contains a black-on-white QR code with a 4-module quiet zone.

  The saved PNG is a Category A artifact (public address information only) unless
  the address input was populated with a private key or other sensitive material,
  which is not an intended use of this field. The PNG does not contain the URI
  string as metadata; only the visual QR pattern is saved.

Copy Address:
  Copies the address string from the address input field to the system clipboard
  with a 1.5-second Copied! confirmation on the button label. The copied value
  is the original address as entered, not the URI string. For example, for a
  Bitcoin address the clipboard receives the bare address (bc1q...) rather than
  the bitcoin: URI. This is the expected format for pasting into wallet software
  that handles scheme prefixes internally.

  Address strings are Category A material. Normal clipboard handling applies.
  Do not use this button to copy private keys or seed phrases into the clipboard
  from any other part of the tool.

Edit:
  Collapses the QR output back to the input step without closing the modal or
  clearing the address field. For XRP addresses, Edit returns to the destination
  tag prompt with focus on the tag input. For all other addresses, Edit returns
  focus to the address input. The current address value is preserved. This allows
  the address to be corrected or the destination tag to be changed without re-
  opening the modal and re-entering the address from scratch.
```

Capacity Limits and Failure Behavior

The encoder supports input strings up to the maximum byte capacity of QR version 10 at level Q, which is 180 bytes. For ASCII-only address strings (the output of all five supported coin address encoders), byte length equals character length.

Practical byte lengths of addresses and URIs generated by the tool:

```
  Bitcoin P2PKH (1...):         34 chars bare / 42 chars with bitcoin: prefix
  Bitcoin P2WPKH (bc1q...):     42 chars bare / 50 chars with bitcoin: prefix
  Bitcoin P2TR (bc1p...):       62 chars bare / 70 chars with bitcoin: prefix
  Litecoin P2WPKH (ltc1q...):   43 chars bare / 52 chars with litecoin: prefix
  Litecoin P2TR (ltc1p...):     63 chars bare / 72 chars with litecoin: prefix
  Dogecoin P2PKH (D...):        34 chars bare / 43 chars with dogecoin: prefix
  Ethereum (0x...):             42 chars bare / 51 chars with ethereum: prefix
  XRP classic (r...):           25-35 chars bare (no scheme prefix)
  XRP with destination tag:     25-35 chars + "dt=" + up to 10 digits
```

All of the above are well within the 180-byte capacity ceiling at level Q. None require a QR version above 3 or 4. The encoder will not be capacity-limited for any address format produced by the derivation engine.

If the encoder is called with input that exceeds the version 10 level Q capacity, generate() throws an Error. The QR UI catches this error in renderQrFromAddress() and displays the error message in the address label. The canvas is hidden. No partial or malformed QR is rendered. This failure path is unreachable in normal use; it can only be triggered by manually typing a string exceeding 180 bytes into the address input field.

Security Considerations for QR Use

Addresses encoded into QR codes are Category A material as defined in OpSec.txt. Generating a receive QR code for an address and scanning it with a wallet application on a networked device does not expose any private key material. The QR code contains only the address (and destination tag for XRP), which is public by construction. This operation is safe to perform on a networked device after the airgapped session is complete.

The following practices are not safe and must be avoided:

```
Do not encode private keys, WIF strings, extended private keys, seed phrases,
or .arc file contents into QR codes. The QR Export modal accepts arbitrary
text input and will encode any string the user provides. The tool does not
validate that the input is a receive address rather than sensitive key material.
The burden of ensuring that only Category A material is encoded is on the user.

Do not save QR code PNGs that encode private keys to networked storage, cloud
photo libraries, or any system with network connectivity. A QR code PNG
containing a private key is a Category B artifact and must be handled with the
same controls as any other private key export.

Do not scan a QR code generated from a derived address while the airgapped
machine is online. The workflow for using a derived address for a receive
payment is: complete the airgapped derivation session, export the address list,
clear the session, reconnect to a network on a separate device, and scan or
copy the address there. The QR Save PNG function can transfer the address from
the airgapped machine to a USB drive for this purpose without requiring the
airgapped machine to go online.
```

Implementation Reference

QRGen module public interface:

```
QRGen.generate(text)
    Input:  UTF-8 string to encode.
    Output: 2D array (matrix) of boolean and integer cell values.
            Structural cells are integers (1 = dark, 0 = light).
            Data/EC cells are booleans (true = dark, false = light).
    Throws: Error if input exceeds version 10 level Q capacity.
    Side effects: none.

QRGen.render(matrix, canvas, opts)
    Input:  matrix from generate(); HTML canvas element; options object.
    Options:
      size    integer  Target canvas pixel dimension (default: 256).
      quiet   integer  Quiet zone width in modules (default: 3; set to 4
                       in the QR Export modal).
      dark    string   CSS color string for dark modules (default: '#000000').
      light   string   CSS color string for light modules (default: '#ffffff').
    Output: canvas is drawn in place. Returns undefined.
    Side effects: sets canvas.width and canvas.height; clears and redraws
                  the 2D context.
```

Helper functions:

```
isXrpAddress(address)
    Returns true if address matches /^r[1-9A-HJ-NP-Za-km-z]{24,}/ after
    trimming. Used by both toBip21Uri() and the QR UI to dispatch the XRP
    two-step workflow.

toBip21Uri(address, memo)
    Returns the URI string to encode. address is the raw address string.
    memo is the destination tag string for XRP; ignored for all other coins.
    Detection rules are applied in the order documented in the URI Scheme
    Auto-Detection section above.

renderQrFromAddress(address, memo)
    Calls toBip21Uri(), then QRGen.generate(), then QRGen.render() with
    fixed parameters (size: 256, quiet: 4, dark: '#000000', light: '#ffffff').
    On success: shows canvas, shows address label with the URI string, shows
    the save row. On failure: hides canvas, shows error message in label,
    hides save row.

resetQrOutput()
    Hides canvas, address label, save row, and memo prompt. Called when the
    modal is opened and when Edit is clicked.

openQrModal()
```

```
    Calls resetQrOutput(), clears address and memo inputs, opens the dialog
    via showModal() (or setAttribute('open') on browsers without showModal
    support), and focuses the address input after a 50ms delay.

bindQrUI()
    Registers all event listeners for the QR modal: open button, close button,
    backdrop click, Generate button, address input Enter key, XRP memo prompt
    Confirm and Skip buttons, memo input Enter key, Save PNG button, Copy
    Address button, and Edit button. Called once at page load as part of the
    main UI initialization. Listener registration is not repeated on subsequent
    modal opens.
```

Arculus Recovery v1.5.0 - Canonical Test Vector Suite

Scope

This document is the authoritative, machine-verifiable test vector suite for Arculus Recovery v1.5.0. It defines a complete set of known-good input/output pairs for every cryptographic pipeline stage the engine implements: BIP39 entropy and checksum derivation, PBKDF2 seed stretching, BIP32 master node construction, CKDpriv child derivation, all supported script types and address families, Taproot key material, and the .arc encryption subsystem.

Every value in this document was produced by running the reference Python implementation against the stated inputs and recording the output verbatim. No value has been computed by hand, approximated, or transcribed from a third-party source without independent verification against the implementation itself. Any deviation between a reimplementation's output and the values below is a regression in the reimplementation. Partial agreement on address format without agreement on private key hex is not sufficient for compliance.

Two independent implementations are considered specification-equivalent if and only if they produce byte-identical output for every vector in this document. This is a binary criterion. There is no partial compliance.

The test mnemonic inputs used in this document are the standard BIP39 community test vectors in common use for validating HD wallet implementations. They appear publicly in the BIP39 specification and in reference implementations. They are used here because their widespread use makes independent cross-verification possible. The private key material derived from these mnemonics controls no funds of value; the mnemonics are used exclusively as deterministic test inputs.

Test Vector Conventions

All hexadecimal strings are lowercase and contain no spaces or separators. All addresses are presented in their canonical encoded form for the coin and script type: Base58Check for P2PKH and P2WPKH-P2SH, Bech32 for P2WPKH, Bech32m for P2TR, EIP-55 checksummed hex for Ethereum, and XRPL-alphabet Base58Check for XRP. Paths use apostrophe notation for hardened components. Extended keys use the SLIP132 prefix appropriate for the script type and network.

The following shorthand applies throughout:

```
  TV1    12-word all-zeros mnemonic, empty passphrase
  TV1-T  12-word all-zeros mnemonic, passphrase "TREZOR"
  TV2    24-word all-zeros mnemonic, empty passphrase
  TV3    12-word all-max mnemonic (zoo x11 + wrong), empty passphrase
```

BIP39 Mnemonic Inputs

```
TV1 (12-word, empty passphrase):
  mnemonic:  abandon abandon abandon abandon abandon abandon abandon abandon
             abandon abandon abandon about
  passphrase: (empty string)
  word_count: 12
  entropy:    5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc1
  checksum:   (0011) = 3 = "about"
  entropy_bits: 128
  checksum_bits: 4

TV1-T (12-word, "TREZOR" passphrase):
  mnemonic:  (same as TV1)
  passphrase: TREZOR

TV2 (24-word, empty passphrase):
  mnemonic:  abandon abandon abandon abandon abandon abandon abandon abandon
             abandon abandon abandon abandon abandon abandon abandon abandon
             abandon abandon abandon abandon abandon abandon abandon art
  passphrase: (empty string)
  word_count: 24
  entropy:    0000000000000000000000000000000000000000000000000000000000000000
  checksum:   (01100110) = 102 = "art"
  entropy_bits: 256
  checksum_bits: 8

TV3 (12-word all-max, empty passphrase):
  mnemonic:  zoo zoo zoo zoo zoo zoo zoo zoo zoo zoo zoo wrong
  passphrase: (empty string)
  word_count: 12
  entropy:    ffffffffffffffffffffffffffffffff
  checksum:   (0101) = 5 = "wrong"
```

```
  entropy_bits: 128
  checksum_bits: 4
```

Checksum verification: The BIP39 checksum for each vector can be verified independently by computing SHA256 of the raw entropy bytes and confirming that the first checksum_bits bits of the digest match the encoded checksum value.

```
  SHA256(0x5eb00bbd...a5fc1) = 374708fff7719dd5979ec875d56cd2286f6d3cf7...
    first 4 bits: 0011 = 3   checksum word index: 3 = "about"   VALID

  SHA256(0x000...000[32 bytes]) = 66687aadf862bd776c8fc18b8e9f8e200897148...
    first 8 bits: 01100110 = 102   checksum word index: 102 = "art"   VALID

  SHA256(0xfff...fff[16 bytes]) = 5ac6a5945f16500911219129984ba8b387a06f2...
    first 4 bits: 0101 = 5   checksum word index: 5 = "wrong"   VALID
```

BIP39 Seed Derivation Vectors

PBKDF2-HMAC-SHA512, 2048 iterations, 64-byte output. password = NFKD(mnemonic), salt = b"mnemonic" + NFKD(passphrase)

```
TV1 seed (empty passphrase):
  5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc1
  9a5ac40b389cd370d086206dec8aa6c43daea6690f20ad3d8d48b2d2ce9e38e4

TV1-T seed (passphrase "TREZOR"):
  c55257c360c07c72029aebc1b53c05ed0362ada38ead3e3e9efa3708e5349553
  1f09a6987599d18264c1e1c92f2cf141630c7a3c4ab7c81b2f001698e7463b04

TV2 seed (empty passphrase):
  408b285c123836004f4b8842c89324c1f01382450c0d439af345ba7fc49acf70
  5489c6fc77dbd4e3dc1dd8cc6bc9f043db8ada1e243c4a0eafb290d399480840

TV3 seed (empty passphrase):
  b6a6d8921942dd9806607ebc2750416b289adea669198769f2e15ed926c3aa92
  bf88ece232317b4ea463e84b0fcd3b53577812ee449ccc448eb45e6f544e25b6
```

The seed output for TV1 with an empty passphrase is 64 bytes encoded above as two 32-byte rows for readability. The implementation must produce these bytes as a contiguous 512-bit value. Leading zero bytes are significant.

BIP32 Master Node Vectors

HMAC-SHA512(key = b"Bitcoin seed", msg = seed). IL = bytes [0:32], IR = bytes [32:64].

```
TV1 (empty passphrase):
  master_private_key: 1837c1be8e2995ec11cda2b066151be2cfb48adf9e47b151d46adab3a21cdf6
                      7  (note: 32 bytes, last nibble pair is 67)
  master_chain_code:  7923408dadd3c7b56eed15567707ae5e5dca089de972e07f3b860450e2a3b70e
  root_fingerprint:   73c5da0a

TV1-T (passphrase "TREZOR"):
  master_private_key: cbedc75b0d6412c85c79bc13875112ef912fd1e756631b5a00330866f22ff184
  master_chain_code:  a3fa8c983223306de0f0f65e74ebb1e98aba751633bf91d5fb56529aa5c132c1
  root_fingerprint:   b4e3f5ed

TV2 (24-word, empty passphrase):
  master_private_key: 235b34cd7c9f6d7e4595ffe9ae4b1cb5606df8aca2b527d20a07c8f56b2342f4
  master_chain_code:  f40eaad21641ca7cb5ac00f9ce21cac9ba070bb673a237f7bce57acda54386a4
  root_fingerprint:   5436d724

TV3 (zoo, empty passphrase):
  master_private_key: f1897f8c5fd5e11814f80c63eb21cc0c63d1623008f895e83c0e9a770fb66544
  master_chain_code:  216e77e7c2472af47b5849cd4d02e41ab62f7d62626aafcbdcda4cc4aeb7da7d
  root_fingerprint:   3f635a63
```

Root fingerprint computation: HASH160(serP(master_private * G))[0:4] where HASH160(x) = RIPEMD160(SHA256(x)) and serP() is compressed public key serialization. The fingerprint is 4 bytes rendered as 8 lowercase hex characters.

```
Root xprv/xpub for all TV1 variants (same master node regardless of passphrase):
  root_xprv: xprv9s21ZrQH143K3GJpoapnV8SFfukcVBSfeCficPSGfubmSFDxo1kuHnLisriDv
             SnRRuL2Qrg5ggqHKNVpxR86QEC8w35uxmGoggxtQTPvfUu
  root_xpub: xpub661MyMwAqRbcFkPHucMnrGNzDwb6teAX1RbKQmqtEF8kK3Z7LZ59qafCjB9e
             CRLiTVG3uxBxgKvRgbubRhqSKXnGGb1aoaqLrpMBDrVxga8

Root xprv/xpub for TV1-T (TREZOR passphrase):
  root_xprv: xprv9s21ZrQH143K3h3fDYiay8mocZ3afhfULfb5GX8kCBdno77K4HiA15Tg23wp
             beF1pLfs1c5SPmYHrEpTuuRhxMwvKDwqdKiGJS9XFKzUsAF
  root_xpub: xpub661MyMwAqRbcGB88KaFbLGiYAat55APKhtWg4uYMkXAmfuSTbq2QYsn9sKJCj
```

```
            1YqZPafsboef4h4YbXXhNhPwMbkHTpkf3zLhx7HvFw1NDy
```

Bitcoin P2PKH Derivation Vectors (TV1, m/44'/0'/0')

```
Script type: p2pkh
Coin:        bitcoin mainnet
Mnemonic:    TV1 (abandon x11 + about, empty passphrase)
Path:        m/44'/0'/0'

Account node:
  account_xprv: xprv9xpXFhFpqdQK3TmytPBqXtGSwS3DLjojFhTGht8gwAAii8py5X6pxeBnQ6eh
                JiyJ6nDjWGJfZ95WxByFXVkDxHXrqu53WCRGypk2ttuqncb
  account_xpub: xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi
                8Aw6ZRc1brxMyWMzG3DSSSSoekkudhUd9yLb6qx39T9nMdj
```

Receiving branch (m/44'/0'/0'/0/i):

```
  index 0:
    path:             m/44'/0'/0'/0/0
    address:          1LqBGSKuX5yYUonjxT5qGfpUsXKYYWeabA
    public_key_hex:   03aaeb52dd7494c361049de67cc680e83ebcbbbdbeb13637d92cd845f70308af5e
    private_key_hex:  e284129cc0922579a535bbf4d1a3b25773090d28c909bc0fed73b5e0222cc372
    private_key_wif:  L4p2b9VAf8k5aUahF1JCJUzZkgNEAqLfq8DDdQiyAprQAKSbu8hf

  index 1:
    path:             m/44'/0'/0'/0/1
    address:          1Ak8PffB2meyfYnbXZR9EGfLfFZVpzJvQP
    public_key_hex:   02dfcaec532010d704860e20ad6aff8cf3477164ffb02f93d45c552dadc70ed24f
    private_key_hex:  5c1141f60edd3095579529db7e88d964cb0a9ec0f814f6a10cd5cbd763078a0c
    private_key_wif:  KzJgGiEeGUVWmPR97pVWDnCVraZvM2fnrCVrg2irV4353HciE6Un

  index 2:
    path:             m/44'/0'/0'/0/2
    address:          1MNF5RSaabFwcbtJirJwKnDytsXXEsVsNb
    public_key_hex:   0338994349b3a804c44bbec55c2824443ebb9e475dfdad14f4b1a01a97d42751b3
    private_key_hex:  cfa32fc333b7297ca611c33f046d96895071672e1e5d9b77f3492b3c8d0149f9
    private_key_wif:  L4BL9ZGzuQJFoRqGfjsgHeYzD1C72y2VmJaY6sqdtaRkfxUFrJXu
```

Change branch (m/44'/0'/0'/1/i):

```
  index 0:
    path:             m/44'/0'/0'/1/0
    address:          1J3J6EvPrv8q6AC3VCjWV45Uf3nssNMRtH
    public_key_hex:   03498b3ac8e882c5d693540c49adf22b7a1b99c1bb8047966739bfe8cdeb272e64
    private_key_hex:  78df181d8d74216a5c1398689b35aada58cc42e5f056b6126c1c4f6e236294c7
    private_key_wif:  L1GfmUBVD88haCukMzGtmR5B5zuQVxd6cmUVe85d66Uq2V13orj3

  index 1:
    path:             m/44'/0'/0'/1/1
    address:          13vKxXzHXXd8HquAYdpkJoi9ULVXUgfpS5
    public_key_hex:   03f26f242c4851fbc74c817dbb1cd3c99993cd078470117d8d0473de3273ad1c92
    private_key_hex:  57f3997c7615a0539ad86397ecbe2b1b20013301db0d63819cae947a94c71a56
    private_key_wif:  KzAgERsWYhtJ23cYUabmK8eVSgXx5vsHqfau4ujjBXRVHk6Aak64

  index 2:
    path:             m/44'/0'/0'/1/2
    address:          1M21Wx1nGrHMPaz52N2En7c624nzL4MYTk
    public_key_hex:   03a1a2116eb8bb53408926a304427d2077b7e02ed849baf24d6cc091575efa2185
    private_key_hex:  179cc36a94bf2eb024c744de12a0d94072bdf902f11f8ef3db0f4c5359ea1017
    private_key_wif:  Kx1cM2PneESSNQA8B4g94dtH1sHywso9Mt76JVTnHm4PKrkX4Kth
```

Bitcoin P2PKH Derivation Vectors (TV1-T, TREZOR passphrase, m/44'/0'/0')

```
Script type: p2pkh
Coin:        bitcoin mainnet
Mnemonic:    TV1-T (abandon x11 + about, passphrase "TREZOR")
Path:        m/44'/0'/0'

Account node:
  account_xprv: xprv9z3rKW6EbJfnxwVmsXBRWABiT5Zfh37GsqTXrcdaMssUntP2wumTtqKxSkZs
                ytaxQZknwAhb3U8UR5cc3cxoMxdo4871tPPCTmeqckJyrWL
  account_xpub: xpub6D3Cj1d8RgE6BRaEyYiRsJ8T17QA6Vq8F4P8f13BvDQTfgiBVT5iSdeSJ2QL
                SRijq2PMBXRSgduEUq11mYggQz6vUEe7Ga9e86urZjkrmeR
```

Receiving branch (m/44'/0'/0'/0/i):

```
  index 0:
    path:             m/44'/0'/0'/0/0
    address:          1PEha8dk5Me5J1rZWpgqSt5F4BroTBLS5y
```

```
    public_key_hex:   027440c6c46ec617a202f44bc886a249b10f98a8ff5d8a0aa56a350ab930a0ec79
    private_key_hex:  cdd74cbef2372344879b8a0aa8799435ff55bf5bde335638cb7a8d09fd0f9759
    private_key_wif:  L47qcNDdda3QMACwfisBm5XHrXvzTLd9H9Cxz3LBH2J8EBPFvMGo

  index 1:
    path:             m/44'/0'/0'/0/1
    address:          1NWFrd38ng3DRt3oqAP1rAJZeDunek7E33
    public_key_hex:   03b64236b2c8f34a18e3a584fe0877fb944e2abb4544cb14bee5458bcc2480cefc
    private_key_hex:  eb9cbdfcfcdf6b682fae39cd133e587b6f238027ef541a1a28bb370edc085493
    private_key_wif:  L57i9t8wnUTEuMuqRJPbn5DJuzb5kMart79aSNQLzfBjuLgCve9x
```

Change branch (m/44'/0'/0'/1/i):

```
  index 0:
    path:             m/44'/0'/0'/1/0
    address:          1MVkh45Zn9A7ZBga697r34DBiRFLXD4UKG
    public_key_hex:   02410fcdf3a7401abf65c925c5aaf13797aeb0d4a2fa93006b427495dc5c5ba0fe
    private_key_hex:  9491d1539810480f012fb3ec8cf293984257afa7cf68f47a1175ae36ebe39bc6
    private_key_wif:  L2CWaV2UFB7bUqWA3tjaqtu4opBzCw5ZdDwEwFxUjxj4yr4Lqq8g
```

This vector demonstrates that identical mnemonic with a non-empty passphrase produces an entirely disjoint key tree with zero cryptographic relationship to the empty-passphrase derivation. TV1 index 0 receiving address 1LqBGSKuX5yY... and TV1-T index 0 receiving address 1PEha8dk5Me5... share no common key material, extended key, chain code, or fingerprint with each other.

Bitcoin P2WPKH-P2SH Derivation Vectors (TV1, m/49'/0'/0')

```
Script type: p2wpkh-p2sh
Coin:        bitcoin mainnet
Mnemonic:    TV1
Path:        m/49'/0'/0'

Account node:
  account_xprv: xprv9y7S1RkggDtZnP1RSzJ7PwUR4MUfF66Wz2jGv9TwJM52WLGmnnrQLLzBSTi7
                rNtBk4SGeQHBj5G4CuQvPXSn58BmhvX9vk6YzcMm37VuNYD
  account_xpub: xpub6C6nQwHaWbSrzs5tZ1q7m5R9cPK9eYpNMFesiXsYrgc1P8bvLLAet9JfHjYX
                KjToD8cBRswJXXbbFpXgwsswVPAZzKMa1jUp2kVkGVUaJa7
  account_yprv: yprvAHwhK6RbpuS3dgCYHM5jc2ZvEKd7Bi61u9FVhYMpgMSuZS613T1xxQeKTffhr
                HY79hZ5PsskBjcc6C2V7DrnsMsNaGDaWev3GLRQRgV7hxF
  account_ypub: ypub6Ww3ibxVfGzLrAH1PNcjyAWenMTbbAosGNB6VvmSEgytSER9azLDWCxoJwW7K
                e7icmizBMXrzBx9979FfaHxHcrArf3zbeJJJUZPf663zsP
```

Receiving branch (m/49'/0'/0'/0/i):

```
  index 0:
    path:             m/49'/0'/0'/0/0
    address:          37VucYSaXLCAsxYyAPfbSi9eh4iEcbShgf
    public_key_hex:   039b3b694b8fc5b5e07fb069c783cac754f5d38c3e08bed1960e31fdb1dda35c24
    private_key_hex:  508c73a06f6b6c817238ba61be232f5080ea4616c54f94771156934666d38ee3
    private_key_wif:  KyvHbRLNXfXaHuZb3QRaeqA5wovkjg4RuUpFGCxdH5UWc1Foih9o

  index 1:
    path:             m/49'/0'/0'/0/1
    address:          3LtMnn87fqUeHBUG414p9CWwnoV6E2pNKS
    public_key_hex:   022a421fa4a65a87d1c3e4238155d85f7bd2c5bb87632f331b5722f110586aa198
    private_key_hex:  464c5dd427dcf1e2791b97a1aa9348647d3a55e1223b4e58cb663b49fd12e0ca
    private_key_wif:  KyaMvgopkPDQMQUx2w9a8AiEtA7A84hYzASJWGQiKZ8AJUEj77iV
```

Change branch (m/49'/0'/0'/1/i):

```
  index 0:
    path:             m/49'/0'/0'/1/0
    address:          34K56kSjgUCUSD8GTtuF7c9Zzwokbs6uZ7
    public_key_hex:   02b4019c64bb1347bd729a6afa11348bd80be4ebc314df03f654f786bfe2b4a728
    private_key_hex:  c51d7f978812260fd5cf2271dbb9c5ba955eb3c956e5d68a3c5cfd4489f442ee
    private_key_wif:  L3pspue7Ag5bBvfo4EHBASUQfhBAfh4WWU9XwGf6mBXy5GNY2UnS

  index 1:
    path:             m/49'/0'/0'/1/1
    address:          3516F2wmK51jVRrggEJsTUBNWMSLLjzvJ2
    public_key_hex:   02fc279564cb56cfe4a9ff2473727476ad6ad5f6b4c5bb012ffbf4a4fbb0f713c2
    private_key_hex:  e797c8adfb33101943f570e4e6fb0dcedbe1cf80e9e0695daa3e2d8c6c31020c
    private_key_wif:  L4ytzXRVMru6Xgd77Hm2a3DztLupRJZu9nqHcaURLPDFAnp18kon

Address construction verification for P2WPKH-P2SH at m/49'/0'/0'/0/0:
  pubkey:        039b3b694b8fc5b5e07fb069c783cac754f5d38c3e08bed1960e31fdb1dda35c24
  pkh:           RIPEMD160(SHA256(pubkey)) = 336caa13e08b96080a32b5d818d59b4ab3b36742
  redeem_script: 0x00 0x14 <pkh> = 0014336caa13e08b96080a32b5d818d59b4ab3b36742
```

```
  script_hash:   HASH160(redeem_script) = 1f1f4b87f6c7de67c0bafb01e483cd62a4f6abb3
  address:       Base58Check(0x05 || script_hash) = 37VucYSaXLCAsxYyAPfbSi9eh4iEcbShgf
```

Bitcoin P2WPKH (Native SegWit) Derivation Vectors (TV1, m/84'/0'/0')

```
Script type: p2wpkh
Coin:        bitcoin mainnet
Mnemonic:    TV1
Path:        m/84'/0'/0'

Account node:
  account_xprv: xprv9ybY78BftS5UGANki6oSifuQEjkpyAC8ZmBvBNTshQnCBcxnefjHS7buPMkk
                qhcRzmoGZ5bokx7GuyDAiktd5HemohAU4wV1ZPMDRmLpBMm
  account_xpub: xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvq
                AQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVUhLv1VMrjPC7PW6V
  account_zprv: zprvAdG4iTXWBoARxkkzNpNh8r6Qag3irQB8PzEMkAFeTRXxHpbF9z4QgEvBRmfv
                qWvGp42t42nvgGpNgYSJA9iefm1yYNZKEm7z6qUWCroSQnE
  account_zpub: zpub6rFR7y4Q2AijBEqTUquhVz398htDFrtymD9xYYfG1m4wAcvPhXNfE3EfH1r1A
                DqtfSdVCToUG868RvUUkgDKf31mGDtKsAYz2oz2AGutZYs
```

Receiving branch (m/84'/0'/0'/0/i):

```
  index 0:
    path:             m/84'/0'/0'/0/0
    address:          bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu
    public_key_hex:   0330d54fd0dd420a6e5f8d3624f5f3482cae350f79d5f0753bf5beef9c2d91af3c
    private_key_hex:  4604b4b710fe91f584fff084e1a9159fe4f8408fff380596a604948474ce4fa3
    private_key_wif:  KyZpNDKnfs94vbrwhJneDi77V6jF64PWPF8x5cdJb8ifgg2DUc9d

  index 1:
    path:             m/84'/0'/0'/0/1
    address:          bc1qnjg0jd8228aq7egyzacy8cys3knf9xvrerkf9g
    public_key_hex:   03e775fd51f0dfb8cd865d9ff1cca2a158cf651fe997fdc9fee9c1d3b5e995ea77
    private_key_hex:  2fd0affa51529f940a358ec0c50de81267d0bf5158ca61887347676946362c5b
    private_key_wif:  Kxpf5b8p3qX56DKEe5NqWbNUP9MnqoRFzZwHRtsFqhzuvUJsYZCy

  index 2:
    path:             m/84'/0'/0'/0/2
    address:          bc1qp59yckz4ae5c4efgw2s5wfyvrz0ala7rgvuz8z
    public_key_hex:   038ffea936b2df76bf31220ebd56a34b30c6b86f40d3bd92664e2f5f98488dddfa
    private_key_hex:  7ffd78b405ea6fbcf0cbc67ae6d87aea9036b72b7fab59fec45e218262927424
    private_key_wif:  L1WWMekCNikUJwrkmxqXWafFu3gmzJ777WpPGgz5Y1o7U11hrDDs
```

Change branch (m/84'/0'/0'/1/i):

```
  index 0:
    path:             m/84'/0'/0'/1/0
    address:          bc1q8c6fshw2dlwun7ekn9qwf37cu2rn755upcp6el
    public_key_hex:   03025324888e429ab8e3dbaf1f7802648b9cd01e9b418485c5fa4c1b9b5700e1a6
    private_key_hex:  3277578a56b721e4c9f071f1e24aa0f94c4ff72e7967fea03b134f605f07c8fd
    private_key_wif:  KxuoxufJL5csa1Wieb2kp29VNdn92Us8CoaUG3aGtPtcF3AzeXvF

  index 1:
    path:             m/84'/0'/0'/1/1
    address:          bc1qggnasd834t54yulsep6fta8lpjekv4zj6gv5rf
    public_key_hex:   03dcf71df71c755b3af46f7e84b4182e6291cec5a8c630f775638a739da29adcb6
    private_key_hex:  3b78e4757b2c4ae2bca054b059cb84b3da7e3440edc7b5782dbc04fbec93d310
    private_key_wif:  KyDKM6os4SNpyCN79CGaZF91vVtzmnragXN7A3qAxVvFDws9jBqh

  index 2:
    path:             m/84'/0'/0'/1/2
    address:          bc1qn8alfh45rlsj44pcdt0f2cadtztgnz4gq3h3uf
    public_key_hex:   029ba8d4b13466ad8f78cd1208f60023ee568e94a3fe3c3ff4a4ccdbc5ba0627fa
    private_key_hex:  054cc87a6253d85a269c31b0b3debbe9d37fdf9ceca2e1c72f3dadf2d218430b
    private_key_wif:  KwQ1irwXPWRmfEH7CtyiFSA2dZ8TCrczAa7onpr3yNEEPrxGmnfS
```

Bitcoin P2TR (Taproot) Derivation Vectors (TV1, m/86'/0'/0')

```
Script type: p2tr
Coin:        bitcoin mainnet
Mnemonic:    TV1
Path:        m/86'/0'/0'

Account node:
  account_xprv: xprv9xgqHN7yz9MwCkxsBPN5qetuNdQSUttZNKw1dcYTV4mkaAFiBVGQziHs3NRS
                WMkCzvgjEe3n9xV8oYywvM8at9yRqyaZVz6TYYhX98VjsUk
  account_xpub: xpub6BgBgsespWvERF3LHQu6CnqdvfEvtMcQjYrcRzx53QJjSxarj2afYWcLteoGV
                ky7D3UKDP9QyrLprQ3VCECoY49yfdDEHGCtMMj92pReUsQ
```

Receiving branch (m/86'/0'/0'/0/i):

```
  index 0:
    path:                           m/86'/0'/0'/0/0
    address:
bc1p5cyxnuxmeuwuvkwfem96lqzszd02n6xdcjrs20cac6yqjjwudpxqkedrcr
    public_key_hex:
03cc8a4bc64d897bddc5fbc2f670f7a8ba0b386779106cf1223c6fc5d7cd6fc115
    private_key_hex:
41f41d69260df4cf277826a9b65a3717e4eeddbeedf637f212ca096576479361
    private_key_wif:                KyRv5iFPHG7iB5E4CqvMzH3WFJVhbfYK4VY7XAedd9Ys69mEsPLQ
    internal_public_key_hex:
cc8a4bc64d897bddc5fbc2f670f7a8ba0b386779106cf1223c6fc5d7cd6fc115
    internal_private_key_hex:
be0be296d9f20b30d887d95649a5c8e6d5bfff27c1526849ad08552759eeade0
    internal_private_key_wif:       L3b8rmMoiFLdFvBMvdX9cTGKi7rg4xcG89TEZzyK3yzzD4Rjn8hh
    tweak_hex:
2ca01ed85cf6b6526f73d39a1111cd80333bfdc00ce98992859848a90a6f0258
    output_public_key_hex:
a60869f0dbcf1dc659c9cecbaf8050135ea9e8cdc487053f1dc6880949dc684c
    output_private_key_hex:
eaac016f36e8c18347fbacf05ab7966708fbfce7ce3bf1dc32a09dd0645db038
    output_private_key_wif:         L55t8bDSMHpURxd8RY8UfdNou5v1xD82xiZCorARGa4ZozTbfRA8
    output_key_parity:              1

  Note: parity = 1 at index 0 indicates that the internal public key has an odd
  Y coordinate, triggering the BIP340 scalar negation: internal_private is
  (n - child_private) mod n. Implementations must handle both parity values.

  index 1:
    path:                           m/86'/0'/0'/0/1
    address:
bc1p4qhjn9zdvkux4e44uhx8tc55attvtyu358kutcqkudyccelu0was9fqzwh
    public_key_hex:
0283dfe85a3151d2517290da461fe2815591ef69f2b18a2ce63f01697a8b313145
    private_key_hex:
86c68ac0ed7df88cbdd08a847c6d639f87d1234d40503abf3ac178ef7ddc05dd
    private_key_wif:                L1jhNnZZAAAppoSYQuaAQEj935VpmishMomuWXgJ3Qy5HNqkhhus
    internal_public_key_hex:
83dfe85a3151d2517290da461fe2815591ef69f2b18a2ce63f01697a8b313145
    internal_private_key_hex:
86c68ac0ed7df88cbdd08a847c6d639f87d1234d40503abf3ac178ef7ddc05dd
    internal_private_key_wif:       L1jhNnZZAAAppoSYQuaAQEj935VpmishMomuWXgJ3Qy5HNqkhhus
    tweak_hex:
84a88d9651f7cbc831e3b2a7800f2e572b6a719c3352dd9e6d3125cd21827237
    output_public_key_hex:
a82f29944d65b86ae6b5e5cc75e294ead6c59391a1edc5e016e3498c67fc7bbb
    output_private_key_hex:
0b6f18573f75c454efb43d2bfc7c91f7f88cb802c45a7821e820402fcf2836d3
    output_private_key_wif:         KwbwJNcL5DKiDfwNNu9EQvyr5Q36oZg8PHfGnzHZmb3BxEzDhhDu
    output_key_parity:              0

  Note: parity = 0 at index 1 indicates even Y coordinate; no negation occurs.
  internal_private_key == private_key_hex in this case. Both parity paths are
  present within the first two indices of this vector set, providing coverage
  for both branches of the BIP340 parity normalization logic.
```

Change branch (m/86'/0'/0'/1/i):

```
  index 0:
    path:                           m/86'/0'/0'/1/0
    address:
bc1p3qkhfews2uk44qtvauqyr2ttdsw7svhkl9nkm9s9c3x4ax5h60wqwruhk7
    internal_public_key_hex:
399f1b2f4393f29a18c937859c5dd8a77350103157eb880f02e8c08214277cef
    internal_private_key_hex:
6ccbca4a02ac648702dde463d9c1b0d328a4df1e068ef9dc2bc788b33a4f0412
    tweak_hex:
563b7c38f218e910fefc744150ed5d5597e4f6dabff583ec56cfac08a9a70235
    output_public_key_hex:
882d74e5d0572d5a816cef0041a96b6c1de832f6f9676d9605c44d5e9a97d3dc
    output_private_key_hex:
c3074682f4c54d9801da58a52aaf0e28c089d5f8c6847dc8829734bbe3f60647
    output_key_parity:              1

  index 1:
    path:                           m/86'/0'/0'/1/1
    address:
bc1ptdg60grjk9t3qqcqczp4tlyy3z47yrx9nhlrjsmw36q5a72lhdrs9f00nj
```

```
    internal_public_key_hex:
da01548c72c04619d079ec0118a48bda801cc9391b00de5c846bfdc9807905bd
    internal_private_key_hex:
c8d522210e3bc028586dcb7cf7dce3937440ad8132765ecdfa2ff78d0e9a5d32
    tweak_hex:
ac209c114a9bff5ed14ac66d4e4c587abb7ba4029cf26b75c31c99b42ac19a04
    output_public_key_hex:
5b51a7a072b157100300c08355fc8488abe20cc59dfe39436e8e814ef95fbb47
    output_private_key_hex:
74f5be3258d7bf8729b891ea46293c0f750d749d20202a07fd7a32b46925b5f5
    output_key_parity:             1

TapTweak computation at index 0 (m/86'/0'/0'/0/0):
  tagged_hash("TapTweak", internal_x) where internal_x is the 32-byte x-only
  representation of internal_public_key:
  tag_hash = SHA256("TapTweak") =
e80fe1639c9ca050e3af1b39c143c63e429cbceb15d940fbb5c5a2f4af59347d
  tweak = SHA256(tag_hash || tag_hash || internal_x) mod n = 2ca01ed8...a6f0258
  This value is confirmed in the tweak_hex field above.
```

Bitcoin P2WPKH Derivation Vectors (TV3, zoo vector, m/84'/0'/0')

```
Script type: p2wpkh
Coin:        bitcoin mainnet
Mnemonic:    TV3 (zoo x11 + wrong, empty passphrase)
Path:        m/84'/0'/0'
```

This vector confirms that maximum-value input entropy produces structurally correct derivation with no special-casing or wraparound in scalar arithmetic.

```
Root node:
  root_xprv: xprv9s21ZrQH143K2PfMvkNViFc1fgumGqBew45JD8SxA59Jc5M66n3diqb92JjvaR6
             1zT9P89Grys12kdtV4EFVo6tMwER7U2hcUmZ9VfMYPLC
  root_xpub: xpub661MyMwAqRbcEsjq2muW5PYkDikFgHuWJGzu1WrZiQgHUsgEeKMtGducsZe1iRs
             GAGNGDzmWYDM69ya24LMyR7mDhtzqQsc286XEQfM2kkV
```

Receiving branch (m/84'/0'/0'/0/i):

```
  index 0:
    path:             m/84'/0'/0'/0/0
    address:          bc1qk0a9hr7wjfxeenz9nwenw9flhq0tmsf6vsgnn2
    public_key_hex:   030756317ceec8a038de22a93abac5ea45b6cf496a9a32fb19008c460f468da4ed
    private_key_hex:  8e0a6c04131238359ec7ed175b315fa743374caadfccba4949543d2dfa7f12fc
    private_key_wif:  L1ypUvV7SHhvuC4FgueiGLhCf8JSpUhtQ5ojN824KdA88pGcykm9

  index 1:
    path:             m/84'/0'/0'/0/1
    address:          bc1qkd33yck74lg0kaq4tdcmu3hk4yruhjayxpe9ug
    public_key_hex:   03c010b38763c43a0420a4d4827ec6a4d0041dfcf0fa37b2f4bb4447c4701afe84
    private_key_hex:  5829393926adaf8a4aae0d674148264022a9b03f601aa98fff0cf03a77acd0b8
    private_key_wif:  KzB5rBWeDxHuT1JqKiQ3DVgDPA7YYfLg6jkKF2shSi92cJiR4Vey
```

Change branch (m/84'/0'/0'/1/i):

```
  index 0:
    path:             m/84'/0'/0'/1/0
    address:          bc1q3pgnz3gszmccgxlrygmp086w8j5rkhynx7cckt
    public_key_hex:   033057150eb57e2b21d69866747f3d377e928f864fa88ecc5ddb1c0e501cce3f81
    private_key_hex:  f0115346b010d326f7d616ecff0347c3dafb5c00f16ce382df9da24f2c9c4075
    private_key_wif:  L5GNUYm175x9BWhRTf4dzFGZCR72FnENEUMv1pUEGjWSdSNeiQ9f

  index 1:
    path:             m/84'/0'/0'/1/1
    address:          bc1q3ws8shjjmwx8y9p2hq8rw3lphfj4mukgqxwjfr
    public_key_hex:   0345b5926d5215f8cb9083e6f7aa2b5ee5ecccc1ddd0dfb8c906d516c0f5f40e0b
    private_key_hex:  12781f569fbc6f9289e714a1ff836085ff6a296c0e462ab2f28ea051cea66275
    private_key_wif:  KwqcVCX4Cvjsh4vBnv1RxVo6HnxBjzMFac9zJq5S75LgsfisbRsv
```

Bitcoin P2PKH Derivation Vectors (TV2, 24-word, m/44'/0'/0')

```
Script type: p2pkh
Coin:        bitcoin mainnet
Mnemonic:    TV2 (abandon x23 + art, empty passphrase)
Path:        m/44'/0'/0'

Root node:
  root_xprv: xprv9s21ZrQH143K4VHfAaPWRTm4aoHAZhJHunsZZTQptR82FSTZRjBGXBP8kQKHrUV
             UE8vMM2Z3h7UoG9x9XCt9FHQ1t1nHU7zQDqrEszAg28q
```

```
  root_xpub: xpub661MyMwAqRbcGyN8GbvWnbho8q7eyA29H1oAMqpSSkf18EnhyGVX4yhcbfjXa8j
             9KW7APXiBmXXpfseCZy4whWaFo1xsQoTxLPYJH17sBeA
```

Receiving branch (m/44'/0'/0'/0/i):

```
  index 0:
    path:             m/44'/0'/0'/0/0
    address:          1KBdbBJRVYffWHWWZ1moECfdVBSEnDpLHi
    public_key_hex:   0342d943b8dba93a4ce29b858479c67f1e4f1110eecbe1f83dc01b455eb8b123b3
    private_key_hex:  cb4793fa23e98437e2d37655eaaa59cf03a0637ea890f85a85d91946e9214c64
    private_key_wif:  L42rpqMcjt1LtyvZCSTLkaif5mjFyTXTHSVuckRZEM7GaD2KLCkc

  index 1:
    path:             m/44'/0'/0'/0/1
    address:          1EiJMaaahrhpbhgaNzMeUe1ZoiXdbBhWhR
    public_key_hex:   03208ff7044ba71c9604c42b9083344d6347a53a23f537f8880d052e681038d7da
    private_key_hex:  5f88e8693da0d61128b33298eb045eb33ce05e153171aaa5333236a6a54564fb
    private_key_wif:  KzRRCYYT8ys3dxwQ7aeqXazH11ZsLqNu9uTCwTUcEGBWiMySmwxN
```

Change branch (m/44'/0'/0'/1/i):

```
  index 0:
    path:             m/44'/0'/0'/1/0
    address:          1Jp3eQcCLpdE36vt5jguwjYaL2dWZCEgwB
    public_key_hex:   024f7f02f1338817bc5b448e83f56c6f67419945743cc7ec69cc1c7f2b5bcd9792
    private_key_hex:  e1bb72c89c433ff6e7a6e454ced0b7071fd564a0c3a9fc7d36c6de60dccc1f83
    private_key_wif:  L4nWEU68Dq1UPq1CqBhqkLfSZ1NAaA2vsgvtpHrp7ngbp43pcbxG

  index 1:
    path:             m/44'/0'/0'/1/1
    address:          1BT1w4HKJXkUL7gDFpfxBwRXFu8y65WNSe
    public_key_hex:   02a31f3dd56eb0cb37c70cc9ffd46754eefc29dea96a2ccbd4676554e15aa389e0
    private_key_hex:  4b681b990b7f151c3ad5df85eaafa8a794c44f850ad583308d9bae67611a9fa6
    private_key_wif:  KykHsBaFQtNfDQQstXtwWbDKzHShFtGiSnHZsEu4cvpVHimThs1r
```

Ethereum Derivation Vectors (TV1, m/44'/60'/0')

```
Coin:        ethereum mainnet
Mnemonic:    TV1
Path:        m/44'/60'/0'
Address:     EIP-55 checksummed 0x-prefixed hex

Account node:
  account_xprv: xprv9zDSoJv1aBcjX6sNgEpE2J9K6MV2MUnXuqXsFgzVn3zY2aHyupaFQdYCtdCbNM
                kvcTdx9FeN49sgXw6mjrhrFLRSzJVnRYPfSCCgjeg4GxY
  account_xpub: xpub6DCoCpSuQZB2jawqnGMEPS63ePKWkwWPH4TU45Q7LPXWuNd8TMtVxRrgjtEshu
                qpK3mdhaWHPFsBngh5GFZaM6si3yZdUsT8ddYM3PwnATt
```

Receiving branch (m/44'/60'/0'/0/i):

```
  index 0:
    path:                       m/44'/60'/0'/0/0
    address:                    0x9858EfFD232B4033E47d90003D41EC34EcaEda94
    public_key_hex:
0237b0bb7a8288d38ed49a524b5dc98cff3eb5ca824c9f9dc0dfdb3d9cd600f299
    ethereum_public_key_hex:
37b0bb7a8288d38ed49a524b5dc98cff3eb5ca824c9f9dc0dfdb3d9cd600f299

a6179912b7451c09896c4098eca7ce6b2e58330672795e847c4d6af44e024230
    private_key_hex:
1ab42cc412b618bdea3a599e3c9bae199ebf030895b039e9db1e30dafb12b727
    erc20_address:              0x9858EfFD232B4033E47d90003D41EC34EcaEda94

  index 1:
    path:                       m/44'/60'/0'/0/1
    address:                    0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0
    public_key_hex:
039fd0991d0222b4e1339c1a1a5b5f6d9f6a96672a3247b638ee6156d9ea877a2f
    private_key_hex:
9a983cb3d832fbde5ab49d692b7a8bf5b5d232479c99333d0fc8e1d21f1b55b6

  index 2:
    path:                       m/44'/60'/0'/0/2
    address:                    0xb6716976A3ebe8D39aCEB04372f22Ff8e6802D7A
    public_key_hex:
03880bcb4bf46b49bdb071e307e282b11b9166907d9708f8c706092c44743f3e67
    private_key_hex:
5b824bd1104617939cd07c117ddc4301eb5beeca0904f964158963d69ab9d831
```

Change branch (m/44'/60'/0'/1/i):

```
  index 0:
    path:                     m/44'/60'/0'/1/0
    address:                  0x399Db6Ed32539fbDF44c3e7678b5b428e378F666
    public_key_hex:
03abdc0424e5a951abb5e12df771738e269566fc170dfe8a5f0dae27052a168559
    private_key_hex:
dcea9371bd9641a066ad61b6542ffa3eb2835cababbf202ce2250c726ce736b3

  index 1:
    path:                     m/44'/60'/0'/1/1
    address:                  0x26db4d065800Bd118928848E69A1cBF956Cff1D0
    public_key_hex:
02dd553226bc6d4d2efafabfe7600a10d4d069de1371dd3207a293fb96766b3073
    private_key_hex:
011fe87587edf9d5bad64e253778ed929ecf319302bd41f4ce051e2678052cc1
```

Note: The Ethereum public key is the uncompressed 64-byte x||y representation without the 0x04 prefix byte. EIP-55 checksum is computed via Keccak-256 of the lowercase hex address string, not SHA3-256. These are distinct hash functions; substituting NIST SHA3-256 produces incorrect checksums. The implementation's inline pure-Python Keccak-256 must produce identical checksums to this vector.

Keccak-256 self-check: Keccak-256(b"") is not tested here, but implementors may verify their Keccak implementation against the known-good value c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470.

XRP Derivation Vectors (TV1, m/44'/144'/0')

```
Coin:        xrp mainnet
Mnemonic:    TV1
Path:        m/44'/144'/0'
Address:     XRPL classic r-address (Base58Check with XRPL alphabet)

Account node:
  account_xprv: xprv9yFya3vnAMVKmALsNBnw3G98Trzy4AmqciPkhNSZUNUuhTvTA3gWQRg2ecJFS3
                PDLsfYFgwDW1UukaapjTDUENfiwg22ryd4mWiph8Faw3p
  account_xpub: xpub6CFKyZTfzj3cyeRLUDKwQQ5s1tqTTdVgywKMVkrB2i1taGFbhazkxDzWVsfBHz
                pv7rg6qpDBGYR5oA8iazEfa44CdQkkknPFHJ7YCzncCS9
```

Receiving branch (m/44'/144'/0'/0/i):

```
  index 0:
    path:                m/44'/144'/0'/0/0
    address:             rHsMGQEkVNJmpGWs8XUBoTBiAAbwxZN5v3
    xrp_classic_address: rHsMGQEkVNJmpGWs8XUBoTBiAAbwxZN5v3
    public_key_hex:      031d68bc1a142e6766b2bdfb006ccfe135ef2e0e2e94abb5cf5c9ab6104776fbae
    private_key_hex:     90802a50aa84efb6cdb225f17c27616ea94048c179142fecf03f4712a07ea7a4

  index 1:
    path:                m/44'/144'/0'/0/1
    address:             r3AgF9mMBFtaLhKcg96weMhbbEFLZ3mx17
    public_key_hex:      038bf420b5271ada2d7479358ff98a29954cf18dc25155184aead05796da737e89
    private_key_hex:     0974b4cfe004a2e6c4364cbf3510a36a352796728d0861f6b555ed7e54a70389
```

Change branch (m/44'/144'/0'/1/i):

```
  index 0:
    path:                m/44'/144'/0'/1/0
    address:             rE8wWANXkzNgRx6noykbeXQRcfVRNoZQv4
    public_key_hex:      02852f3ccc7e31159e3e8afdf84fa919b6313197b7fd8f454139aec2a6d9306721
    private_key_hex:     ec5756c9d1abc5626e434e047f4c6afb791ffec66d5396bdbe5c119d7326240b

  index 1:
    path:                m/44'/144'/0'/1/1
    address:             rwEcK2kRntGkzULqYndeHW1Qq32zeK5fwG
    public_key_hex:      0229fbc91aa74c94fa0b0078385de4ea96bda59c5a76b8ce50a7aef86860e39321
    private_key_hex:     291b4b8ad8e50a6e9da60c64007e1d6ad346f450eda289daaf8448560535d79b
```

Litecoin P2WPKH Derivation Vectors (TV1, m/84'/2'/0')

```
Coin:        litecoin mainnet
Mnemonic:    TV1
Path:        m/84'/2'/0'
```

Receiving branch (m/84'/2'/0'/0/i):

```
  index 0:
```

```
    path:             m/84'/2'/0'/0/0
    address:          ltc1qjmxnz78nmc8nq77wuxh25n2es7rzm5c2rkk4wh
    public_key_hex:   02e49c9b9b5d0f127235dc26a0c252814c52fb333d651a946773f59d72c2da9904
    private_key_hex:  4ab54480cbbaa53d5d3ba43a3e85d221df085490e88346ad11579e17000db7bb
    private_key_wif:  T5ZCYhLqXu6EJKk2nhjvwsaLH357CisixhLGWpKXEiqWTUtzte6o

  index 1:
    path:             m/84'/2'/0'/0/1
    address:          ltc1qwlezpr3890hcp6vva9twqh27mr6edadreqvhnn
    public_key_hex:   021c1750d4a5ad543967b30e9447e50da7a5873e8be133eb25f2ce0ea5638b9d17
    private_key_hex:  61fa14b9b3d872aaa327855cc94b5a220094395d55d01319cfee32e236a26d1e
    private_key_wif:  T6LRyVtoN2JxywNyYXwk9KoMpWa34Fjino4upyy9RKs5QQmC8RSr
```

Change branch (m/84'/2'/0'/1/i):

```
  index 0:
    path:             m/84'/2'/0'/1/0
    address:          ltc1qyeljcy9v88jg8sqvnqh0m5q390xruc5r98q9yy
    public_key_hex:   029857513f0fe1dc125f219ffef22098e14c653c4bcb0a2aaeff47b3a252569f1a
    private_key_hex:  dbf6dd9d691b00d0a7f6f93605a67ddb42ba4baafcf9b0d98979b66f8a21184e
    private_key_wif:  TARZNteayzJqRiXnqKJX3h5zr6H4tx4sXvoZ21pHjAEgNc1g2MWm
```

Note: Litecoin uses HRP "ltc" for Bech32 encoding. The extended key version bytes for Litecoin P2WPKH use the zprv/zpub SLIP132 values (0x04B2430C / 0x04B24746) producing a zpub/zprv prefix identical in structure to Bitcoin P2WPKH extended keys, differentiated only by the version byte content.

Dogecoin P2PKH Derivation Vectors (TV1, m/44'/3'/0')

```
Coin:        dogecoin mainnet
Mnemonic:    TV1
Path:        m/44'/3'/0'
```

Receiving branch (m/44'/3'/0'/0/i):

```
  index 0:
    path:             m/44'/3'/0'/0/0
    address:          DBus3bamQjgJULBJtYXpEzDWQRwF5iwxgC
    public_key_hex:   02cc6b0dc33aabcf3a23643e5e2919a80c50fb3dd2129ce409bbc5f0d4643d05e0
    private_key_hex:  21f5e16d57b9b70a1625020b59a85fa9342de9c103af3dd9f7b94393a4ac2f46
    private_key_wif:  QPkeC1ZfHx3c9g7WTj9cQ8gnvk2iSAfAcbq1aVAWjNTwDAKfZUzx

  index 1:
    path:             m/44'/3'/0'/0/1
    address:          DAcDAtJRztxBHyA6D6h8du1HguyTR43Mas
    public_key_hex:   025a8ad8881f6facdc949c4a4d03257414153faea67e96acf57344660080610788
    private_key_hex:  0ce96fd14dd5bd468a0927b5016d138e06d5dfb1db64c69a1b22e79c65120b63
    private_key_wif:  QP3j5SpiwZk9k8z9kaV2S6gbrVkoPaWyfW1Wy4x1WUQoyTab9VfH
```

Change branch (m/44'/3'/0'/1/i):

```
  index 0:
    path:             m/44'/3'/0'/1/0
    address:          D7ReBLrRv12mi9pYh5HtfFLTt1PSoeAa7e
    public_key_hex:   020745c065a8b7cb03a843a57339cf9164d675a2e5010c6fbd61c09239bbb5f4df
    private_key_hex:  da27a8fcc03519980a5db5dfdceaeab7b61985e9463f3fb750a1d86b2cefd65c
    private_key_wif:  QVvh62CvvLSWYTTJXS62RdyuXnBn9adqgzJ6xrZRjpwYRvuHLCtw

  index 1:
    path:             m/44'/3'/0'/1/1
    address:          D92vUBQJjMhmczzGf7ReUswoDEZuqaNout
    public_key_hex:   02475f40b385bb486ef27171fe119a89ce9b46fa1ab0f5bc542ee0e1e1fcd78623
    private_key_hex:  6799f60741b5023781d69ef9b865714d12d99ffa2ee1c66d1edd091cab07182d
    private_key_wif:  QS61pQ4VhzRFbfLyxTDBSeztbuk2pu15bRw1Z6LAgpFzp8fHdF25
```

Note: Dogecoin uses WIF prefix 0x9E. Taproot is explicitly disabled for Dogecoin. Any implementation that attempts Taproot derivation for Dogecoin must raise a hard error rather than silently producing output with incorrect version bytes.

.arc Encryption Subsystem Vectors

The .arc encryption subsystem uses fresh CSPRNG-generated salt and nonce on every export, making individual ciphertext vectors non-reproducible without fixing the random inputs. This section instead specifies three categories of verifiable test conditions for the encryption subsystem.

Structural compliance tests (verifiable without a fixed salt):

- Export produces exactly the ARCULUS-ARC-V2 armor header on line 1.

```
2. Inner bundle after base64 decode contains all required fields:
     magic, format, version, created_at, kdf, cipher, ciphertext_b64, mac_b64

3. kdf.name == "PBKDF2", kdf.hash == "SHA-512", kdf.iterations == 1000000
   for all new exports.

4. salt length == 32 bytes after base64 decode.
   nonce length == 24 bytes after base64 decode.
   mac length == 64 bytes after base64 decode.
   ciphertext length == plaintext length (no padding; XOR preserves length).

5. Two consecutive exports of the same (mnemonic, password) pair must produce
   different ciphertext values and different MAC values with overwhelming
   probability, due to independent CSPRNG salt and nonce sampling.
```

Deterministic key derivation vector (fixed salt, no CSPRNG):

```
password:          "correct horse battery staple"
salt (32 bytes):   000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
iterations:        1000000
KDF output:        PBKDF2-HMAC-SHA512(NFKD(password).encode("utf-8"), salt, 1000000, 64)

master_key bytes 0-31 (enc_key derivation input):
  HMAC-SHA512(master_key, b"Arculus ARC v2 encryption key")
  enc_key: 3c8d5f9435d4d935f8dbcb48285ca8d578079eded9ba046a58fb4a6c6b73e6c
           fca43a93488c5bc8757310108f3eb7b05089fd84f5bc393300affeaf79d3e5f8f

  mac_key: d5d4d1a6f5f036d040e502ab0c9759ca517463f9dbd971bc0cf3a27b5a728a9
           320079532475422c1b118e53f55d33b9858342fefd7f666f570ffe5dba717b9a3

A compliant implementation must produce these exact enc_key and mac_key values
given the stated password, fixed salt, and iteration count as inputs.
```

Round-trip authentication tests:

```
1. Correct password: encrypt_v2(mnemonic, password) followed by
   decrypt_v2(bundle, password) must return the original mnemonic,
   byte-for-byte identical after NFKD normalization.

2. Wrong password: decrypt_v2(bundle, wrong_password) must raise an
   authentication error. The error message must not distinguish between wrong
   password, corrupted ciphertext, or tampered MAC.

3. Tampered MAC: flipping any single bit in mac_b64 must cause MAC
   verification failure. The implementation must call hmac.compare_digest
   or equivalent constant-time comparison; a simple == comparison is
   insufficient for security even if functionally correct for this test.

4. Tampered ciphertext: flipping any single bit in ciphertext_b64 must cause
   MAC verification failure, because ciphertext is included verbatim in the
   MAC transcript.

5. Tampered iteration count: modifying kdf.iterations in the bundle before
   decryption must cause MAC verification failure, because the iteration count
   string is included in the MAC transcript as a bound field.

6. Iteration count floor: any bundle with kdf.iterations < 600000 must be
   rejected even if the MAC would verify under that iteration count. This
   floor is a hard coded constant, not a policy flag.
```

Negative Test Vectors (Error Rejection)

The following inputs must be rejected with explicit errors and must produce no derivation output of any kind.

```
Word count errors:
  - "abandon" (1 word): rejected with word-count error
  - "abandon abandon abandon" (3 words): rejected with word-count error
  - "abandon" x11 (11 words): rejected with word-count error
  - "abandon" x13 (13 words): rejected with word-count error
  - "abandon" x24 (24 words, no checksum word): rejected - all 24 "abandon"
    words is not a valid BIP39 mnemonic; the checksum for 24 words of zeros
    requires "art" as the final word, not "abandon"

Unknown word errors:
  - "notaword abandon abandon abandon abandon abandon abandon abandon abandon
    abandon abandon about" - rejected because "notaword" is not in the BIP39
    English word list

Checksum failure:
  - "abandon abandon abandon abandon abandon abandon abandon abandon abandon
```

```
    abandon abandon abandon" (12 x "abandon") - all words are valid BIP39
    entries but the checksum encoded in "abandon" (index 0) does not match
    SHA256 of the 128-bit zero entropy. Must be rejected as checksum invalid.
    This rejection must be operationally distinguishable in error messaging from
    the unknown-word rejection above.

Invalid path components:
  - "m/44'/abc'/0'" - non-numeric component "abc", must be rejected
  - "m/2147483648'/0'/0'" - hardened index 2147483648' = 0x80000000 + 0x80000000
    overflows a 32-bit unsigned index, must be rejected
  - "m//0'" - empty path component, must be rejected

Unsupported coin/script combinations:
  - coin=dogecoin, script-type=p2tr - Taproot is explicitly disabled for
    Dogecoin, must be rejected with an explicit error before producing any output

Unsupported testnet configurations:
  - coin=litecoin, --testnet flag - Litecoin has no testnet configuration in
    the engine; must be rejected with an explicit unsupported-testnet error
```

Cross-Implementation Compliance Criterion

An implementation is considered specification-equivalent to Arculus Recovery v1.5.0 if and only if all of the following conditions are met:

1. All address strings in the Bitcoin, Ethereum, XRP, Litecoin, and Dogecoin derivation vectors above match exactly, character for character, including case (Bech32 addresses are lowercase; EIP-55 addresses are mixed case; Base58Check addresses are mixed case).

- All private_key_hex values match exactly, byte for byte.
- All public_key_hex values (compressed 33-byte form) match exactly.

4. All extended key strings (xprv, xpub, yprv, ypub, zprv, zpub) match exactly, character for character.

5. All Taproot fields (internal_public_key_hex, tweak_hex, output_public_key_hex, output_private_key_hex, output_key_parity) match exactly.

- All root fingerprint values match exactly.

7. The .arc encryption round-trip test passes: encrypt then decrypt with the same password returns the original mnemonic.

8. The .arc deterministic key derivation vector produces the exact enc_key and mac_key bytes specified above.

- All negative test vectors raise explicit errors and produce no output.

Condition 1 is necessary but not sufficient. An implementation that produces correct addresses but incorrect private keys has a bug in its scalar arithmetic that may only manifest in keys it happens to generate correctly by coincidence. All conditions must be met simultaneously.

Packaging Regression Checks

The following checks cover packaging changes that do not alter the deterministic cryptographic vectors above but can affect user-visible behavior:

1. The standalone Arculus_Recovery.html file must contain the inline jsPDF script block and must not require a network request for PDF export.

2. The packaged HTML asset under src/arculus_recovery/assets/ must match the canonical root HTML file when a release is prepared.

3. The PySide6 GUI must preserve CLI behavior: python Arculus_Recovery.py --help must remain available, and python -m compileall over Arculus_Recovery.py and src/ must pass.

4. The Tauri project must include src-tauri/capabilities/default.json and src-tauri/permissions/export.toml. These files are required for the injected WebView export bridge and native save_export command. Packaged release checks must confirm PDF, JSON, TXT, CSV, .arc, and QR PNG exports save to disk through the native bridge.